



Technical University of Munich
TUM School of Engineering and Design
Institute of Micro Technology and Medical Device Technology (MIMED)
Prof. Dr. rer. nat. Dipl.-Ing. Tim C. Lüth

Bachelor's Thesis

Development of a Self-Sensing Piezoelectrically Actuated Pendulum

Julius Gun

Matr.: 03728036

Principal Investigator	Univ.-Prof. Dr. Tim C. Lüth Institute of Micro Technology and Medical Device Technology
Supervisor	Jan Rodewald, M. Sc. Institute of Micro Technology and Medical Device Technology
Beginning	01.11.2023
Submission	31.05.2024

Abstract

Klaus Moldovan's 1993 piezoelectric pendulum, a key educational demonstrator at the Technical University of Munich, faced obsolescence from aging electronics and inaccessible Assembler software. This thesis details the design and implementation of a modern successor, overhauling its technology while preserving its educational core. The project involved replacing legacy components with a Raspberry Pi Pico microcontroller programmed in C++, a half-bridge driver with fast-switching MOSFETs, and a compact, 3D-printed enclosure that integrates the actuator with the control unit. The resulting enhanced pendulum successfully replicates all core functionalities of the original, including self-sensing impulse detection and a bounce-free catch of a suspended ball. A simplified user interface, featuring a single rotary encoder and LCD screen, significantly improves usability. This work ensures a valuable, hands-on tool continues to effectively teach complex piezoelectric phenomena to engineering students.

12 Keywords:

Self-Sensing Piezoelectrically Actuated Pendulum

Smart piezoelectric actuator

Piezoelectric pendulum

Smart piezoelectric pendulum

Piezo pendulum

Self-sensing piezo actuator

Demonstrator of the piezoelectric effect and inverse piezoelectric effect

Piezoelectric demonstration model

Piezoelectric damping

Piezoelectric feedback control

Actively damped piezoelectric system

Piezoelectric actuation pendulum

Forward to Revised Version

I have revised my submitted bachelor thesis, removing minor mistakes. I have inserted the updated version of code which has new functionalities and and now is optimized for a 4g ball. I also used Gemini 2.5 Pro Preview 05-06 for paraphrasing of my text so that the active voice is used.

If somebody wants to build a new piezo pendulum, then rather use a piezo actuator with a higher capacitance than the one used in this project and add a shunt resistor in front of it to avoid the resonance frequency (also see 2.6.1.2 Piezo Actuator, and 1.2.1.3 Characteristics of Piezo-electric Materials).

Also redesign the complete case, so that it is easier to open it (right now 4 screws need to be undone). Rather have the piezo connector outside the box and not inside so that an oscilloscope can easily be connected. It could even be desirable to have the piezo actuator mounted on a separate base, just like Moldovan 1993 did, if experiments are to be carried out.

For any questions regarding this pendulum, feel free to contact me.

Julius Gun, 07.06.2025

Acknowledgment

This thesis would not be complete without acknowledging those who offered their expert guidance and support. My profound appreciation goes to Klaus Moldovan, originator of the 1993 piezo pendulum, for his readiness to assist through extensive correspondence, which proved critical to my study. My supervisor, Jan Rodewald, deserves special thanks for his continuous support and valuable counsel throughout this endeavor.

Gratitude is also owed to Dr. Franz Irlinger, who facilitated the acquisition of critical documents and my introduction to Klaus Moldovan. My respectful acknowledgment goes to Prof. Dr. Tim C. Lüth, whose teachings ignited my fascination with electronics. The collaborative and welcoming environment provided by the team at MIMED has greatly enhanced my research experience. Acknowledgment is due to Christoph Rehekampff for his readiness to assist and his technical guidance with the electronic circuit.

Also, thanks to Andreas Haas from Physical Instruments (PI), Markus Wörl and Florian Schweidler.

Finally, I would like to thank my dear mother, Emma Weiß, for her support throughout all those years. You are wonderful.

Contents

Abstract.....	3
Acknowledgment.....	4
Contents.....	5
1. Introduction.....	6
1.1 Application.....	6
1.2 State of the Art.....	8
1.2.1 Definition of Terms.....	8
1.2.2 The Piezo Pendulum Engineered by Klaus Moldovan.....	15
1.2.3 Research about Piezoelectricity.....	19
1.2.4 Research about Piezoelectrically Actuated Pendulums.....	20
1.3 Disadvantages of the State of the Art.....	21
2. Methodology.....	23
2.1 Problem Definition.....	23
2.2 Expected Advantages and Functions of the Solution.....	23
2.3 Structure of the Solution.....	23
2.4 Process of the Solution.....	25
2.5 Delimitations of the Solution.....	26
2.6 Laboratory and Equipment Setup.....	26
2.6.1 Choice of Solution.....	26
2.6.2 Functional Model.....	37
2.6.3 Prototype.....	51
3. Design Evaluation.....	52
3.1 Modernization of the Pendulum: Comparative Hardware Analysis.....	52
3.2 Programming Language: From Assembler to Accessibility.....	53
3.3 Retention and Rationalization of the Pendulum's Features.....	54
3.4 Continuity of Core Functionalities.....	54
3.5 Simplification of Activation Procedures.....	55
3.6 Fiscal Overview.....	56
3.7 Errors in Initial Design.....	56
3.8 Summary of Results.....	57
4. Summary and Outlook.....	58
References.....	61
Table of Figures.....	63
Appendices.....	66
A Schematic.....	66
B PCB layout.....	72
C Bill Of Materials.....	74
D Flowcharts.....	78
E Code.....	79
F Drawings.....	97
G Matlab Code.....	98
H Oscilloscope Readings.....	98

1. Introduction

1.1 Application

Piezoelectric principles are pivotal in the "Mechatronic Device Technology" and "Microtechnical Sensors and Actuators" lectures at the Technical University of Munich. The piezoelectric demonstrator within this context provides students with an empirical understanding of the conversion of mechanical stress into electric charges and vice versa. Engaging with this demonstrator imparts valuable practical experience and technical knowledge essential for precise applications in advanced engineering sectors.

The application of piezoelectric effects in academic settings helps demonstrate and clarify advanced concepts. This leads to measurable educational outcomes in student engagement and understanding. This demonstrator transforms abstract textbook theories into a concrete example.

The practical deployment of piezoelectric devices spans a range of applications that benefit from high precision and quick responsiveness. Self-sensing piezoelectric ceramics offer an innovative method by combining actuation and sensation. This combination reduces physical dimensions and response time, significantly contributing to space-sensitive and finely tunable technologies.

In the medical field, the non-magnetic nature of piezo actuators is a major asset. These devices play a vital role in MRI scanners by enabling precise adjustments without interfering with the imaging process (Fischer et al., 2011; Wang et al., 2009). Piezo technology is essential for administering accurate doses in medical pumps, like those for insulin. It is also used in ultrasonic surgical tools and diagnostic devices, such as ultrasonography transducers, where electromagnetic disturbance must be avoided (Li et al., 2009). Additionally, these actuators facilitate precise fluid control in miniaturized lab-on-a-chip systems used for medical testing.

However, piezo actuators have a higher cost because of their sophisticated technology. They are also limited in their movement range, and their piezoelectric materials can be delicate, risking damage under high stress or if mishandled. Their operation typically requires high-voltage inputs, complicating the design. They are also sensitive to temperature changes, which could affect their functionality. Lastly, they exhibit varying levels of hysteresis and nonlinear behavior (Chopra and Sirohi, 2014, p. 154). The perception of piezoelectric effects varies depending on the application.

Self-sensing piezoelectric ceramics combine action and sensing in one unit. These materials eliminate the need for separate sensors, offering both cost and space benefits (Muth et al., 1995, p. 692). Traditional sensors provide precision but often result in costlier and more cumbersome setups. These setups add excess wiring and demand more component room, as seen with encoders (Liseli et al., 2020, p. 2). The compact design of self-sensing piezoelectric ceramics proves ideal for miniaturized systems and nano-positioning tasks where space is at a premium (see Liseli et al., 2020 for an overview).

In some applications, the piezoelectric effect is invisible, while in others, it can be perceived visually, acoustically, or haptically. The piezo effect (a piezo actuator subjected to force generates voltage) is visually evident in the spark of a lighter, where applied pressure creates a visible, abrupt voltage between contacts (APC International, 2024) . Additionally, the inverse effect—a piezo actuator subjected to voltage generates force—is most notably observable in devices like high-precision stages. Here, piezo elements enable movement through the inverse piezoelectric effect. This same principle of voltage generation is used in scientific settings, like producing corona discharges in mass spectrometry (Wang et al., 2015) .

In practical applications, piezoelectric actuators exemplified in the Arburg Freeformer 3D printer's printhead demonstrate controlled, nuanced operations. These piezos produce an audible click as they dispense precise droplet formations at frequencies between 60 and 200 Hz, which is crucial for high-quality 3D printing (Arburg, 2024) .

Energy harvesting represents a silent yet impactful application of piezoelectric materials. Away from the audible clicks in 3D printing, these materials discretely capture electricity from vibrations, such as those from vehicles on roads, by embedding piezoelectric elements beneath the pavement (Kaur, 2016) .

The Technical University of Munich is pioneering piezoelectric energy harvesting, as represented in Figure 1. This prototype, a joint endeavor by Prof. Johann Plank and Prof. Xin Cheng, demonstrates the instant energy yield in Joules from pedestrian movement. Future improvements could lead to powering sensors and LEDs in high-traffic areas like bridges, subway stations, and nightclubs.

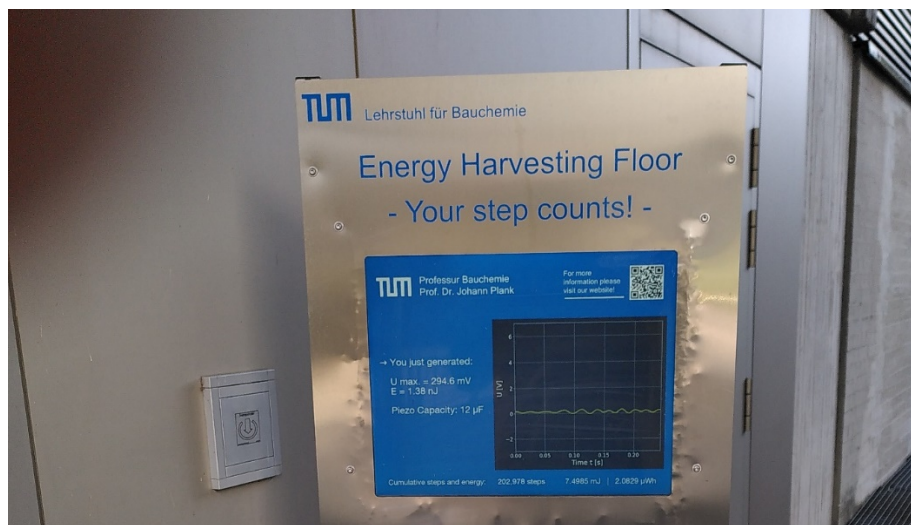


Figure 1 Energy harvesting floor at the entrance to the Chair for Construction Chemicals, Technical University of Munich.

In Rotterdam, the innovation of piezoelectric energy has been embraced in a dance club, which features a 38m² area lined with piezoelectric tiles. This novel implementation results in a noteworthy 30% reduction in energy consumption, although the initial investment stood at \$257,000 (Moussa et al., 2022, p. 4) .

The bulk of this expenditure is often on the piezoceramic material, which in many cases consists of lead zirconate titanate (PZT). With ongoing research, more cost-effective lead-free piezoceramics may emerge on the market (Shrout and Zhang, 2007). Seyfert et al., 2021 from the Technical University of Munich researched the use of piezoelectric PVDF film and its potential for energy harvesting. This study demonstrates the film's applicability in capturing energy from environmental mechanical forces.

Having reviewed some applications of piezoceramics, the next section examines the current state-of-the-art piezoelectric pendulum after some definitions.

1.2 State of the Art

This section delineates the current advancements in self-sensing piezoelectrically actuated pendulums. Piezoelectric devices operate through two fundamental mechanisms: the direct effect, where applied force is transformed into voltage detectable by specialized electronics, and the inverse effect, which involves expansion upon voltage application. Understanding these underlying principles is critical before discussing the current state-of-the-art piezoelectric pendulum.

The reader is assumed to have a foundational understanding of electronic concepts and formulae. Should further clarification or in-depth exploration be necessary, standard electronic textbooks are recommended as reference materials. Notable examples include the works of Tietze et al., 2016 and Horowitz and Hill, 2015, which serve as comprehensive resources in the field.

1.2.1 Definition of Terms

The discovery of piezoelectric materials is attributed to French physicists Jacques and Pierre Curie in 1880. They unveiled the property of specific crystalline structures, like quartz, to generate an electric charge under mechanical stress. This bidirectional conversion capability between mechanical and electrical energies marks the cornerstone of piezoelectricity. The etymology of "piezoelectricity" is rooted in the Greek term 'piezein,' which translates to press, describing the materials' electric reaction to pressure.

1.2.1.1 The Piezoelectric Effect

When piezoelectric materials are subjected to stress, they produce a voltage. This is known as the piezoelectric effect. It is observed when a piezoelectric actuator, subjected to force F [N], generates voltage V [V], captured by equation (1). This shows how a piezoelectric actuator converts mechanical pressure (force per area) into an electrical signal (voltage). This is crucial for the device's ability to detect a ball without other sensors, as I will discuss later.

$$V = \frac{d_{33} * F}{C} = \frac{Q}{C} \quad (1)$$

V	Voltage	[V]
d_{33}	Piezoelectric module	[m/V]
F	Force	[N]
C	Capacitance	[F]
Q	Electrical charge	[C]

Here, the constant d_{33} [m/V] represents the longitudinal effect - the piezoelectric charge constant along the polarization axis of the piezoelectric material, while Q [C] is the electrical charge produced, C [F] is the capacitance of the piezoelectric material. My discussion focuses on a one-dimensional scenario, considering the linear relationship between the force applied along the piezoelectric disc's polarization direction and the resulting electrical output.

The elongation $\Delta l/l$ in one dimension is proportional to the longitudinal effect d_{33} along the polarization axis of the piezoelectric material, see equation (2). This is important because it mathematically expresses the direct proportionality between the mechanical stress σ (which is force per area) experienced by the device and the electric field/response ε . This relationship further underpins the actuator's ability to convert electrical signals into mechanical pressure. Thus, a force of a ball on an actuator produces an electrical signal and vice versa, validating the piezoelectric behaviour explored in this research.

$$\frac{\Delta l}{l} = \frac{\sigma * 1}{E} + d_{33} * \varepsilon \quad (2)$$

$\Delta l/l$	elongation	[-]
σ	Mechanical stress	[N/m ²]
E	Young's modulus	[N/m ²]
d_{33}	Piezoelectric module	[m/V]
ε	Electric field	[V/m]

1.2.1.2 Inverse Piezoelectric Effect

Conversely, when a voltage is introduced to the piezoelectric material, its shape alters by extending in length Δl . This is called the inverse piezoelectric effect.

1.2.1.3 Characteristics of Piezoelectric Materials

When examining piezoceramics, it is critical to understand that the absorbable and releasable energy W_a , measured in joules (J), is calculated by formula:

$$W_a = \frac{1}{2} * C_p * V_p^2 \quad (3)$$

The capacitance and voltage range determine the energy a piezo can absorb and release; therefore, this is an important consideration when selecting a piezo actuator. Most notably, the energy quadruples when the voltage V_p is doubled.

Characterizing the piezoelectric device electrostatically, one can conceptualize it as a capacitive entity. The capacitance C_p [F] of such a device is quantitatively defined by equation (4),

$$C_p = \frac{\varepsilon_0 * \varepsilon_r * A * n}{h} = \frac{\varepsilon_0 * \varepsilon_r * \pi * r^2 * n}{h} \quad (4)$$

where ε_0 is the electrostatic constant, ε_r is the relative permittivity of the piezoelectric material, A is the area of the individual ceramic disc, r is the radius of the disc, n denotes the total number of piezoelectric ceramic discs in the stack, and h is the distance between the piezoceramic discs. From equations (3) and (4) it follows that the absorbable energy is proportional to the capacitance C_p . Notably, the capacitance is directly proportional to the square of the actuator's radius and to the number of discs, which accounts for the actuator's dimensional properties – width and length, respectively.

The function of the piezoelectric pendulum, which uses a linear piezo actuator, is the focus of my investigation. Figure 2 portrays the process of energizing the piezoelectric material by applying voltage through wires, resulting in its expansion by a length of Δl ; the material contracts to its original size upon removal of the voltage. In the depiction, the left side shows the connection to the voltage source and the grounding at the other end. Typically, the piezoelectric stack is composed of numerous layers of ceramic, often using a material known as lead zirconate titanate (PZT).

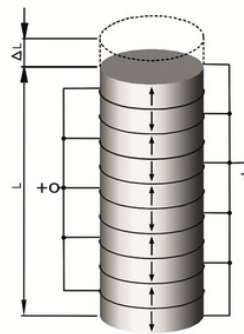


Figure 2 Structure and contacting of a stacked piezo actuator (Image used with kind permission from PI Ceramic GmbH, available at <https://www.piceramic.com/en/expertise/piezo-technology/properties-piezo-actuators/electrical-operation> [Accessed 14 May 2024])

Figure 3 shows a sectional view of a linear piezo actuator. Its multilayered structure increases its capacitance according to equation (4). A hardened tip prevents damage to the actuator's head. For secure attachment, the mounting plate has a thread to enable screwing it into position.

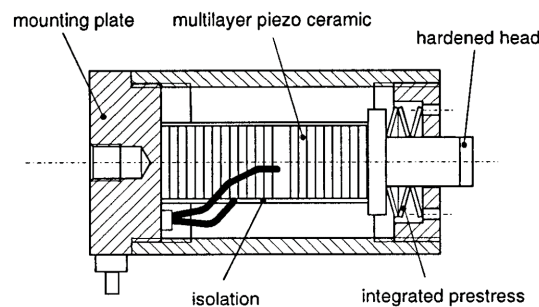


Figure 3 Linear piezoelectric actuator in cut view. The ball hits the hardened head on the right side. (Image used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 699))

As depicted in Figure 4, the piezo actuator's behavior can be likened to that of a capacitor (C_p). It is critical to recognize that the piezo and parasitic resistance form a low-pass filter, with a cutoff frequency f_p given by equation 5:

$$f_p = \frac{1}{2\pi R_p C_p} \quad (5)$$

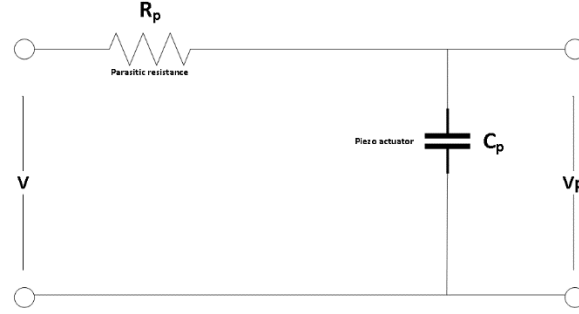


Figure 4 Schematic of parasitic resistance and capacitance of piezo actuator, forming a low-pass filter.

A design must consider the piezoelectric actuator's capacitance and the parasitic resistance. In a numerical simulation of a piezo ceramic at impact by a steel ball by Schweidler, 2004, p. 42, post-impact oscillations of the piezoceramic endure for approximately 5ms. This phenomenon is visually represented in Figure 5, capturing the attenuated response past the moment of impact. The actuator is vibrating, as it is subjected to both tensile and compressive stress in a fast-reciprocating manner.

The frequency with which the piezo oscillates in this process is dependent on equation (5). It would be desirable to have a cutoff frequency f_{3db} ¹ from below the resonance frequency to avoid destroying the piezo. Otherwise, it could oscillate so severely at its resonance frequency that the many layers of glued piezoceramic would separate (Moldovan, 1994a, p. 22), effectively destroying the device. Achieving a cutoff frequency below the resonance frequency can be done by using a piezo with a high capacitance C_p or by increasing R_p , for example, by adding a resistor in series with the piezo.

¹ The cutoff frequency f_{3db} is the point of a filter where the attenuation of the signal is reduced to 0.707 times the maximum gain.

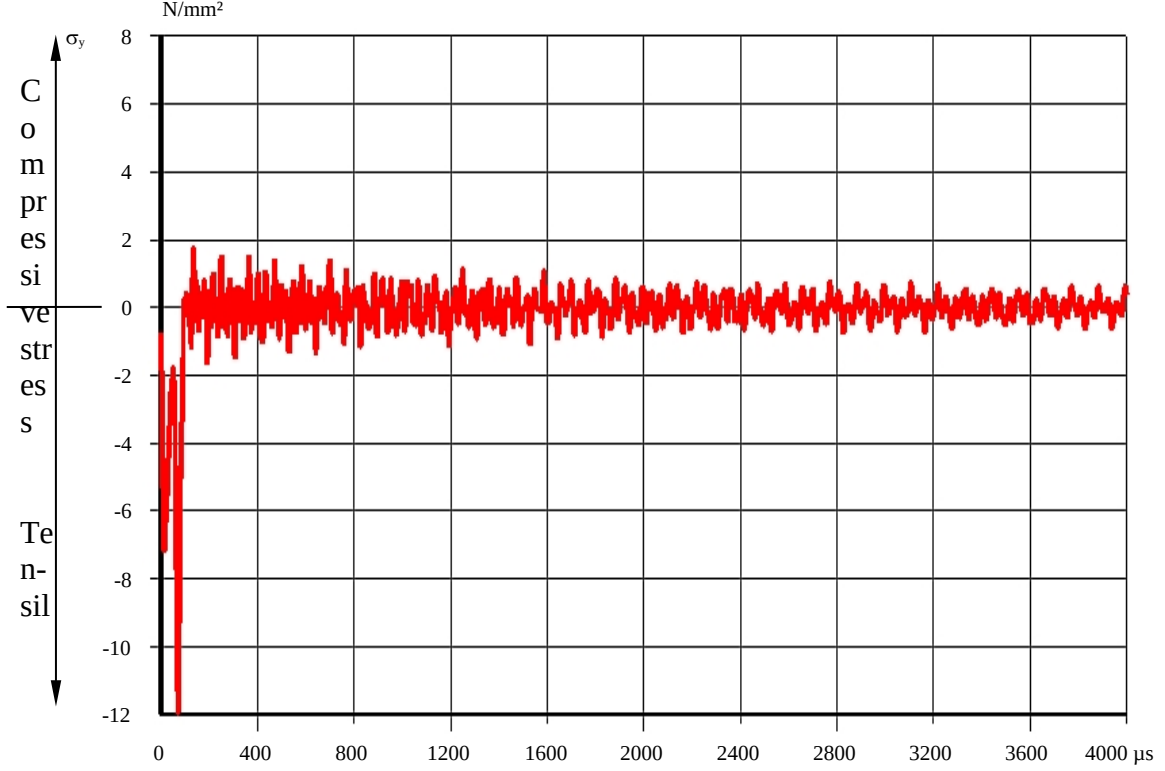


Figure 5 Simulated stress of the piezo actuator on in contact with a ball in the axial direction (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 42))

The voltage across the piezo is:

$$V_p = R_p * I \quad (6)$$

where R_p is the parasitic resistance before the piezo and I is the current across it.

The dynamic voltage at charging of the piezo $V_p(t)$ when charging the piezo is given by equation (7), and when discharging by equation (8), where t represents the time. Figure 6 shows a plot with exemplary values. This resemblance to capacitor voltage characteristics is expected because the piezoelectric device has its own capacitance, as per equation (4). Upon activation, the piezoelectric device's voltage rises to a peak value V , and upon deactivation, it decreases back to 0 volts.

$$V_p(t) = V * e^{\frac{-t}{R_p * C_p}} \quad (7)$$

$$V_p(t) = V * \left(1 - e^{\frac{-t}{R_p * C_p}} \right) \quad (8)$$

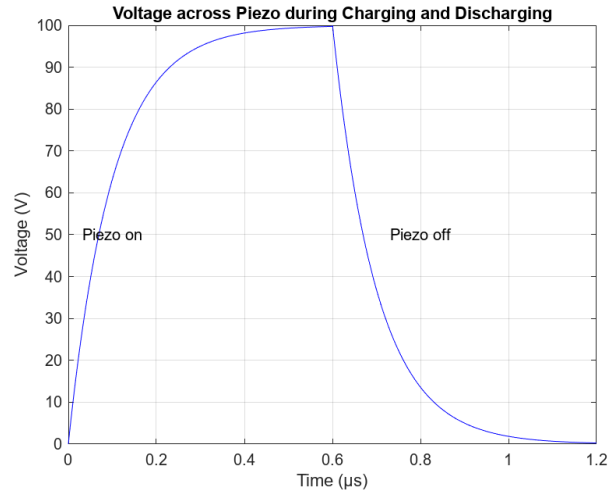


Figure 6 Voltage across piezo during charging and discharging with exemplary values.

1.2.1.4 Test Setup

Figure 7 shows a schematic of the demonstrator's test setup, consisting of a piezo actuator and pendulum. A ball with a mass m_b is suspended by a string from above. Gravity is signified by g (9.81 m/s^2), h is the height, and φ is the angle at which the ball is at the moment. When the piezo is turned on, it charges. Consequently, electrical energy (equation (3)) is turned into kinetic energy (equation (9)). The ball is shot away with a velocity v_b and flies to a certain height, h . There, the potential energy is at a maximum, as denoted in equation (10).

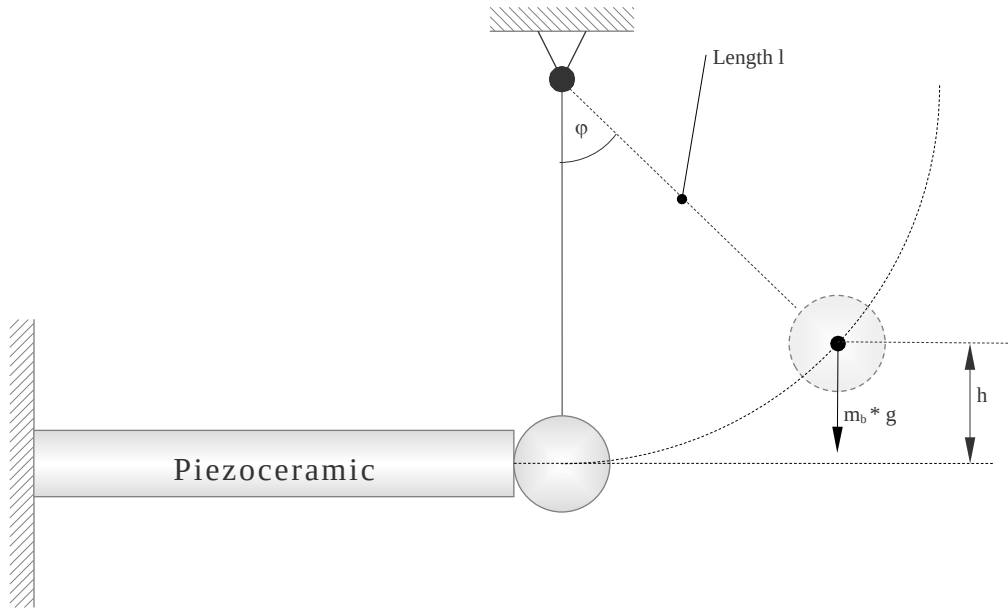


Figure 7 Schematic of piezo actuator and pendulum (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 23))

At the lowest point ($h = 0\text{m}$) the kinetic energy E_k is highest and is given by equation (9).

$$E_k = \frac{1}{2} * m_b * v_b^2 \quad (9)$$

The potential energy E_p is given by equation 10.:

$$E_p = m_b * g * h \quad (10)$$

In an ideal setting, E_p equals according to energy conservation principles, and so the resulting velocity v_b is given by:

$$v_b = \sqrt{2 * g * h} \quad (11)$$

This equation allows calculation of the ball's velocity at its lowest point, just before contacting the actuator, simply by knowing the height to which it was shot. In this ideal setting, the velocity with which the ball is shot is approximately the same as the final velocity when it hits the piezo again. The consistency in initial and return velocities is essential for determining the correct delay for discharging the piezo's stored energy. This allows for a bounce-free catch of the ball, which I will discuss in the next section.

1.2.1.5 Bounce-free Catching of a Ball

For the ball to adhere instantly, the actuator must absorb all the collision's energy. As the ball hits the expanded ceramic material, it applies a compressive force, altering the electrical charge and causing a detectable voltage shift. The electronics of the system then release the energy from the ceramic at the precise moment to ensure the ball remains in place (Muth et al., 1995, p. 692). Because the initial velocity is approximately the same as the final velocity when the ball returns, only the correct time delay to discharge the piezo needs to be found.

Figure 8 shows the force-time curve of a ball on impact with the piezo pendulum with exemplary values (Schweidler, 2004, p. 12). The sketches above the curve show the position of the ball at each point in time. $F_b(t)$ represents the curve of the force of the ball over time. At t_1 the ball hits the piezo and starts to compress. Then the force rises until it reaches its maximum force F_{max} at t_2 when the ball is in the most compressed state. After this, the force falls again, reaching the bottom end at t_3 when the ball leaves the actuator again and flies off (this is not bounce-free catching).

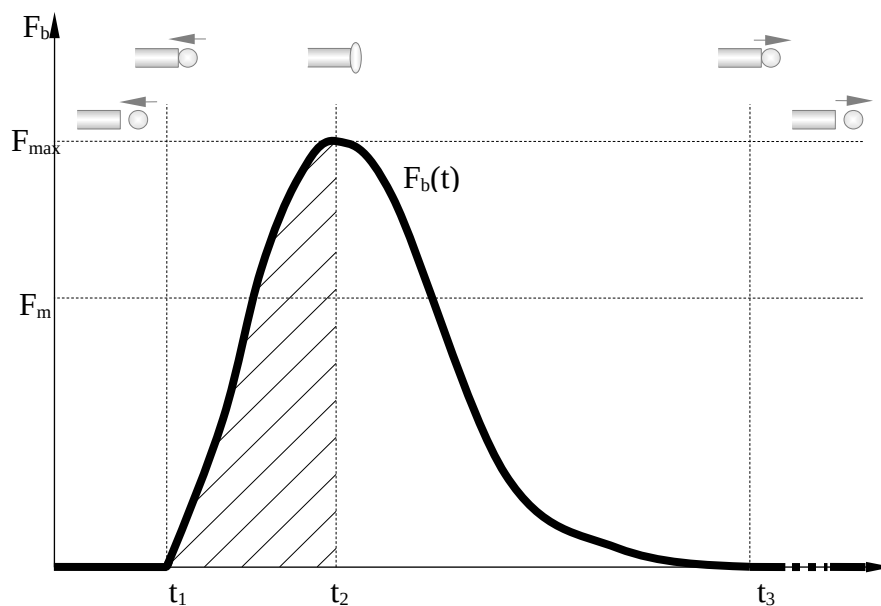


Figure 8 The characteristic curve of the force F_b during the first contact of the ball with the piezoceramic. The sketches above the curve show the position of the ball at each point in time. (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 12))

The average force F_m [N] over the impact duration is equal to:

$$F_m = \frac{1}{t_2 - t_1} \int_{t_1}^{t_2} F_b * dt = \frac{1}{t_2 - t_1} (p_2 - p_1) \quad (12)$$

The momentum p [N/s] is defined as:

$$p = m_b * v_b \quad (13)$$

The impulse Δp [N * s], the change in an object's momentum when a force is applied over time, is the area under the curve in Figure 8 and is given by the following equation:

$$\Delta p = p_2 - p_1 = m_b * (v_2 - v_1) \quad (14)$$

When the piezoelectric actuator in the demonstrator detects the incoming ball, its task is to generate enough counter-impulse to stop the ball, effectively making the final momentum of the ball zero and, therefore, preventing it from bouncing off. The piezoelectric actuator must produce a force for a specific duration to create a counter-impulse equal and opposite to the ball's momentum. It achieves this by discharging the piezo at the correct moment. This effect permits a ball to "stick" to the actuator upon impact without bouncing back, as noted by Muth et al., 1995, p. 692 . It gives the illusion that the ball is magnetically attached to the actuator.

The timing of this force is critical - if released too late or too early, the desired effect of "sticking" will not be achieved. This delay time can be determined experimentally. Alternatively, a sophisticated algorithm could be developed. Accurate timing for a bounce-free catch may be determined through real-time voltage monitoring at the point of contact. This requires a calculation that accounts for initial voltage, ball mass, electronic response time, and the piezoelectric material's delay properties.

1.2.2 The Piezo Pendulum Engineered by Klaus Moldovan

Currently, the piezo pendulum developed by Klaus Moldovan in 1993 at the Technical University of Munich represents the state of the art for academic demonstrations (Moldovan, 1993) . No other similar piezo pendulum demonstrator is known to the author. Figure 9 depicts the design of Moldovan's device: A ball is attached to a string (1) so that the tip of the piezo actuator (2) touches it when the ball is at its lowest point. The piezo is screwed to a piezo holder (3), which itself is connected to the piezo base (4). Cable connection (5) to the control unit permit the piezo to be energized. The control unit, incl. display (6) and buttons (7), enable the user to choose functions. Extensive documentation of the pendulum by Moldovan allows for an in-depth understanding of the system (Moldovan, 1994a, 1994b, 1994c) .

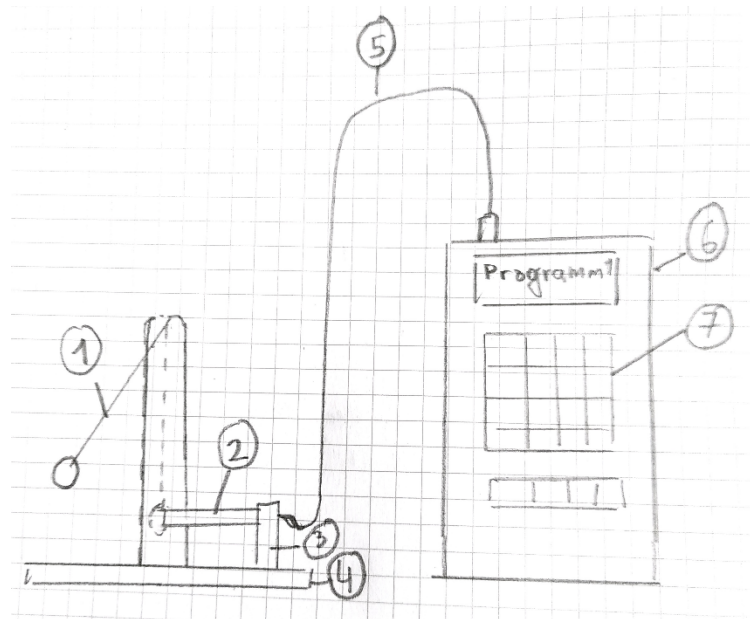
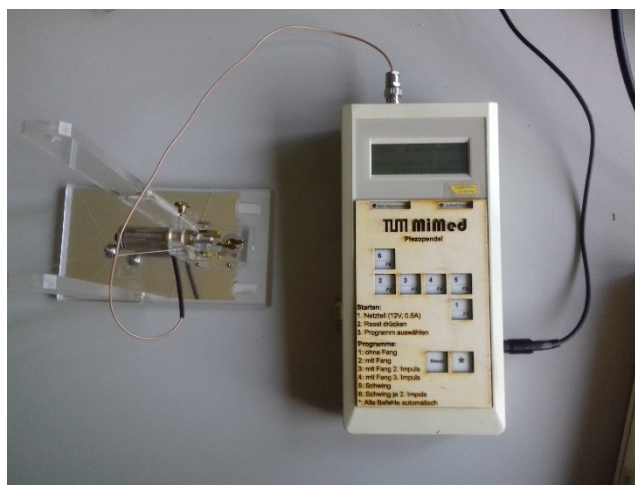


Figure 9 Representation of the piezo pendulum engineered by Klaus Moldovan. Ball on a string (1), piezo actuator (2), piezo holder (3), piezo base (4), cable connection to the control unit (5), control unit incl. display (6) and buttons (7)

Figure 10 a) shows a photo of the pendulum developed by Moldovan. A wooden panel was added later for simplicity. The control unit without the wooden panel is shown in Figure 10 b). There are 16 buttons to operate the unit.



a)



b)

Figure 10 a) Piezo pendulum engineered by Klaus Moldovan with later wooden panel modification
Right: Control of the piezo pendulum engineered by Klaus Moldovan without wooden panel

Control Unit

Figure 11 and Figure 12 provide insight into the innovative use of power electronics. Moldovan's choice to implement a flyback converter for piezo voltage management and two forward converters for controlling the two Metal Oxide Field Effect Transistors (MOSFETs)—a type of switch—is of particular interest. These three transformers were custom-made for this purpose (Moldovan, 1993, pp. 29–34). This level of craftsmanship underscores the custom-designed nature of the pendulum's electronic system. Additionally, the incorporation of a rechargeable battery alongside the 12V power supply enhanced the device's portability.

A full explanation of the symbols displayed and terms being used can be found in the referenced materials or standard electronic textbooks, such as those provided by Tietze et al., 2016 and Horowitz and Hill, 2015 .

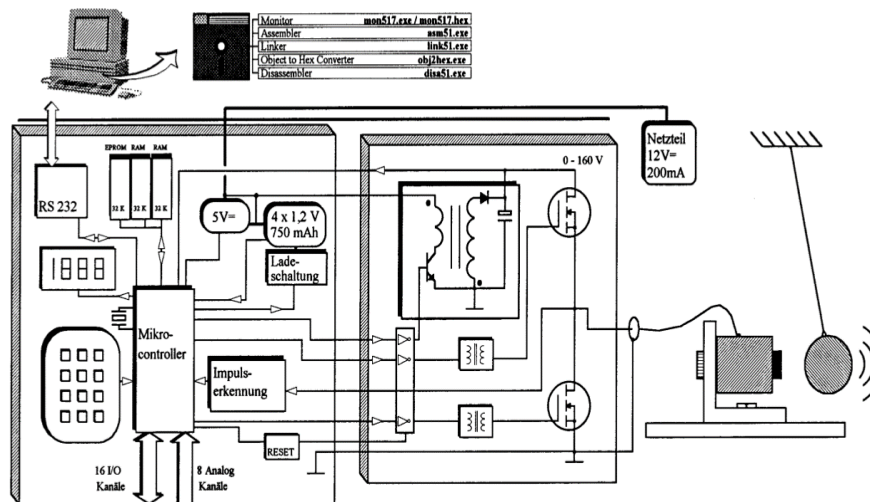


Figure 11 Basic view of the parts in the piezo pendulum engineered by Klaus Moldovan (Image used with kind permission from Klaus Moldovan (Moldovan, 1993, p. 8))

Figure 12 offers a more detailed view of the hardware configuration within the piezo pendulum engineered by Klaus Moldovan. Notably, Moldovan's design incorporates two MOSFETs connected in parallel. This connection increases the maximum current the piezo receives. He also implemented a hardware oscillator and counter circuit crucial for controlling operational delay times. This configuration ensures exact timing for the pendulum ball's launch and capture, which is pivotal for system accuracy, as discussed in section 1.2.1.5. Such precision enables timing adjustments to within 55.6 nanoseconds (Moldovan, 1994a, p. 9) .

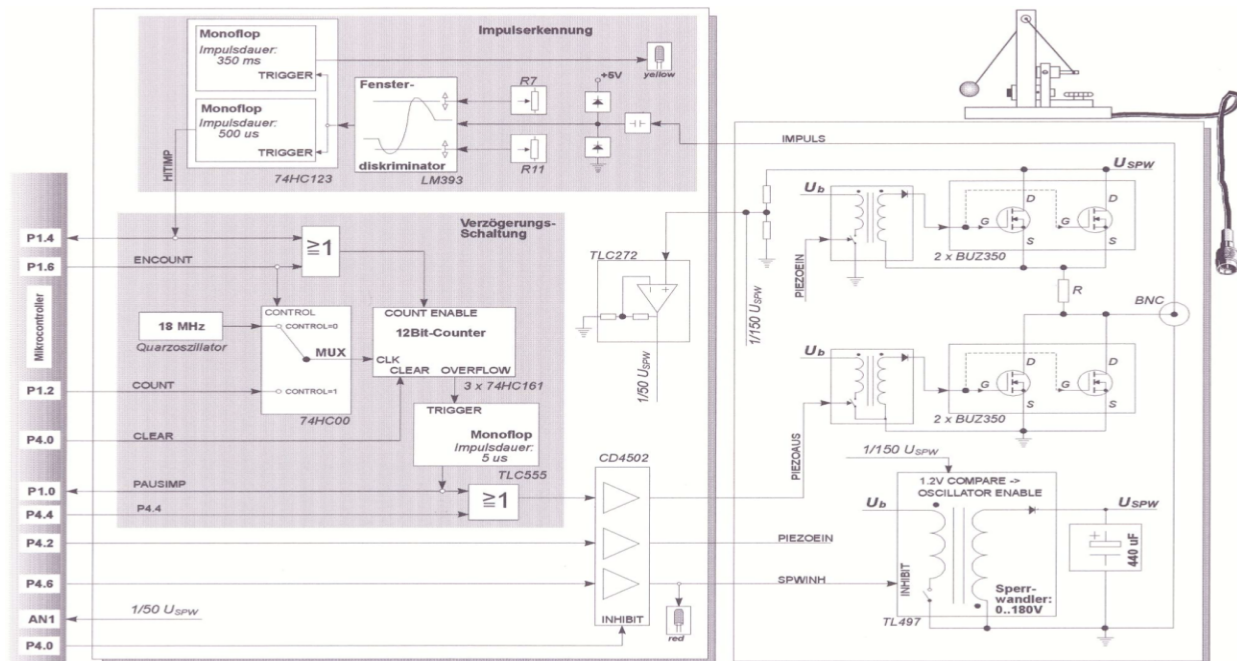


Figure 12 Hardware structure of the piezo pendulum engineered by Klaus Moldovan (Image used with kind permission from Klaus Moldovan (Moldovan, 1994a, p. 3))

Functions of the Control Unit of Moldovan

The control unit's functions, listed from 1 to 7, illustrate the system's versatility. The system was carefully designed to store parameters in an electrically erasable programmable read-only memory (EEPROM).

The functions of the control unit are:

1. *Shoot*: Launches the pendulum ball.
2. *Shoot and Catch after First Impulse*: Catches the ball as it returns.
3. *Shoot and Catch after Second Impulse*: Captures the ball after a second impulse.
4. *Shoot and Catch after Third Impulse*: Secures the ball post-third impulse.
5. *Resonance*: Propels the ball upon immediate impulse detection.
6. *Resonance after Second Impulse*: Projects the ball after the second impulse is detected.
7. *All Functions*: Cycles through all modes sequentially.

Also, there is a mode of simultaneous operation. When linked via RS232, two control units can operate two pendulums in alternation.

A helper function in the pendulum assists in finding optimal delays, as shown in Figure 13. The correct delay for discharging the piezo is very important for a bounce-free catch, as discussed in the section 1.2.1.4. Each catch mode necessitates specific delays; voltage changes require delay recalibration for all functions, and this helper function speeds up the process. The ball is shot periodically in this mode. The delay time to discharge the piezo is incremented or decremented in steps according to the user input. It is not fully automatic; it needs operator confirmation of ball catch but refines delays in steps to enhance performance.

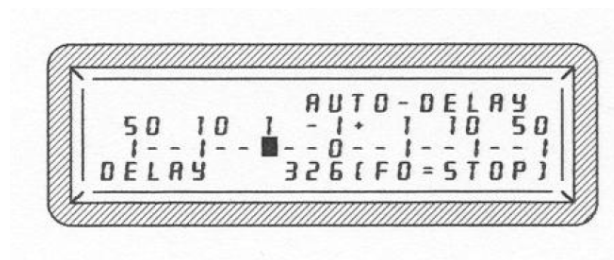


Figure 13 Automatic time delay configuration of the piezo pendulum engineered by Klaus Moldovan (Image used with kind permission from Klaus Moldovan (Moldovan, 1994a, p. 11))

The remarkable durability of Klaus Moldovan's pendulum, with an operational lifespan of 30 years, can likely be attributed to the strategic incorporation of high-quality digital components. Analog components, which are often more susceptible to inaccuracies, were judiciously employed only when absolutely essential.

1.2.3 Research about Piezoelectricity

Having examined the state-of-the-art pedagogic demonstrator, attention now turns to contemporary research in piezoelectricity and the concept of self-sensing actuators powered by piezoelectricity. A self-sensing piezo device uses piezoelectric ceramics for both actuation and detection. This eliminates the need for an external sensor to determine, for example, position (Dosch et al., 1992, p. 167) . Expanding the knowledge on piezoelectric materials, Dosch et al., 1992 showcased that a piezoelectric element could simultaneously serve as an actuator and sensor. They proposed this method as an efficient vibration damping mechanism in a cantilever beam. This integration reportedly decreased the system's stabilization time by a factor of ten compared to systems not using piezoelectric technology.

In another study, it was shown that it is possible to recover the energy stored in a linear piezo actuator during the discharge phase and, with this, to estimate the piezo position with an error of less than 5% (Chiaberge et al., 2009, p. 8) . The system was developed for fuel injection in the cylinder combustion chamber.

Despite their typically limited range of motion, piezo actuators' displacement can be significantly augmented through the application of elevated voltages or lever amplification mechanisms. Li et al., 2024, p. 55 reported achieving a 558.3 μm displacement using a lever mechanism at 150V, although the resulting motion exhibits nonlinearity. To address this, the researchers devised a control algorithm tailored to the multistage amplifying mechanism's complex hysteresis behaviour.

At the Technical University of Munich, significant research on piezoelectric ceramics has been carried out within the Institute of Microtechnology and Medical Device Technology (MIMED), as well as its predecessor, the Institute of Microfabrication Technology.

For example, in the realm of printing technology, Rumschoettel et al. crafted a predictive model for a piezoelectric drop-on-demand (DoD) printhead (Rumschoettel et al., 2014; Rumschoettel et al., 2015) . Their model forecasts key droplet attributes, facilitating swifter printing speeds and enhanced precision in droplet placement. Furthermore, Kagerer developed a piezo inkjet drop-on-demand experimentation platform manufactured with rapid prototyping techniques. (Kagerer et al., 2012) . He further showed a cheap and fast way to make unique models of piezoelectrically operated microdroplet generators (Kagerer et al., 2012; Kagerer, 2016) .

1.2.4 Research about Piezoelectrically Actuated Pendulums

Heider, 1976 , demonstrated the effective use of piezoceramics in dot matrix printers. His research focused on the capture process of a rapidly activated print element. Heider's work on dot matrix printers is a foundational study of piezoceramics handling fast, successive impacts. This process parallels the ball being stopped without rebounding in the model I developed. For visual reference, Figure 14 shows one of Heider's experimental setups Figure 14 (Heider, 1976, p. 15) , which resembles my demonstrator's setup shown in Figure 7.

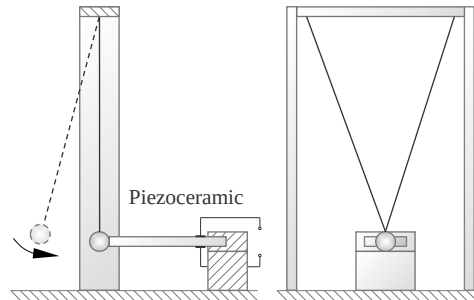


Figure 14 Experimental setup of the piezo pendulum according to Heider (Heider, 1976, p. 15) (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 5)).

To validate the findings by Heider, 1976 , Florian Schweidler (2004) conducted numerical simulations to analyze stress within a piezoceramic pendulum, using the finite element method (FEM). Through ANSYS, a software dedicated to engineering simulations, he mapped the pressure wave propagation within the piezoceramic upon collision with a steel ball. Figure 15 in his publication presents a three-dimensional model depicting the sizes of a piezoceramic (left) and a steel ball (right) during impact.

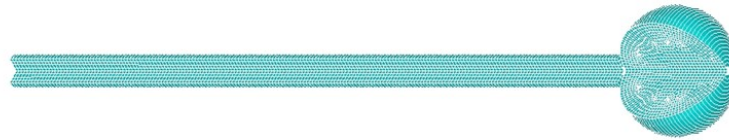


Figure 15 Three-dimensional representation of piezoceramic(left) and steel ball (right) in the correct size ratio (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 28))

Figure 16 displays one of Schweidler's simulation that captures the stress experienced by the piezoceramic at the moment of impact with the steel ball. At this instant, the compressive stress peaks at the piezoceramic's free end, located on the right side (illustrated in red).



Figure 16 Two-dimensional representation of stress in the piezoceramic at impact of steel ball. The compressive stress at the free end (right) reaches its first maximum. The ball is (approximately) stationary in this moment. (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 32))

The stress curves observed from Schweidler's analysis show a considerable match with the experimental stress curves Heider demonstrated in 1976 . This comparison offers a comprehensive examination of the stress response in a piezoceramic subjected to impact by a steel ball. The congruence between the simulated stress distribution and Heider's data strengthens the understanding of the piezoelectric effect in dynamic situations. Together, the analyses affirm the foundational design concepts of the device and underscore the viability of piezoelectric applications in practical settings.

Muth et al., 1995 further used the same experimental setup in Figure 7 to evaluate the ball's velocity in correlation with its mass (Figure 17 a)). They deduced the velocity from the highest point of the ball's post-launch trajectory, using equation (11). Measuring the electrical energy, they calculated the electromechanical efficiency as a function of the steel ball mass (Figure 17 b)). Most notably, the most efficient ball in their setup weighs approximately 24g, achieving a small electromechanical efficiency of 4.8%, while a 3g ball had an efficiency of 1.8%. Leveraging insights into bounce-free catching of the ball, the researchers explored the use of piezo actuators in active damping within optical scanners (Muth et al., 1995, p. 701) .

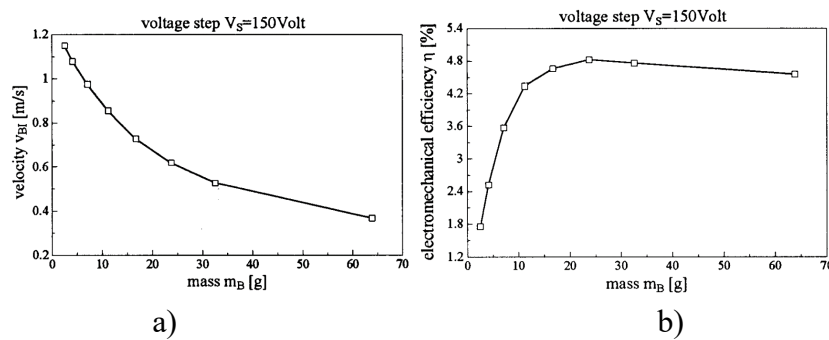


Figure 17 a) Velocity v_{BI} of the steel ball as a function of its mass m_B (Image used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 698)); b) Electromechanical efficiency η as a function of the steel ball mass m_B (Image used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 698))

1.3 Disadvantages of the State of the Art

With three decades of service, the pendulum is approaching the end of its operational lifespan. Given its age, mechanical and electrical malfunctions are increasingly likely. Future repairs may be challenging because of obsolete parts or design complexity. Moreover, the existing apparatus presents logistical challenges: As illustrated in Figure 9, the piezo actuator (2) is connected to the controller (6) via a cable (5), creating a vulnerability. This design leaves the system at risk of cable damage, which could easily occur through accidental mishandling or neglect.

The original documentation for the pendulum was misplaced for a period, presenting a hurdle for new operators unfamiliar with the system. To mitigate this challenge, a wooden panel was added to the control unit. Illustrated in Figure 10 a), this panel distinctly identifies each button and provides direct instructions, improving user navigation and simplifying the operation process. Nonetheless, users must manually initiate the reset button upon system startup, a step that persists despite enhancements.

Additionally, the software for Klaus Moldovan's piezo pendulum was written in Assembler—a common choice in 1993 because few programming languages were available for microcontroller development. While Assembler is a fast-executing language, its low-level nature, akin to machine code, renders code modifications quite challenging when compared to more contemporary languages. The software development was not only complex and time-consuming but also demanded specialized expertise. Each alteration required opening the device, physically removing the EPROM chip, and reprogramming it with an EPROM programmer. This labor-intensive procedure added a layer of difficulty to maintaining and updating the pendulum's functionalities.

Besides the difficulty of changing the software, future failures are possible, and repairs would be complicated.

Figure 18 reveals the interior of the piezo pendulum engineered by Klaus Moldovan. While many components are socketed for easy replacement, a significant obstacle exists: the discontinuation of certain components. For example, the Infineon BUZ350 MOSFET, integral to the device, is no longer in production, complicating maintenance and potentially rendering future repairs infeasible or at best, problematic. Contemporary components may share the same form factor, and with diligent searching, suitable replacements might be found. However, this does not guarantee ease of repair given the obsolescence of these essential parts.

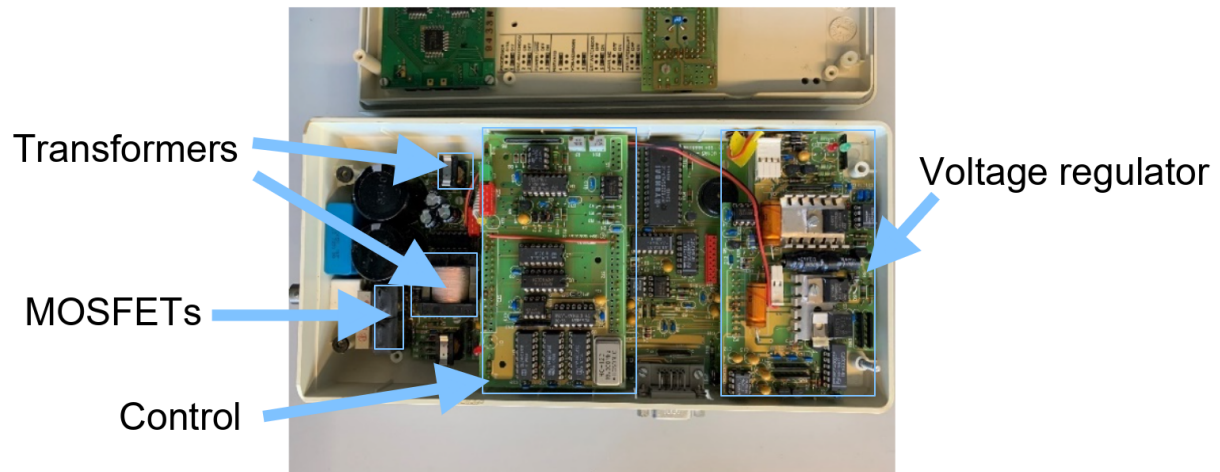


Figure 18 Piezo control of Moldovan from the inside

2. Methodology

2.1 Problem Definition

In this thesis, I aim to create an improved piezoelectric pendulum for educational demonstrations at Munich's Technical University, particularly for courses on Mechatronic Device Technology and Microtechnical Sensors and Actuators. The objective is to upgrade the pendulum's electronics and replace Assembler with a user-friendly programming language, without losing essential features. Additionally, the project intends to simplify the operational process by reducing the number of steps required for activation of functions. Finally, I will evaluate whether the new design maintains the foundational features of Klaus Moldovan's piezo pendulum.

2.2 Expected Advantages and Functions of the Solution

Modern programming languages like C++ or Python, with their more accessible syntax and rich libraries, are expected to offer significant benefits in the development and maintenance of the pendulum system, ensuring its longevity and adaptability. Adopting such languages for programming the pendulum system would also allow for more agile responses to changes and potential issues, because the software is easier to update.

Interaction with this demonstrator will afford students practical insight. Their academic growth will intersect with the application of piezoelectricity in precision engineering, medical technology, and microsystems. The adoption of this application-oriented educational method enhances the learning experience significantly. It also imparts crucial skills, preparing students to emerge as visionary engineers, innovators, and inventors in the ever-evolving domains of mechatronics and microtechnology.

The pendulum's upgraded design aims for enhanced usability, functionality, and simple maintenance. It also supports long-term financial viability by reducing the need for specialized expertise or equipment for upkeep.

2.3 Structure of the Solution

Figure 19 a) shows a basic diagram of my novel pendulum system, capturing its essential innovation. It serves as an initial visual representation of how the actuator interacts with the controller. A ball is suspended from a cord (1), and a piezo actuator (2) is capable of rapidly launching and catching the ball. This actuator is mounted on a holder (3) and its cable is invisibly connected to the control unit (4). This design leaves the system not at risk of cable damage, which could easily occur through accidental mishandling or neglect. Finally, an LCD display (5) informs the user with relevant data, e.g. of the available functions, and a push-button encoder (6) allows user interaction.

The GUI uses a minimalist design, promoting an intuitive interface with a single encoder and a pushbutton. Such a configuration streamlines interaction by focusing on simplicity and mitigating potential confusion from excess controls. The user-friendly design follows Norman's (2013) principles, advocating for simplicity, reduced cognitive load, and easily reachable functions. Such adherence ensures efficient, precise interaction without requiring extensive guidance.

The GUI forgoes an On-switch, activating instantly with power cord connection. Functions are readily accessible, with navigation via encoder and selection using the button, prioritizing straightforward operation. Figure 19 b) provides a basic visual of the menu. A green text display indicates the device is ready. The voltage is shown in blue on the right.

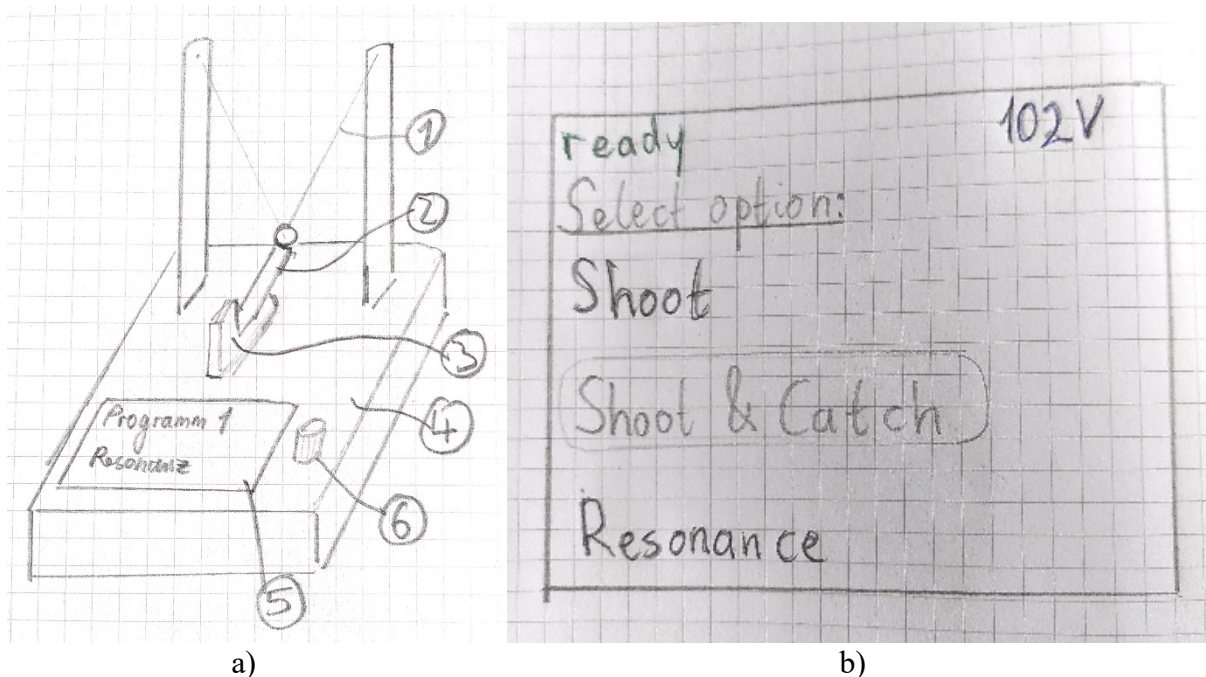


Figure 19 a) Concept of new pendulum. Ball on string (1), piezo actuator (2), piezo holder (3), piezo control unit(4), LCD display (5), encoder with push-functionality (6); b) Basic structure of the GUI

2.4 Process of the Solution

Figure 20 clarifies the interaction between the control unit, the actuator, and sensors. The control unit uses the piezo as both a sensor and an actuator. The process involves the interaction with the ball. When the control unit charges the piezo, the ball is pushed away or caught. This is the piezo acting as an actuator. Once a returning ball touches the piezo, it alters its electric charge and causes a voltage shift, which is then detected by the control unit. Users can view data and system commands on a display and manipulate the system by selecting options via an encoder with inherent push capability.

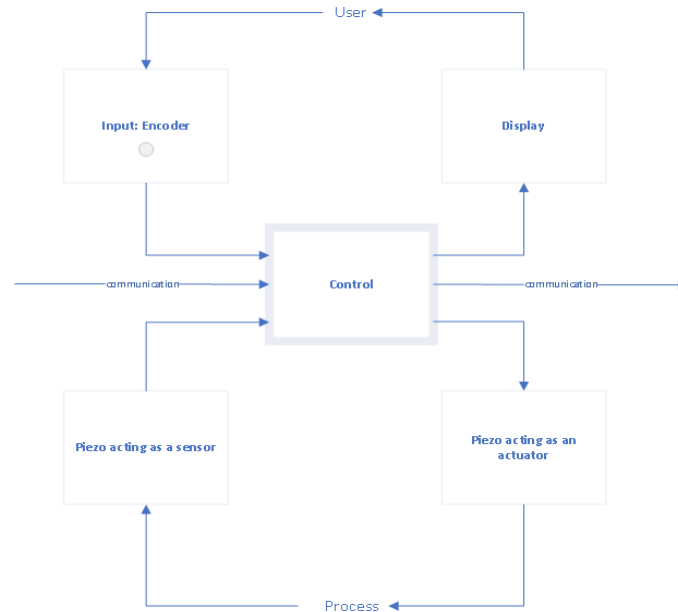


Figure 20 Structure of communication between control unit and the actuator and sensors

Figure 21 presents a streamlined flowchart detailing user interactions within the menu framework. Upon device initialization, a menu with multiple selections appears. A user's choice dictates the program's subsequent operation, including options such as "Play all functions," "Push," "Push and Catch after n Bounces" (with n being 1, 2, or 3), and "Resonance after n bounces." After completing a selected activity, the user can choose another operation; however, this subsequent selection process is not depicted because of spatial constraints.

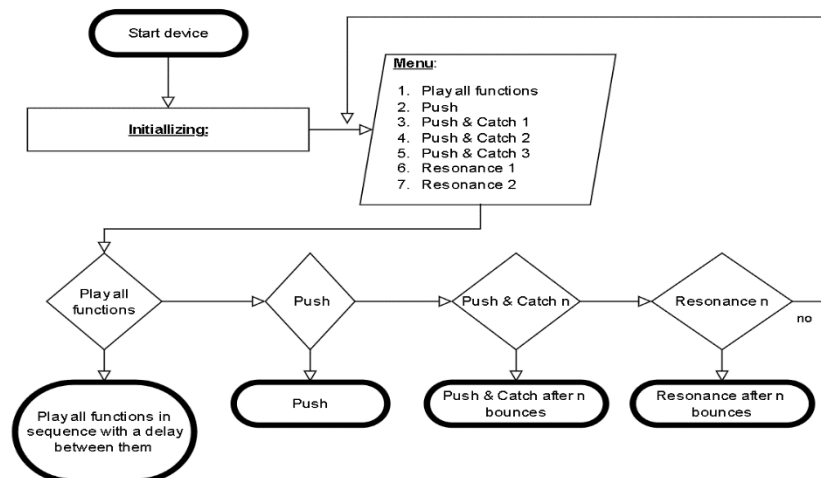


Figure 21 Flow chart of simplified menu selection process

Figure 22 shows the process of the simplified program for catching a ball bounce-free. Once started, the device needs to initialize. Then it is idle. If the user selects the function for shooting and catching the ball in the menu, it is executed. The piezo charges, the ball flies away and re-

turns. Once the ball hits the piezo, it is detected. After a catching delay (explained in the section 1.2.1.4), the piezo discharges, and the ball is caught bounce-free. Then the device is idle again, until the user selects the function again.

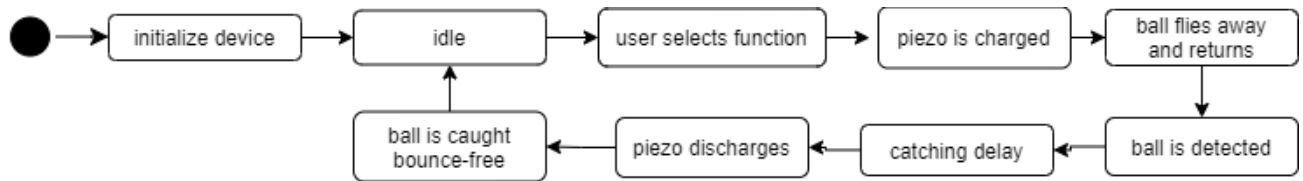


Figure 22 Activity diagram of a simplified program for catching a ball bounce-free

2.5 Delimitations of the Solution

The proposed solution is distinguished from the state-of-the-art pendulum by several distinct features. Specifically, it will undergo a full renovation that includes the integration of modern electronics and the adoption of a new microcontroller. Additionally, it will use a comprehensive programming language and feature a straightforward GUI design, distinguishing it from the existing state-of-the-art pendulum.

2.6 Laboratory and Equipment Setup

Section 2.6 outlines the basic components and structures of my lab setup so that others can accurately replicate it. This section explains the rationale behind selecting specific components and methods used in my solution, briefly touching on critical decision-making processes. Detailed descriptions of the software will not be provided, but all algorithms critical to the operation will be briefly outlined, along with their relevant parameters.

Electrical Hazard from Experimental Setup

The principal risk associated with this research is potential electric shock from the current used to power the piezoelectric actuator. It is of paramount importance to exercise caution when using tools such as oscilloscopes or multimeters on powered circuits, as contact with high-voltage components can result in injury. It is advisable to work in pairs when dealing with live wires to provide assistance in the event of an emergency. Only individuals with specialized training should be permitted to handle high-voltage systems, and regular safety training updates are essential to maintain a secure environment.

2.6.1 Choice of Solution

Section 2.6.1 presents an overview of the systematic method for selecting the most effective configuration for a piezoelectric pendulum. The search for an ideal solution includes the analysis of critical components and techniques, including the selection of an appropriate microcontroller and piezo power source, as well as the determination of efficient piezo actuation and switching methods. This overview includes the examination of MOSFET control, the comparison between bootstrap circuits and isolated gate drivers, the decision to exclude an amplifier, the methods of impulse detection and considerations for casing design. Each of these subtopics is discussed in detail to provide a complete understanding of the decision-making process.

2.6.1.1 Microcontroller

The selected microcontroller must support a comprehensive programming language to accommodate the intricate functions required by the project. It is also essential for the microcontroller to possess fast hardware timers which are crucial for managing time-sensitive interruptions. Specifically, the detection of the returning ball by the piezo element should trigger a fast response from the microcontroller, pausing its current task to initiate a program designed for precise catching without bounce. Comprehensive documentation of the microcontroller is also important to facilitate the accurate programming of these critical time delays.

Additionally, the specification that the microcontroller should be compact was considered critical to maintain the convenience of the demonstrator. Moreover, the requirement for a minimum of 20 General Purpose Input/Output (GPIO) pins was deemed essential for my configuration. These pins are indispensable for enabling a variety of functions critical to the system's operation, such as controlling the piezo activation circuitry, voltage regulation, interfacing with a display, activating LEDs for power and signal detection, and linking an encoder and a push button.

The selection process for an appropriate embedded system platform entailed an analysis of crucial features, encapsulated in **Table 1**, which presents data on five prominent devices. Factors such as timer speed, memory capacity, and development environments were compared.

Because of their size, the Arduino Due and BeagleBone Black were excluded from consideration, as the demonstrator should not have overly large dimensions. Connecting a microcontroller with either a complex programmable logic device (CPLD) or a field-programmable gate array (FPGA) offers a potential solution for managing time-sensitive functions, using their enhanced processing speeds. However, modifying their programs typically demands more specialized knowledge, which constitutes a barrier to their implementation in this project given the requirement for ease of use and adaptability. This complexity was a determining factor in deciding against their use.

Instead, the Raspberry Pi Pico emerged as the chosen microcontroller for this project, as delineated in Figure 23, because of its fast timer of 62.5MHz, small size and good documentation. This module has 26 multifunctional general-purpose input/output pins (GPIO) with pulse width modulation capability (PWM) and three analog inputs. The RP2040 microcontroller underpins the Pico, featuring a dual-core Arm processor with a 133 MHz clock speed, packaged efficiently in a cost-effective unit. Despite the presence of equivalent alternatives, the negligible price differential and projected longevity of the Pico validate its adoption.

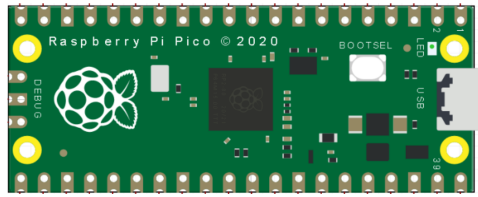


Figure 23 Raspberry Pi Pico

Table 1 Comparing Embedded System Platforms

Feature	Arduino Due	BeagleBone Black	Raspberry Pi Pico	FPGA	CPGA
Category	Microcontroller	Microcontroller	Microcontroller	Field-programmable Gate Array	Complex Programmable Logic Device
Processor	Atmel SAM3X8E ARM Cortex-M3	TI Sitara AM335x ARM Cortex-A8	RP2040 Dual-core ARM Cortex-M0+	Configurable Logic Blocks (CLBs)	Multiple FPGAs in a single package
Timer Speed	42 MHz	200 MHz, using programmable real-time unit	62.5 MHz	Varies	Varies
Memory (RAM)	96 KB	512 MB DDR3 SDRAM	264 KB	Varies	Varies
Memory (Flash)	512 KB	4 GB eMMC Flash	2 MB Flash	Varies	Varies
Size	101.52mm*53.3 mm	86mm*53.3mm	51mm*21mm	Varies	Varies
Development Environment	Arduino IDE	Linux distribution	MicroPython, C/C++, Arduino IDE	Hardware Description Language (HDL)	Hardware Description Language (HDL)
Learning Curve	Beginner-friendly	Intermediate	Beginner-friendly	Advanced	Advanced

2.6.1.2 Piezo Actuator

The P-820.30 linear piezo actuator from Physik Instrumente (PI) aligns with the project's specifications because of its high voltage range (up to 100V) and its capacity for substantial displacement within a compact size. Its 45-micrometer displacement capability is significant. This functional range is achieved in an actuator smaller than the one in Moldovan's 1993 device (which is no longer produced), making it suitable for the demonstrator. Moreover, its balance of aesthetic appeal and precise specifications, including force capacity and stiffness, ensures that it meets both the functional and design criteria of the project.

Further technical data of the actuator is on the next page:

Displacement at 0 to 100 V	45 $\mu\text{m} \pm 20 \%$
Static large-signal stiffness	4 N/ $\mu\text{m} \pm 20 \%$
Push/pull force capacity	50 / 10 N
Max. torque limit (on tip)	0.08 Nm
Electrical capacitance	1.0 $\mu\text{F} \pm 20 \%$
Unloaded resonant frequency f_0	12 kHz $\pm 20 \%$
Mass	14 g $\pm 5 \%$
Length L	62 mm ± 0.3

The P-820.30 was acquired before this thesis began, and its application is based on its readiness, rather than a deliberate selection by the author. Its low capacitance is a downside, and it is recommended to buy a piezo actuator with a higher capacitance to avoid the resonance frequency. In the next section, I discuss the resonant frequency associated with the piezoelectric actuator.

Resonance frequency of piezo

This section delves into the dynamic interplay between the system's capacitive and resistive components and how this interplay affects operability and risk management. In particular, I will focus on achieving an optimal cutoff frequency to prevent the detrimental effects of resonance without compromising actuator efficiency. This discussion will culminate in a detailed analysis of the design choices made to balance performance and durability, thus providing a comprehensive understanding of the technical complexity of the actuator. In doing so, I aim to contribute to a more refined design method that prevents damage to the piezoelectric material while maintaining the required mechanical performance.

The piezo actuator in question possesses a capacitance of 1 microfarad (μF) and is subject to parasitic resistances originating from the actuator itself, connected wiring, and switching mechanisms. The used piezo has a resonance frequency of 12,000 Hz. It would be desirable to have a cutoff frequency $f_{3\text{db}}$ according to equation (5) beneath the resonance frequency to avoid the destruction of the piezo. It could oscillate so severely at its resonance frequency that the many layers of glued piezoceramic would separate (Moldovan, 1994a, p. 22) .

Equation (5) predicts a cutoff frequency of approximately 159 kHz when parasitic resistance is approximated at 1Ω . This simplification doesn't account for lower observed resistance values in the system components, potentially resulting in an actual cutoff frequency as high as 1 MHz.

It is difficult to measure the actual cutoff frequency without possibly damaging the piezo. The resonance frequency would have to be skipped as fast as possible in a test setupw. Lowering the cutoff frequency f_p can be done by inserting an additional resistor (a current shunt) between the piezo and the switch, or by using a piezo actuator with higher capacitance. To achieve a target cutoff frequency of 10 kHz, inserting a resistor with a minimum value of 16Ω is necessary,

calculated using Ω , and $R_p = \frac{1}{2\pi f_p C_p}$ from equation (5).

A 16 Ω resistor in series with the piezo significantly reduces current flow. This causes the actuator's force output to drop below practical levels, resulting in a shooting height for a 4g ball below 1 centimeter at 100V, as determined by experiments. Consequently, the shunt resistor was removed from the final design in order to maintain effective operation. While this omission increases the likelihood of actuator damage over time, the device's limited use justifies this method.

Compared to the piezo used by Moldovan in 1993, the contemporary PI P-820.30 actuator has 12% of the 1993 actuator's capacitance, adversely affecting the system's cutoff frequency. Moldovan's pendulum, equipped with a 2.7 Ω shunt resistor and an 8 μF piezo, attained a lower cutoff frequency of 7 kHz without substantially impairing amplitude (Moldovan, 1994a, p. 22) .

Strategies to lower the cutoff frequency involve trade-offs, such as reduced actuator power, and hence, have been discarded in the current design to preserve functionality. Nevertheless, a 0 Ω resistor was added to simplify any potential future modifications. Comparison with previous work highlights the technical challenge in maintaining system performance while mitigating risks- a balance that remains integral to the design process.

2.6.1.3 Piezo Power Supply

The piezo can be used at voltages up to 100V DC. Table 2 shows four available options to reach such a high voltage. When selecting a suitable piezoelectric power supply, capable of operating at voltages as high as 100V, the key considerations included system simplicity, safety, necessity for low voltage input, adjustable output voltage, and maintenance of a minimal increase in weight.

The analysis concluded that the 12V DC with custom designed step-up converter circuitry was excessively intricate. Step-up converters inherently require additional components such as inductors, diodes, capacitors, and control mechanisms to facilitate voltage variation and stabilization. The inclusion of these additional components tends to complicate the overall circuit architecture, introducing additional points of failure that can adversely affect system reliability. In addition, achieving a consistent 100V output requires careful component selection and thorough circuit optimization.

The 230V to 100V AC transformer with rectifier was also assessed. It revealed significant safety concerns because its high input power could potentially increase the risk of dangerous incidents during testing. Additionally, this option might add considerable size and weight to the device, compromising its intended compactness, and the voltage would be fixed to 100V. Therefore, it was considered less ideal for the project requirements.

The PI 841.05 is a compact step-up power supply module developed by Physik Instrumente (PI) for use with piezo actuators. It was also eliminated because of its high cost, although it offered advantages in simplicity and safety.

As a result, the Recom R12-100B DC/DC converter was selected because it met the functional and fiscal requirements of the project, as well as ensuring safety at low input voltages and simplicity of design. A notable, viable alternative not mentioned earlier is the PDu100B by Piezodrive, which offers a 100V power supply and amplifier for \$90.

Table 2 Comparison of voltage source options

Feature	12V DC with custom designed Step-Up Converter circuitry	230V to 100V AC Transformer with Rectifier	12V DC with PI 841.05 Step-Up Power Supply Module	Recom R12-100B DC/DC Converter
Input Voltage	12V DC	230V AC (mains power)	10-30V DC (variable)	12V DC
Output Voltage	100V DC (adjustable)	100V DC (fixed)	127V DC (fixed)	50 - 135V DC (adjustable)
Safety	Lower risk (low voltage input)	Requires caution with mains power	Lower risk (low voltage input)	Lower risk (low voltage input)
Cost	Moderate (converter only)	Low (transformer + rectifier)	High (337€)	Moderate (45€)
Complexity	Moderate	Simple (transformer + rectifier)	Simple (single module)	Simple (single module)
Availability	Widely available	Widely available	Requires specific supplier	Widely available

2.6.1.4 Switching the Piezo

The piezo must be switched fast in the piezo pendulum, as the delay between ball detection and discharging must be precisely timed for a bounce-free catch, as discussed in the section 1.2.1.5. Metal-Oxide-Semiconductor Field-Effect Transistors (MOSFETs) exhibit the swiftest switching, superior to devices such as electromechanical relays and bipolar junction transistors (BJTs) which have slower operational speeds (Bogdanović et al., 2021, pp. 22–27) .

I measured the peak current flowing to the P-820.30 linear piezo actuator by shooting a ball using Moldovan's (1993) piezo control unit; it was around 50A at 90V. This measurement was performed with an oscilloscope using a resistor of known resistance, in conjunction with equation (6). Considering this finding, the switching mechanism's drain current should logically exceed the 50A threshold to accommodate the system's requirements.

Similarly, I measured the charging duration with Moldovan's piezo control unit, resulting in an approximate timeframe of 250 nanoseconds, as depicted in Figure 24. This period differs slightly in the final prototype as other electronics are used. Knowing this charging time is critical for establishing the duration the assumed switching mechanism must remain active to ensure the piezo is fully charged.

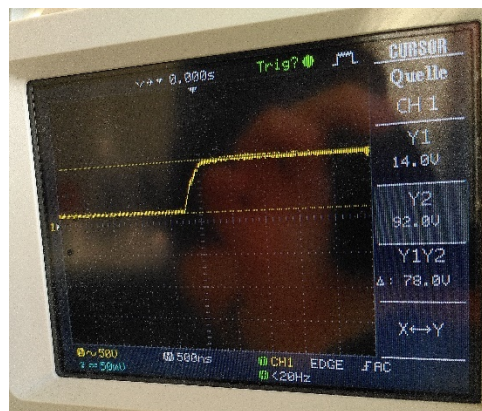


Figure 24 Voltage curve of P-820.30 actuator when charging

Although half-bridge modules with integrated MOSFETs and piezo drivers also demonstrate fast switching speeds, their performance is highly contingent upon individual integrated circuit design. For instance the Infineon IRSM808-204MH emerges as a small 20A, 250V half-bridge module (9mm x 8mm). It is characterized by its compatibility with 3.3V input logic and a notably low on-resistance of 0.15Ω . However, its construction within a PQFN (Power Quad Flat No-Lead) package presents complexities in the soldering process. This intricacy often necessitates advanced techniques and equipment. And because it can only be operated at a 20A output, I did not consider this idea further.

Piezo drivers, such as the DRV2700 device from TI on the other hand are especially developed to control piezo actuators, and often have the power supply included. But most of them were created especially for periodic oscillations and not intended for on-off switching operations. Lastly, they are often too slow for my purpose.

Therefore, given the paramount importance of rapid and reliable switching in driving piezoelectric devices, MOSFETs emerge as the superior option for the piezo pendulum.

2.6.1.5 Bridge Configurations

My study further investigates methods to energize the piezo actuator, narrowing down to half-bridge and full-bridge configurations. I have excluded an exhaustive review of piezo driving techniques, as prior studies, such as Gnad, 2005 , provide extensive analyses.

The **full-bridge** setup can charge a piezoelectric actuator to both a positive voltage, V_p in Figure 25 a), and a negative voltage, $-V_p$ in Figure 25 b). This would double the voltage that the piezo receives, which according to (3), would quadruple the absorbable and releasable energy. Nonetheless, the piezo in my experiment is limited to a 0V to 100V range and cannot sustain a -100V charge without damage.

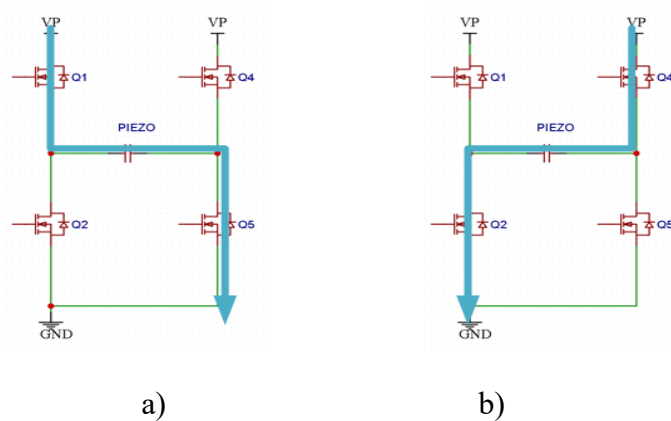


Figure 25 Way of current when controlling the MOSFETs in a simple full-bridge setup; a) Piezo is charging; b) Piezo is discharging

Figure 26 shows a half-bridge setup. When the upper MOSFET Q1 is activated and Q2 is deactivated, the piezo charges Figure 26 a). When Q1 is deactivated and Q2 is activated, the piezo discharges Figure 26 b).

The half-bridge configuration's simpler design and cost-saving potential make it advantageous for my purposes. The design requires fewer components, which may reduce expenses and simplify the circuit design. However, it is critical to ensure that transistors Q1 and Q2 are not activated simultaneously to prevent shoot-through, a destructive phenomenon in MOSFETs (Figure 26 c).

The MOSFET on the high side (Q1) needs a higher gate voltage than the voltage at its source end, which connects to the positive terminal of the piezo device. I will discuss how to control the MOSFETs in the next two sections.

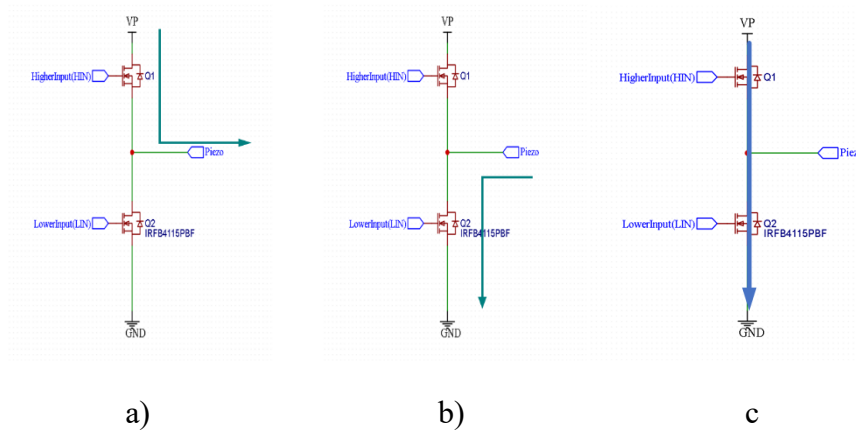


Figure 26 Way of current when controlling the MOSFETs in a half-bridge setup; a) Piezo is charging, only Q1 active; b) Piezo is discharging, only Q2 active; c) Way of current at destructive shoot-through when Q1 and Q2 active

2.6.1.6 Control Strategies for MOSFETs

Now that the half-bridge configuration has been chosen, an effective control strategy must be used to control both MOSFETs, with options including microcontroller direct drive, forward converters, and gate driver ICs.

Microcontroller direct drive offers a simpler alternative where a microcontroller provides signals directly to the MOSFET gates. It is inexpensive and straightforward, ideal for low-power situations. However, it is limited in drive capability, more susceptible to noise, and may require additional components for stability, especially in high-side switching. Because the microcontroller can only provide 3.3 volts—insufficient to fully turn on a MOSFET suitable for 100V operation—this method is not feasible in my case.

Forward converters are adept at generating the necessary gate voltage for MOSFETs, particularly useful when stepping up from a lower DC input to a higher DC gate drive voltage. Moldovan (1994) used two forward converters, see the transformers connecting to the gate of the MOSFETs in Figure 11. These devices use a transformer to provide a constant output and provide galvanic isolation, so that voltage-spikes from the high-side cannot affect the digital circuit. However, adding forward converters increases the complexity of the circuit, making the circuit larger and the device heavier.

Gate driver circuits use special ICs that provide strong isolation from high power areas and protection against electrical noise. This method is suitable for complex applications like the piezo pendulum, where high-side switching is required, and the design is generally more straightforward, requiring fewer parts.

Consequently, a discrete gate driver was chosen for its simpler design. However, this choice raises the issue of driving the high-side MOSFET (Q1 in Figure 26). Because its source terminal connects directly to the positive node of the piezoelectric actuator, it requires a gate voltage exceeding this node's voltage. This could be achieved by using various methods. Therefore, a critical comparison is required.

2.6.1.7 Driving the High-Side MOSFET:

Bootstrap circuits, isolated gate drivers, or forward converters can be used to switch the high-side MOSFET in the half-bridge configuration. Table 3 shows a detailed comparison between the devices.

Firstly, an **isolated gate driver** requires a transformer or isolated DC-DC converter, and provides uninterrupted operation, meaning it can turn on a MOSFET continuously. As the name indicates, the device provides galvanic isolation. However, it comes at a higher cost and adds complexity to the circuit.

Forward converters were discussed in 2.6.1.6, but now I am discussing only driving the high-side MOSFET. As mentioned, forward converters provide galvanic isolation but may require a custom-made transformer, resulting in a more extensive and expensive assembly.

Moving on, Figure 27 illustrates a basic **bootstrap** configuration: Initially, with no charge, the potential of MOSFET Q1 mirrors that of ground. During this phase, capacitor C accumulates charge up to the supply voltage V_{cc} through the diode. As the piezo accumulates charge, Q1's source rises above ground, affecting system dynamics. The fully charged capacitor C, impeded by the diode, maintains the full voltage across Q1's gate source when the switch is engaged, driving the MOSFET to full saturation (Günther et al., 2013, p. 32).

A bootstrap circuit, using a gate driver IC instead of the mechanical switch in Figure 27, does not provide galvanic isolation. Instead, it offers simplicity through fewer needed components. Their straightforward design and commercial availability make them accessible and relevant for the intended use. I selected the bootstrap method for its simplicity and cost-effectiveness. It stands out for easy implementation and broad availability but presents challenges, including a limited duty cycle—the MOSFET cannot be continuously active because it relies on the boot capacitor's charge—and the absence of galvanic isolation. Consequently, careful choice of boot capacitor and boot diode is a primary focus in the design process.

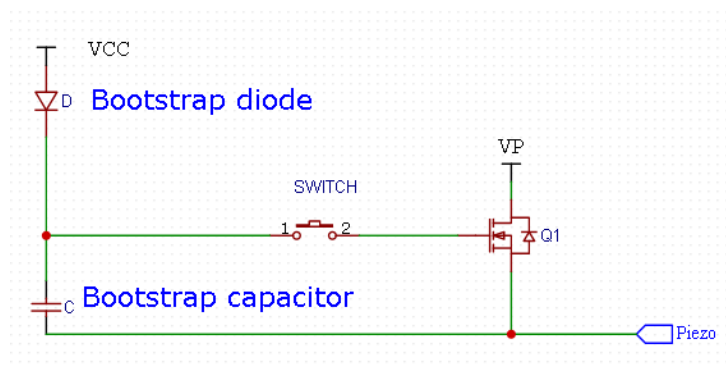


Figure 27 Simple bootstrap circuit

Table 3 Comparison between bootstrap circuit, isolated gate driver and two forward converters

Feature	Forward converters	Bootstrap configuration with gate driver	Isolated gate driver
Galvanic isolation	Yes	No	Yes
Uninterrupted operation	Yes	No	Yes
Number of components	More	Less	More
Transformer	Required, eventually custom build	Not required	Required or isolated DC-DC converter
Size	Bigger	Simpler	Bigger
Cost	More expensive	Less expensive	More expensive
Ease of implementation	More complex	Simpler	More complex

2.6.1.8 Absence of Piezo Amplifier in Control Circuitry

An amplifier serves as an essential device for boosting the signal strength of a control circuit, allowing precise manipulation of outputs such as the voltage of a piezo actuator (V_p). Without an amplifier, it is difficult to make quick and accurate adjustments to the voltage. The ability of an amplifier to make rapid changes is critical when adjusting the launch height of a ball at rapid intervals. It also allows the pendulum to create a variety of acoustic effects, such as mimicking musical beats by creating louder sounds with longer pauses using high voltage, or softer sounds with shorter pauses using low voltage, which would certainly be an exceptional feature for a demonstrator.

However, despite its advantages, the circuit design ultimately did not include an amplifier. The PI E-831.05 amplifier from Physik Instrumente (PI) was considered because of its compatibility with the PI P-820.30 piezo actuator, its compactness, low noise, and stability. It was not used because of its high cost (205 Euro) and long delivery time (8-10 weeks). Because the amplifier is not critical to the demonstrator's basic functions, I determined that these barriers outweighed its potential benefits.

2.6.1.9 Impulse Detection

When the pendulum's ball strikes the piezoelectric actuator, the microcontroller must detect it in my setup. To do this there are various methods, listed in Table 4:

While a Schmitt trigger could eliminate erroneous signals, and a digital signal processor (DSP) offers extensive flexibility, the DSP's complexity is unnecessary for my requirements.

A viable method involves using a high-speed analog-to-digital converter interfaced with a microcontroller for software-based impulse processing. For example the ADC08100CIMTC/NOPB captures voltage at a rate of up to 100 million samples per second; this setup allows for easily adjustable thresholds via software.

Although operational amplifiers with feedback are applicable for impulse detection, comparators are preferable because of their rapid response.

Consequently, I chose a window comparator for its simplicity, adequacy for the intended function, and proven efficacy, as demonstrated by Moldovan, 1994a, p. 12 .

Table 4 Comparison of possible methods for impulse detection

Method	Description	Advantages	Disadvantages
Window Comparator (used by Moldovan, 1994a, p. 12)	Uses two comparators to define a window for the expected impulse signal.	Simple and reliable for basic detection, thresholds can be changed with potentiometers	Sensitive to variations in signal strength.
Schmidt Trigger	Cleans signal and triggers on clear rise/fall above threshold.	Improves noise immunity, prevents false triggers.	Adds complexity compared to window comparator.
Microcontroller with ADC	Monitors voltage and triggers on programmed threshold.	Very fast options available, allows for advanced processing.	More complex setup, expensive
Digital Signal Processor (DSP)	Analyses signal in real-time using algorithms.	Most versatile, allows for complex filtering and recognition.	Expensive, requires significant expertise.
Op-amp with Feedback	Configurable comparator with adjustable sensitivity.	Offers adjustable threshold and clear output signal.	Normally slower than window comparators

For a detailed understanding of each theoretical concept, refer to the work of Tietze et al., 2016 .

2.6.1.10 Casing

The proposed case designed for 3D printing facilitates rapid prototyping. The inherent ease of modification in this process allows for precise personalization to unique requirements, providing an efficient method for creating custom cases. In addition, the aesthetics of the design can be enhanced to improve its visual appeal. By mounting the piezo actuator directly onto the control unit rather than on a separate base, as shown in Figure 9, the susceptibility of the wires to damage is significantly reduced.

2.6.1.11 Absence of Portable Power in Control Circuitry

While Moldovan's device used batteries, this one will be designed without batteries. The device remains simple, compact, and lightweight. It is intended for stationary use in educational settings, which mitigates the need for portability.

2.6.1.12 Summary of Design Decisions So Far

Table 5 consolidates the decisions made for the piezoelectric pendulum system's configuration so far, reflecting a careful balance between performance, simplicity, cost, and practical application.

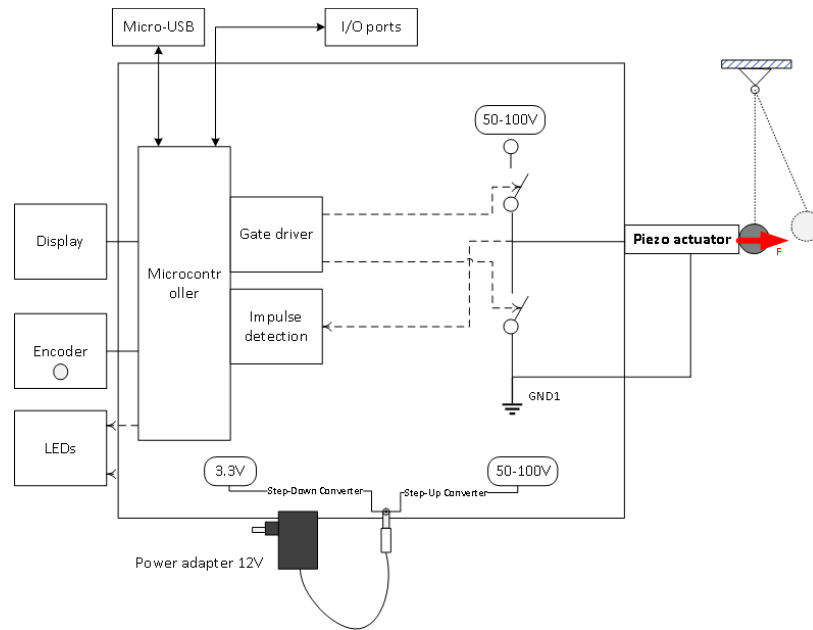
Table 5 Summary of design decisions made in chapter "Choice of Solution"

Component Name	Description	Rationale
Microcontroller (2.6.1.1)	Raspberry Pi Pico	- Supports comprehensive programming languages - Fast 62.5MHz hardware timers - Ample documentation
Piezo Actuator (2.6.1.2)	Physik Instrumente P-820.30	- Meets demonstrator requirements - Appealing design
Piezo Power Supply (2.6.1.3)	Recom R12-100B DC/DC Converter	- Meets safety, simplicity, cost, and input voltage requirements
Piezo Switching (2.6.1.4)	MOSFETs	- Superior switching speed - Reliable for fast piezo actuation
Bridge Configuration (2.6.1.5)	Half-Bridge Setup	- Simpler, cost-effective, and efficient design
MOSFET Control (2.6.1.6)	Gate Driver Circuit	- Simpler design
Driving the high-side MOSFET (2.6.1.7)	Bootstrap Circuit	- Cost-effective - Simpler design
Piezo Amplifier (2.6.1.8)	Not Included	- High cost and long lead time - Not essential for basic demonstrator operation
Impulse Detection (2.6.1.9)	Window Comparator	- Straightforward implementation - Proven performance in similar applications
Casing (2.6.1.10)	3D Printed	- Facilitates rapid prototyping - Allows custom design adaptations - Enhances design aesthetics
Portable Power (2.6.1.11)	Not Included	- Simpler design - Intended for stationary educational use

2.6.2 Functional Model

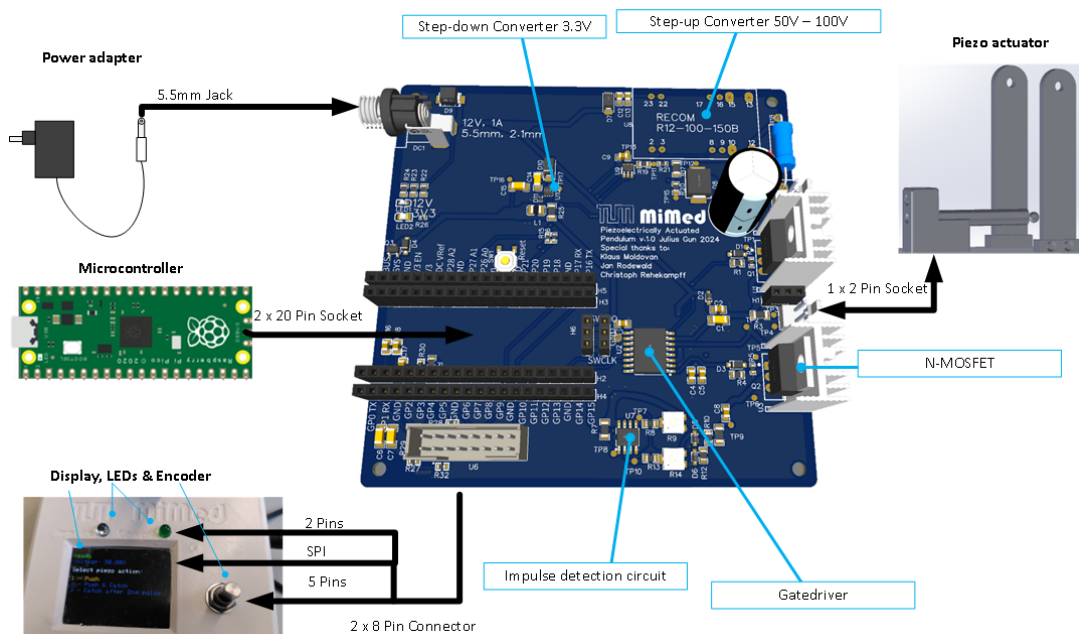
Building on the basic framework outlined in the previous section, this section concentrates on choosing appropriate electronic components. The creation and procurement of the Printed Circuit Board (PCB) are then described. These steps are essential to enhance the preliminary structure into a fully functional model.

Firstly, Figure 28 shows an overview of the hardware components used in the functional model. The piezo actuator is controlled by a series of signal exchanges, starting with the microcontroller, which uses input/output ports and is connected to an impulse detection system. It sends instructions to a gate driver, which then controls MOSFET switches to charge the piezo. The microcontroller also provides updates to the LEDs and interacts with a display and encoder. Power is supplied externally via a 12V input, which is appropriately adjusted: the digital circuit receives its voltage of 3.3V via a step-down converter and the piezo actuator receives its adjustable voltage of 50 to 100V via a step-up converter.



• **Figure 28 Simplified hardware overview of functional model**

Secondly, Figure 29 presents the completed Printed Circuit Board (PCB), detailing the arrangement and connections of constituent components. A 5.5mm power jack facilitates the application of 12V DC from an external power adapter. The design includes a socketed microcontroller that allows for the interchangeability of other 2x20 pin microcontrollers. Connectivity for the display, LEDs, and the encoder is achieved through a 2x8 pin connector. Finally, to connect the piezo, a 1x2 pin socket with a guiding feature ensures correct orientation, preventing incorrect polarization and potential damage to the piezo element.



• **Figure 29 Equipment setup: How the parts are connected to one another**

In the following subsections, I explore the model's building blocks. I start with power MOSFETs, key elements for switching the piezo, then move to the bootstrap half-bridge gate driver, integral for managing the MOSFETs. The power supply is discussed as the foundational support providing energy, followed by user interaction, which describes how individuals interface with the device. The ball used for the experiment is examined next. Subsequent sections dissect PCB design, including circuit layout. I also look at the software that drives the system's functionality and end with impulse detection, detailing how the system senses the returning ball.

2.6.2.1 PCB Design

The completed Printed Circuit Board (PCB) design is displayed in Appendix B and all schematics in Appendix A. In designing the PCB, I pursued the following strategies:

Firstly, I chose simple components to reduce assembly complexity and cost, as assembly by JLCPCB in China incurs additional expenses for "extended" parts, such as non-SMD or rare components. I used inexpensive parts, as documented in Appendix C. For instance, I used series-connected resistors in some cases instead of a single "extended" part (see Figure 35 for example) and occasionally employed parallel capacitors to achieve the required capacitance.

Additionally, the design conformed to guidelines suggested by Scherz, 2007, segregating the high voltage and digital sectors on the PCB, which measures 100x100mm. Efficient routing was prioritized, placing the microcontroller and high-frequency circuits like I2C and SPI near the power source. Decoupling capacitors were systematically placed to minimize logic spikes and electrical noise.

That being said, the design lacks galvanic isolation between the digital and high-voltage circuits (they share one ground), posing a risk of electrical interference and potential damage to components. Galvanic isolation would have made the circuit much more complex and was thus not implemented. While using optocouplers could provide isolation and improve safety, it would necessitate a redesign of the piezo power supply and other components. Nonetheless, two grounding systems, GNDD (for digital parts) and GND (for common parts), were designed and linked later to pave the way for possible future implementation of galvanic isolation.

Lastly, the PCB features numerous TestPoints (TP) for oscilloscopic analysis of signals and voltages. These TestPoints facilitate direct probing of electrical pathways, essential for assessing PCB performance. They prove invaluable for pinpointing faults and validating the design during the circuit development phase.

2.6.2.2 Power MOSFETs

MOSFETs are used to rapidly change the piezoelectric device's state, as discussed in the section 2.6.1.4. The chosen Infineon IRFB4115 provides a drain current significantly higher than the 50A required by the actuator, as described in the section 2.6.1.4.

MOSFET technology has advanced substantially compared to the technology available three decades ago. This indicates significant development since Moldovan's prototype was created, as Table 6 shows. The table compares the BUZ350 used by Moldov and the new IRFB4115. For further understanding of the technical terms being used, one can refer to Tietze et al., 2016.

The IRFB4115 MOSFET can conduct current up to 104A continuously. And because it is activated only for a short time, only the IDM Pulsed Drain Current is relevant. This current is 420A, providing a significant margin over the target drain current. Its selection was based on a favorable balance of performance and cost, considering that lower-performance options cost about the same. The VDS Drain-to-Source Voltage also leaves some margin for voltage spikes, because voltages higher than 150V could damage the MOSFET.

This MOSFET model has an activation time of only 18 nanoseconds and a very low drain-to-source on-resistance (RDS). These characteristics are favorable for achieving rapid charging, because a swift voltage rise enhances the energy absorption rate and subsequent conversion to kinetic and potential energy, as detailed in the section 1.2.1.3. Despite the benefit of rapid charging, a longer activation time is not detrimental. It may reduce the cutoff frequency (equation (5)), an effect that can be mitigated by the introduction of a shunt resistor, as elaborated in subsection 2.6.1.2.

The MOSFET's brief activation, limited to tens of microseconds, contrasts with the ball's longer airborne phase of hundreds of milliseconds, allowing natural convection to suffice for heat dissipation. However, to provide additional thermal protection, heat sinks are attached to the MOSFETs. This precautionary measure aims to prevent overheating during operation, thus ensuring the reliability and longevity of the device.

Table 6 Comparison between MOSFETs: BUZ350 used by Moldovan (1994) and new IRFB4115

MOSFET	BUZ350 used by Moldovan (1994)	IRFB4115
R_{DS}^2 (on) (@10V) max	120 m Ω	11 m Ω (Over ten times lower)
V_{DS}^3 max	200 V	150 V (Lower voltage rating of piezo actuator)
I_D^4 Continuous Drain Current	22 A	104 A (Over five times higher capacity)
I_{DM}^5 Pulsed Drain Current	85 A	420 A (Over five times higher capacity for pulsed operation)

² R_{DS} = Drain-to-Source On-Resistance

³ V_{DS} = Drain-to-Source Voltage

⁴ I_D = Continuous Drain Current is the maximum current a MOSFET can handle continuously without damage.

⁵ I_{DM} = Pulsed Drain Current is the maximum current a MOSFET can handle in short bursts.

2.6.2.3 Bootstrap Half-bridge Gate Driver

The section 2.6.1.7 presents an analysis of bootstrap circuits relative to other options, culminating in my selection of a bootstrap circuit as most suitable for this development. Using a bootstrap circuit contributes to a more efficient circuit design, decreasing size and intricacy, which simplifies production. Additionally, the final device is lighter because it lacks transformers, which are crucial in forward converters.

The final bootstrap circuit is illustrated in Figure 30. This circuit incorporates a bootstrap capacitor configuration-comprising two capacitors in parallel. Using a bootstrap capacitor charged during the high-side MOSFET's off-cycle allows the circuit to maintain a gate voltage above the source voltage. This ensures the high-side MOSFET is driven into full conduction. The bootstrap diode D2 is used to generate a voltage exceeding that of the main power supply, V_{cc} . This elevated voltage is instrumental in the activation of the high-side MOSFET Q1. The diodes D1 and D3 ensure that the MOSFETs are protected from potential voltage spikes that could occur. Resistor R3 has 0 Ohms and could be replaced later, as discussed in the subsection 2.6.1.2. The heatsinks U1 and U3 provide thermal cooling to the MOSFETs.

I use the IR2110STRPBF from Infineon, a half-bridge gate driver, to control two MOSFETs, Q1 and Q2. A gate driver is an essential component in power electronics that serves as an intermediary between control signals (often from a low-power microcontroller or signal processor) and power MOSFETs. It is necessary because it amplifies the control signals to the appropriate levels required to switch the MOSFETs on and off effectively and efficiently. Without a gate driver, the MOSFETs may not receive a high enough voltage to transition rapidly between states, which can lead to increased heat dissipation and slower operation. The IR2110STRPBF offers fast actuation with a 120ns turn-on time, includes safety features like an under-voltage lockout, and is supported by abundant online resources for ease of use.

The microcontroller is configured to transmit two signals to the driver, as depicted in Figure 26: one signal for controlling the high-side MOSFET (HIN) and another for the low-side MOSFET (LIN). In my specific application, the HIN signal to Q1 activates the piezoelectric element. Upon activation, the piezoelectric element propels the ball, which subsequently rebounds, triggering an impulse detection mechanism that initiates a counter. The piezoelectric element's high state is disengaged via the LIN signal to Q2.

It is crucial to note that the selected gate driver lacks inherent shoot-through protection. This condition can occur if both high-side and low-side MOSFETs turn on simultaneously, potentially causing catastrophic MOSFET failure, as discussed in the section 2.6.1.5. To mitigate this risk, I took meticulous precautions in the programming code to prevent concurrent activation of both signals, thus preserving MOSFET integrity and functionality. There are other gate drivers with shoot-through protection, such as the Infineon 2ED2182-4-S06F, but acquiring them can be challenging, while the IR2110STRPBF is readily available.

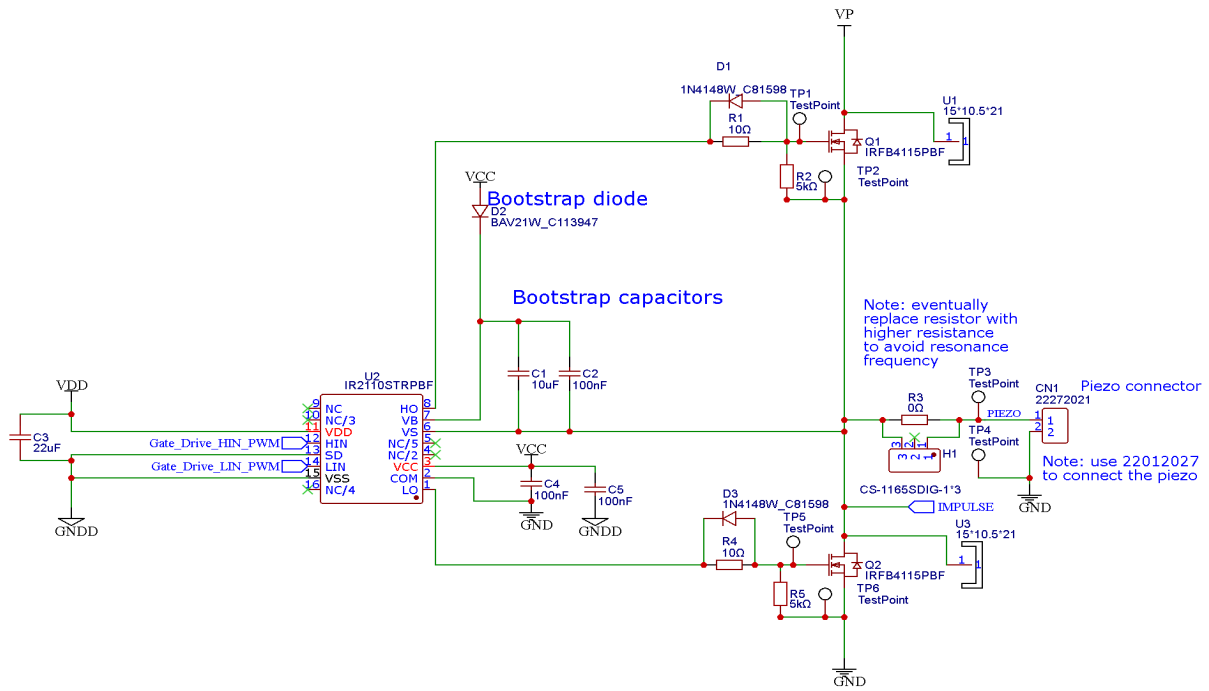


Figure 30 Gatedriver circuit

2.6.2.1 Impulse Detection

When the ball hits the piezoelectric actuator, the microcontroller detects a voltage shift via the comparator circuit. I employ the window comparator setup, described by Moldovan, 1994a, p. 12, is employed for impulse recognition at the moment of impact, as discussed in section 2.6.1.9. A window comparator checks if an input voltage resides within a specific voltage range, set by two predefined reference levels (Tietze et al., 2016, p. 601). The reference levels in my configuration are +40mV and -100mV. Moldovan determined these experimentally, and I have also validated them for the P-820.30 actuator. When choosing another piezo actuator, other reference levels might have to be determined.

Figure 31 illustrates the circuit; the "IMPULSE" from the piezo enters from the right. Here, capacitor C8 filters out the DC offset, allowing only current fluctuations to pass. Resistor R11 reduces current flow. Clamping diodes D5 and D6 ensure the comparator's input voltage remains between -1V and +4V, as it would otherwise reach up to 100V. To adjust the voltages at testpoints TP7 and TP10, one must connect a multimeter appropriately. The red probe of the multimeter is to be attached to either TP7 or TP10, while the black probe is to be connected to TP9. This setup allows for the accurate measurement and calibration of the voltages at the designated testpoints through the manipulation of potentiometers R9 and R13.

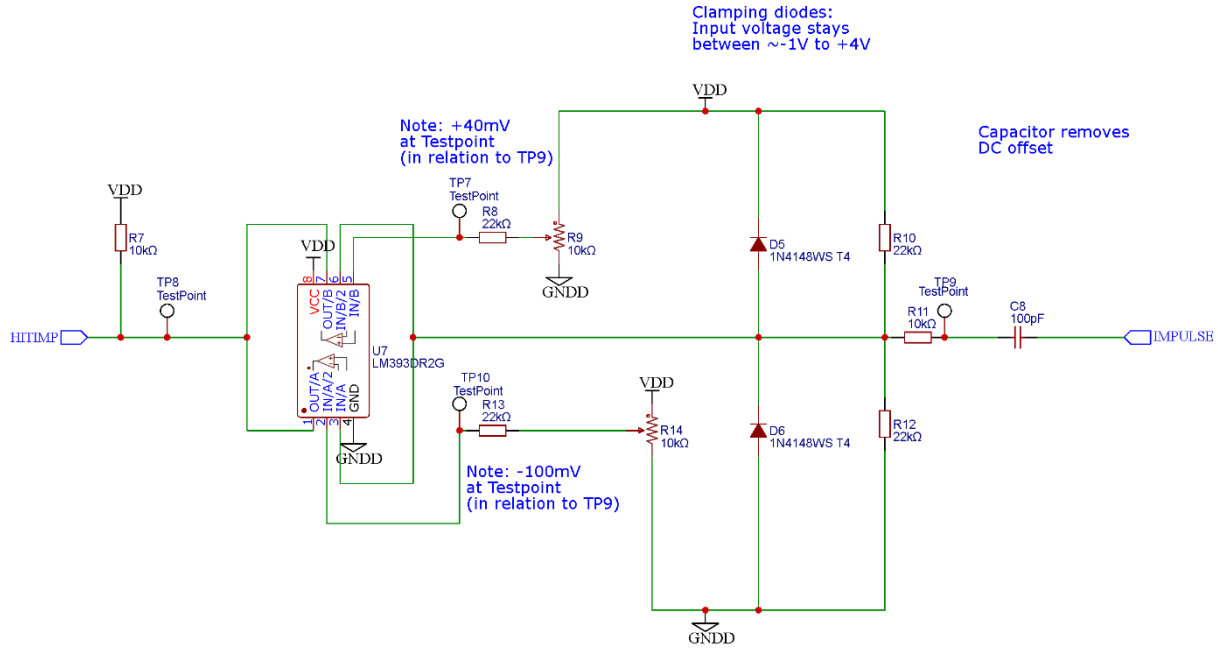


Figure 31 Impulse detection circuitry (based on (Moldovan, 1994a, p. 12))

2.6.2.2 Power Supply

The unit is powered by an adapter that supplies 12 volts of DC (V_{CC}) at 1A. The voltage V_{CC} is then reduced to 3.3 volts to accommodate the digital circuitry. Concurrently, V_{CC} is boosted to 100 volts to power the piezoelectric component.

2.6.2.2.1 Piezo Power Supply

The Recom R12-100B DC/DC converter, designated as U8, energizes the piezo illustrated in Figure 32. Charge-retention is enabled by buffer capacitors C10 and C11, which provide energy upon activation of the piezo. Additionally, the Recom R12-100B requires a baseline load of 10mA for proper operation, a demand met by resistor R10. Diode D8 offers overvoltage protection by capping the potential at 100V, thus safeguarding the piezo.

Further on, the microcontroller reads the current voltage at the piezo V_p via the signal Analog_V_measure through a voltage divider of R17 and R20. To provide a margin to accommodate occasional voltage surges the upper limit for voltage detection by the microcontroller is fixed at 2.2V for a piezo voltage (V_p) of 100V, according to the basic formula:

$$\left(\frac{R_{20}}{R_{17} + R_{20}} \right) \times 100V = 2.2V$$

Within its operational capabilities, the Recom R12-100B can output a voltage between 50 and 100 volts, even though reaching up to 135 volts is technically possible. To adjust the voltage on component U8, one can apply a control voltage ranging from 0 to 4 volts between the VADJ and GND terminals. This adjustment can be achieved using either a digital-to-analog converter (DAC) or a digital potentiometer; for this study, I chose the 8-bit DAC5571 by Texas Instruments as a suitable component.

The microcontroller communicates with U8 via an I2C signal to regulate its voltage. This signal is directed to U9, the digital-to-analog converter, with output voltages depicted in Figure 33. At a V_{ADJ} output of 0V, V_P measures 50V; when V_{ADJ} reaches 3.3V, the maximum for my digital circuits, V_P climbs to a piezo-threatening 120V. To safeguard against overvoltage, a voltage divider, comprising resistors R19 and R21, follows the digital-to-analog converter, capping the output at 2.35V—thereby preventing activation of the piezo beyond 100V.

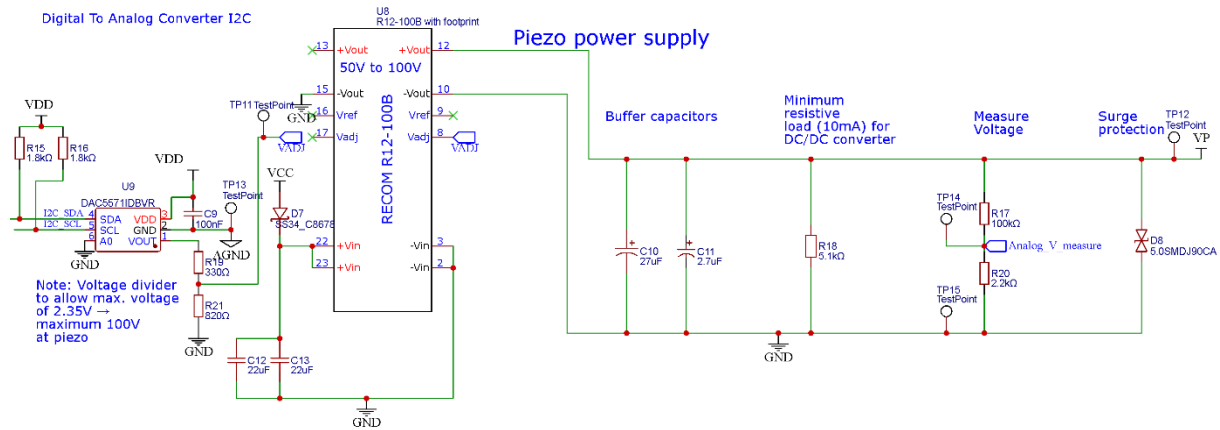


Figure 32 Piezo power supply

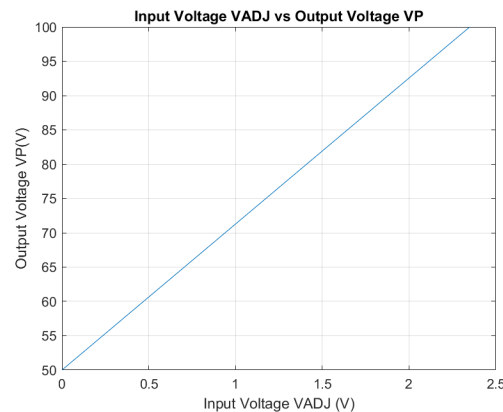


Figure 33 Voltage at the piezo V_P over the the input voltage V_{ADJ} , the voltage that is provided after the voltage divider at Testpoint 11 in Figure 32

But the voltage at the piezo is less than 100V, because activation triggers a voltage decline in the piezo due to charge redistribution from buffer capacitors, as depicted in Figure 34.

Initially, the buffer capacitors charge up to a V_P of 100V (left); subsequently, the engaged piezo causes them to discharge. The voltage source's minor current influence is omitted in this context. Mathematical delineation indicates the piezo actuator acquires a voltage near 96.8V, given the piezo's capacitance of $1\mu\text{F}$ and the buffer capacitors' total capacitance of $30\mu\text{F}$. Hence, the deduced voltage at which the piezo operates is 96.8V, after capacitor discharge, keeping a small buffer between its maximum operational voltage of 100V. The associated Matlab code in Appendix G applies these values to compute the resulting charge and voltage during the discharge process.

The charge is calculated using equation (1) by distributing the initial charge ($Q1_1$) between the piezo and the buffer capacitors, proportional to their capacitances. This redistribution is crucial because the piezo and the buffer capacitors form a network where charges are shared according to their respective capacitances. Hence, the formula for $Q2_2$ uses the ratio of the piezo's capacitance to the total capacitance of the system, multiplied by the initial charge $Q1_1$, to find out how much charge ends up on the piezo after discharge.

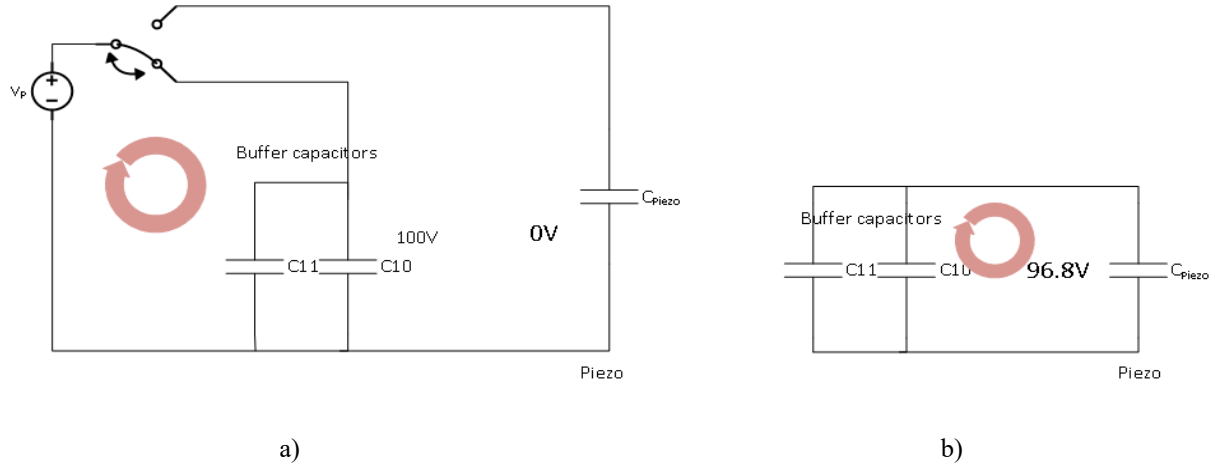


Figure 34 Role of buffer capacitors in charging the piezo; a) Piezo is not charging; b) Piezo is charging

2.6.2.2.2 Power Input

A 12V DC input is furnished through the power jack DC1, as shown in Figure 35. Following this, I chose series resistors (as mentioned in the section 2.6.2.1) to lower the current the indicator LED received, and surge protector D9 is in place. This component shields against potential damage from voltage spikes. It is essential for maintaining the integrity of the circuit's operation. Downstream, two DC/DC converters are tasked with energizing the digital components and the piezo.

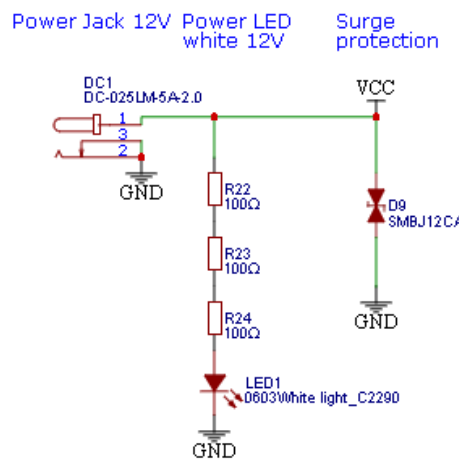


Figure 35 Power jack

2.6.2.2.3 Step-down converter

The step-down converter U10 supplies a stable 3.3V to all digital components on the circuit, with the operational status indicated by a green LED on the PCB:

Surge protector D11 safeguards digital circuits against excessive voltage. The other capacitors and inductor are necessary for the device to work properly, and values were chosen according to the U10 datasheet.

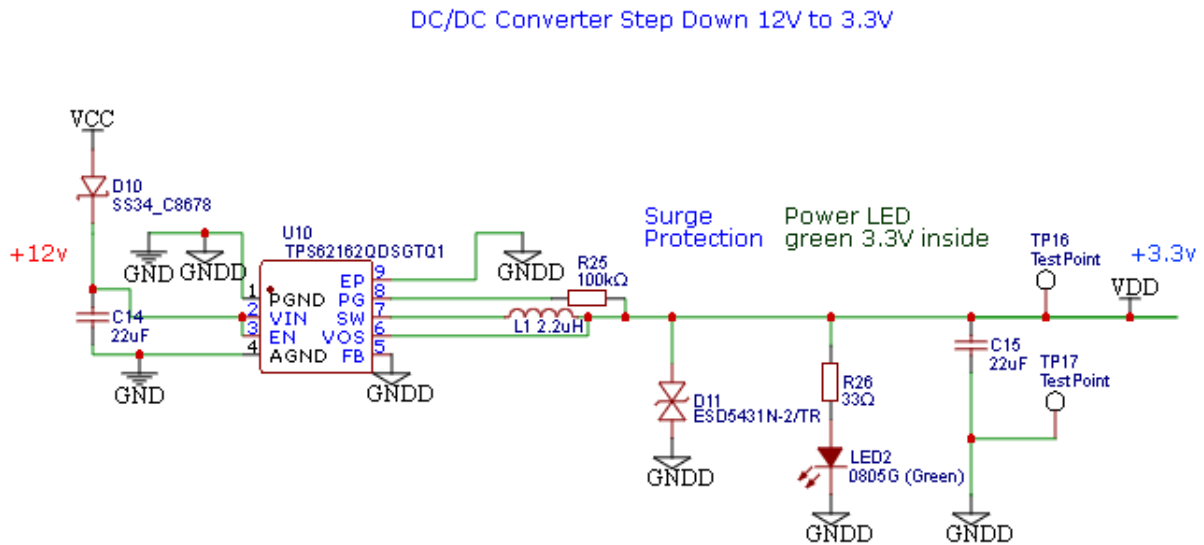


Figure 36 DC/DC converter - 12V to 3.3V

Power to the microcontroller is channelled through its VSYS port, which is engineered to handle an input voltage range from 1.8V to 5.5V, a fact highlighted in Figure 37.

The P-MOSFET Q3 operates as a switch, severing the VDD source circuit when the microcontroller is powered via USB. This ensures power is sourced exclusively from the USB when available, preventing potential overvoltage from dual power sources. Should USB power cease, the P-MOSFET restores the connection, allowing VDD to supply the necessary voltage to the microcontroller. This design elegantly ensures the microcontroller is powered without interruption while safeguarding it from electrical damage.

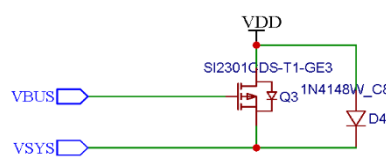


Figure 37 Power connection to Raspberry Pi Pico

2.6.2.3 User Interaction

I considered that the **display** needed to be at least two inches for readability, preferring a color screen for aesthetic appeal in menu design. I deemed including touchscreen functionality unnecessary, as it would make the system more complicated to use. The user interface in this project uses a 2-inch display with an ST778R chip (model DFR0664 from DFRobot) and an encoder with push functionality for interaction. Figure 38 shows the display connecting ports; it uses the SPI (Serial Peripheral Interface) protocol for communication with the microcontroller. An SD card can be used to store or retrieve data, for instance, a logo.

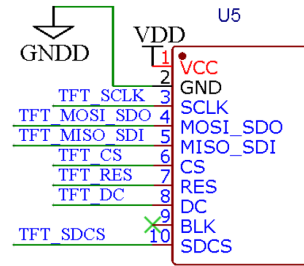


Figure 38 Display connections

The next image displays a rotary encoder with an integrated pushbutton. This enhances the overall appearance by eliminating the need for an additional button. The consolidation into a single unit not only saves space but also simplifies user interaction. I employed multiple pull-up resistors and decoupling capacitors to mitigate noise interference.

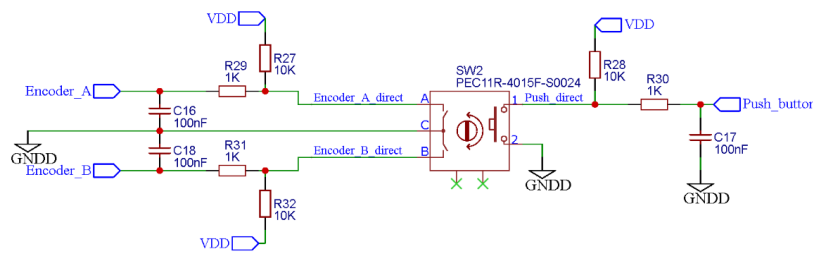


Figure 39 Encoder with pushbutton

The subsequent illustration shows that a connector is used to link the electronics of the front piece, which comprises a display, two LEDs, and an encoder. A green LED, designated as LED3, signals the power status, while a blue LED, named LED_IMPULSE, illuminates for a predetermined duration upon the detection of the ball.

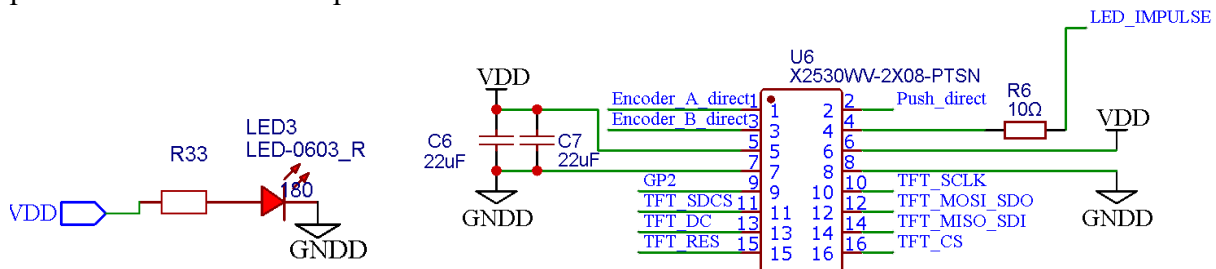


Figure 40 Connector for Display, LEDs on front & Encoder

2.6.2.4 Ball

The steel ball comes from a ball bearing and has a copper wire soldered to it. The ball must first be sanded before soldering for enhanced friction, the result can be seen in Figure 41. The ball weighs 1.4g. Experimental trials using a 4-gram ball yielded shooting heights that lacked sufficient interest for demonstration purposes, leading to the selection of a lighter ball. It is possible to use a ball made of any material, provided it can be appropriately mounted.

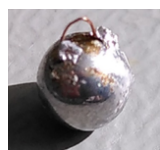


Figure 41 Ball with soldered copper wire.

2.6.2.5 CAD design

I conducted the Computer-Aided Design (CAD) process using Solidworks software. Illustrations of the resulting designs are available for review in Appendix F. Throughout the final prototype's development, I 3D-printed six distinct cases to determine the most appropriate design, as shown in Figure 42. This iterative method allowed for thorough exploration and refinement of design options.

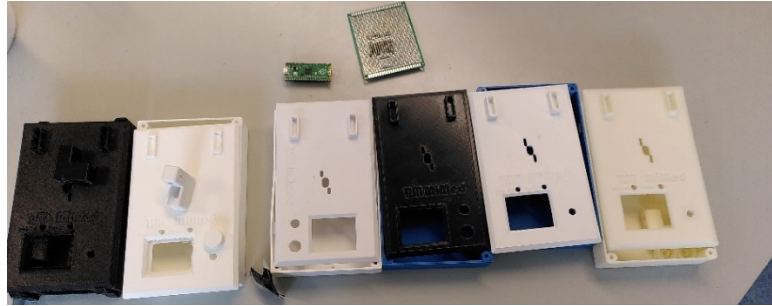


Figure 42 Iterations of the 3D printed cases.

In the construction of the described device, M3 and M2 threaded inserts serve as anchoring points for screw attachments, facilitating secure and stable connections within the plastic components. The control cover attaches to the housing using four M3x10 screws, ensuring a firm enclosure. Additionally, I secured the PCB (Printed Circuit Board) in position using four M2x5 screws, which contribute to the unit's structural integrity.

Thread supports, fashioned from acrylic glass through laser cutting, ensure high precision and a smooth finish. These elements are anchored securely using conical M2x5 screws, which provide stability and ease of maintenance for the assembly:

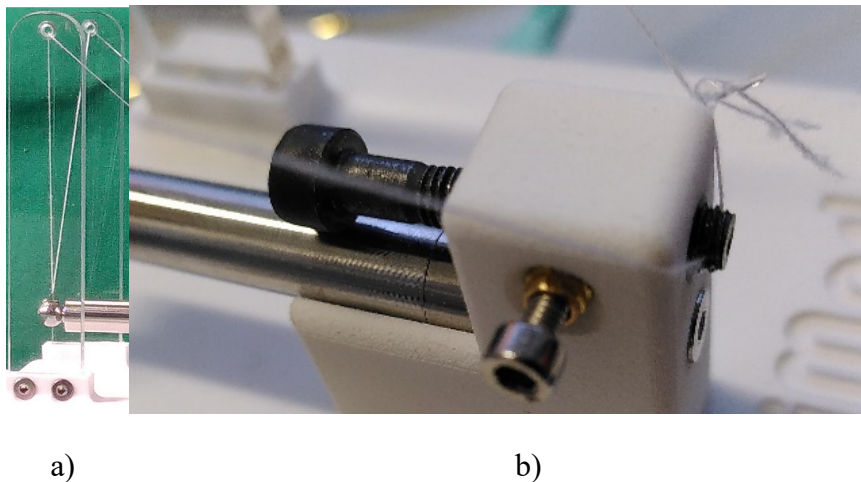


Figure 43 a) Thread supports; b) Thread holding screw

Further adjustable aspects, such as the string height, are facilitated by an M6x50 screw, which features a hole at its end to accommodate adjustments. This adjustment screw can be locked in place with an M3x12 screw, which is secured by an M3 insert, as illustrated in b) above.

2.6.2.6 Flowchart of Software

This section elucidates the chronological procedure implemented in the software's code, examined with the aid of Figure 44. A comprehensive flowchart of the software is available for reference in Appendix D. The software code itself can be seen in Appendix E.

Figure 44 shows the sequence of operations for navigating through the menu. The initialization sequence includes setting an interrupt for pulses, adjusting the voltage, and configuring variables. Delays in the 'timerDelays' array are critical for timing the capture process, with the first item starting with an 8 microsecond delay after one bounce, 5 microsecond delay after two bounces and so on. These delays can be changed by pressing and holding the pushbutton for two seconds. Users interact with the menu using an encoder and a button, while a software debounce mechanism mitigates accidental selections. The program has three main functions: "Push," "Push and Catch," and "Resonance."

To select a function, the user must press the pushbutton for longer than the predefined debounce time to mitigate noise and random activation. After completing a selected activity, the user can choose another operation; however, this subsequent selection process is not depicted because of spatial constraints.

The "Push" program discharges the piezo while the ball is airborne. In contrast, the "Push and Catch" mechanism discharges the piezo upon ball detection after a predetermined interval set by 'timerDelays'. Meanwhile, the "Resonance" mode responds immediately to an impulse interrupt by discharging the piezo without delay. In particular, the "Push and Catch" and "Resonance" functions can initiate the catching process after a predetermined number of bounces (one, two, or three); this detail is condensed to accommodate limited space in the chart.

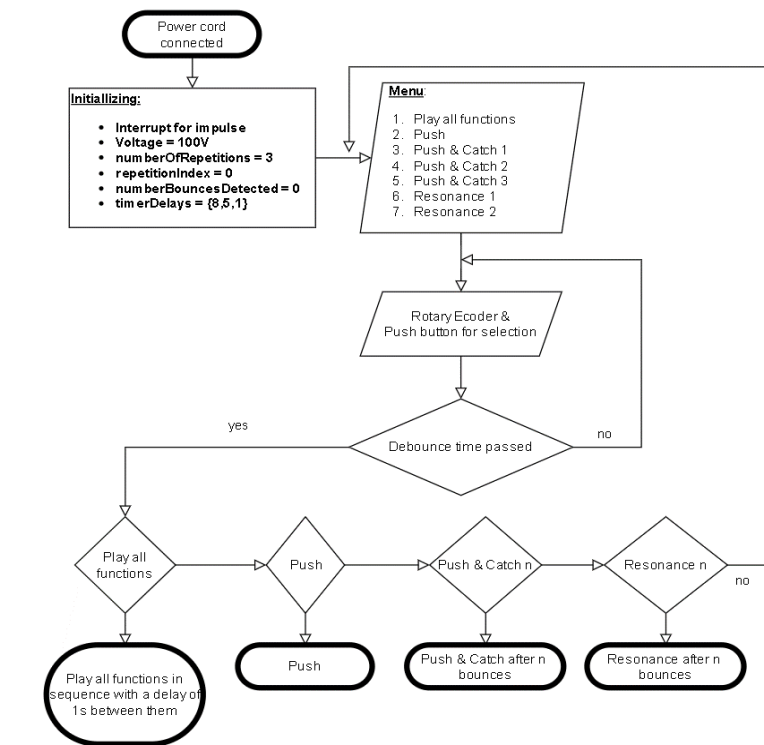


Figure 44 Flow chart of menu selection process

Figure 45 shows only the "Push and Catch" program because of its significance for the demonstrator and because all other programs are slight variations of it. It has further been simplified to accommodate limited space in the chart. A comprehensive flowchart that includes the other functions is available for reference in Appendix D. When the program "Push and Catch" is selected, the lower side MOSFET Q2 is turned on for 30 microseconds to charge the bootstrap capacitor. Then it is turned off, and after a delay of one microsecond to prevent shoot-through, it turns on the higher side MOSFET Q1 for 100 microseconds.

The piezo charges within approximately 250 nanoseconds but only after a delay. This is because the response of the piezo is not instantaneous once high voltage reaches the ceramics. I discuss the details of this actuator response delay and its effect on MOSFET turn-on time in more detail in a later section (see Figure 52 b) for initial insight). Because the ball remains airborne for a duration substantially exceeding the charging time, I deemed an activation span of 100 microseconds for Q1 sufficient, even though it surpasses the minimal necessary interval. Once Q1 is disengaged, the system enters a quiescent state, implemented by a while-loop.

The processor subsequently resides in an idle state until it perceives an impulse interrupt, signifying the detection of the ball. When this interrupt is received, the processor's forthcoming action is postponed for an interval as decreed by `timerDelays[n = 1]`, with 'n' equating to the number of detected ball bounces. For one bounce, this delay is eight microseconds, as indicated in the 'timerDelays' array initialization (related to Figure 44). Following the postponement, the lower-side MOSFET, Q2, is once again activated, which results in the discharging of the piezo and facilitates a bounce-free catching of the ball. After this sequence, the system is primed for a new round of menu selection.

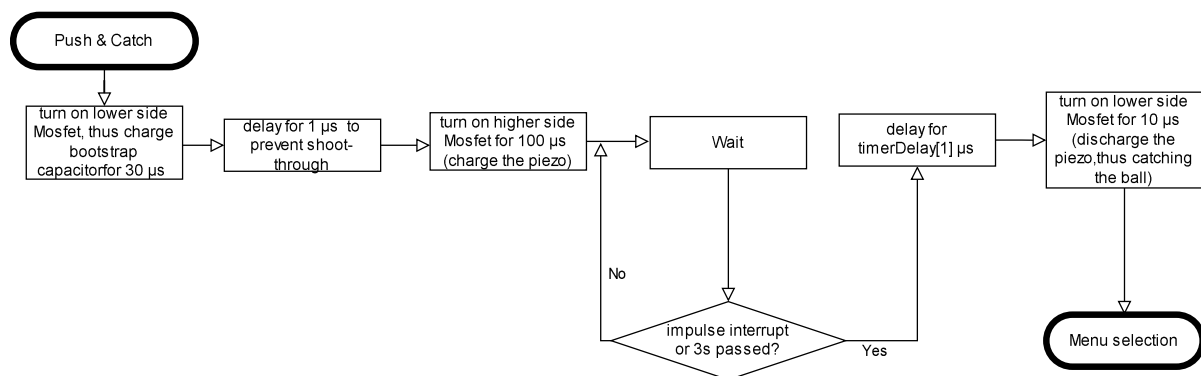


Figure 45 Flow chart of "Push and catch after one bounce"

2.6.2.7 Assessment of Device Operability

Upon receiving the assembled printed circuit board (PCB), I performed a thorough visual inspection of all solder connections to ensure structural integrity. After this examination, I soldered the remaining components to the board and checked their contact.

After soldering the components, I applied a 12V direct current to validate the PCB's functionality. I then scrutinized test points using a multimeter to verify voltage accuracies and an oscilloscope to examine signal transmissions from the microcontroller to the gate drivers and other elements. This comprehensive assessment ensured the PCB's proper operational performance.

2.6.3 Prototype

This section introduces the prototype I investigated: the piezo pendulum I produced. The following figures provide visual documentation of the prototype from various angles, thereby facilitating a comprehensive understanding of its design, functionality, and intricate details.

Figure 46 illustrates two aspects of the piezo pendulum. In image a), the top view of the pendulum is displayed alongside its control menu, where the user can select functions, enabling an examination of the device's user interface and surface design. Conversely, image b) provides a side view of the pendulum, offering insights into its structural design and dimensions.

In Figure 47 the focus shifts to different vantage points of the same device. Image a) shows a side view, emphasizing the pendulum's profile and the mechanical alignment. Meanwhile, image b) reveals the internal components of the piezo pendulum, providing a clear understanding of the device's construction and the layout of its inner workings.

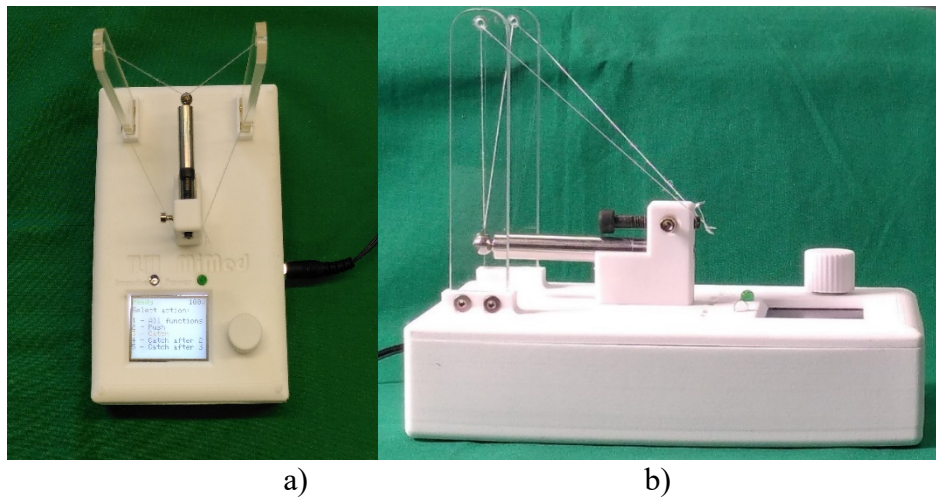


Figure 46 a) Piezo pendulum from top with view on menu b) Piezo pendulum from side

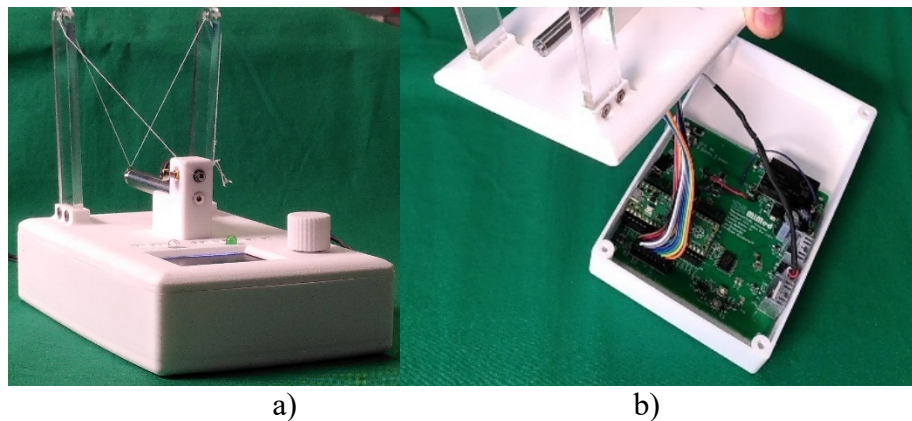


Figure 47 a) Side view of piezo pendulum; b) Piezo pendulum from the inside

3. Design Evaluation

This section now proceeds with a review of the goals. The original efforts aimed to create an improved piezoelectric pendulum suitable for the instructional needs at the Technical University of Munich. This enhanced pendulum was designed specifically for classes like Mechatronic Device Technology and Microtechnical Sensors and Actuators. The objectives included updating the electronic components of the pendulum and switching to a more accessible programming language from Assembler while maintaining all key functionalities. Additionally, I aimed to simplify operation for users by making the pendulum's features easier to access.

Upon completing the evaluation, I compared these enhancements with Klaus Moldovan's initial design. It was necessary to retain the essential features of Moldovan's pendulum, omitting any non-essential elements as needed. I intended the project to find optimal timing delays for efficient functioning through experimentation. Lastly, thorough documentation would accompany these improvements to assist in future work and to provide an academic resource. The achievement of these goals is addressed in the following subsections.

3.1 Modernization of the Pendulum: Comparative Hardware Analysis

I successfully updated the pendulum electronics, which enhanced performance and reliability. Table 7 provides a comparative overview of the hardware components used in Moldovan's piezoelectric pendulum and my newly developed design. This table highlights the careful consideration given to achieving a balance between performance, simplicity, cost-effectiveness, and suitability for practical applications. For visual reference, images of both pendulums are presented in Figure 48.



Figure 48 Moldovan's pendulum (left) and new pendulum (right) next to each other

Table 7 Comparison table between old and new pendulum

	Old Pendulum by Moldovan	New Pendulum
Piezo actuator	Piezosystem Jena PAHL 18/30 NV HL special Voltage range: 0-130V	PI P-820.30 by Physik Instrumente (PI) Voltage range: 0-100V
Microcontroller	SAB80C537, Siemens	RP2040, Raspberry Pi Pico
Programming language	Assembler	C++ with compatibility to the Arduino core framework
Architecture	8-bit	32-bit dual-core ARM Cortex-M0+
Clock Speed	12 MHz	133 MHz (over 11 times faster)
Delay precision (for catching of ball)	Theoretically 55.6ns using 18MHz oscillator circuit	Theoretically 16ns using 62.5 MHz hardware timer
Program Memory (ROM)	32 KB EPROM (limited)	2 MB Flash memory (much larger capacity)
Data Memory (RAM)	2 x 32 KB RAM	264 KB onboard SRAM (more for complex programs)
Display	3-½ Digit LCD Module • smaller • no color	Display DFRobot DFR0664 LCD • 2 inches • Color display
Input Method	Individual buttons for each key press	Rotary knob for turning (encoding) and a button press
Number of Inputs	16 (one for each button)	2 (one for encoder rotation, one for button press)
Communication	RS232 (Serial Port)	Micro USB
Power	12V DC Power supply or 4x 1.2V Rechargeable Battery Pack	12V DC Power supply
Method of driving the MOSFET gates	Two forward converter • galvanic isolation (only high side MOSFET) • more components • bigger • custom build transformer	IR2110STRPBF Gate Driver IC • no galvanic isolation • less components • smaller • readily available
Piezo Voltage	Flyback converter transformer 0 – 150 V custom build	DC/DC Converter Recom R12-100B 50 – 100V readily available
Casing	ABS, bought from Conrad piezo actuator is mounted on an external base	PLA, 3D printed piezo actuator is mounted on the control unit

3.2 Programming Language: From Assembler to Accessibility

I accomplished the shift from Assembler to a more accessible programming language without compromising essential functionalities. I developed the embedded system for the pendulum using Arduino, a C++ based development environment specifically designed for this domain. Alternatively PlatformIO could be used.

3.3 Retention and Rationalization of the Pendulum's Features

The new pendulum's construction involved thoughtful development. Transitioning from a traditional ABS case to a 3D printed exterior has allowed for quick iteration and the creation of a more visually appealing product. The omission of batteries simplifies the design, contributing to a smaller, lighter mechanism, although this choice sacrifices portability.

In terms of electrical design, the decision against galvanic isolation simplifies the circuitry and reduces the number of components needed. Furthermore, the piezo actuator's placement atop the control unit, rather than on a separate base, exemplifies the principle of a compact and straightforward design. This configuration eliminates the potential for cable damage, thereby enhancing the device's durability and maintaining the integrity of its function.

Regarding functionality, the updated pendulum operates with a smaller piezo actuator with less capacitance, working at a lower voltage than Moldovan's version. Consequently, I use a lighter ball to ensure the pendulum can still propel it to a significant height. This adaptation does not detract from the fundamental objectives of the device; rather, the pendulum maintains the intended functionality while offering other improvements.

Ultimately, the design revisions for the pendulum underscore a successful balance between innovation and functionality. I made each choice, from materials to electrical layout, to optimize the device's practicality and aesthetic appeal without compromising its core purpose.

3.4 Continuity of Core Functionalities

The project preserved the essential features of the piezoelectric pendulum originally developed by Klaus Moldovan, retaining its integrity as an instructional apparatus. The redesigned pendulum adheres to Moldovan's conceptual framework, functioning as a demonstrative device to exhibit various piezoelectric phenomena.

The pendulum I developed incorporates an identical suite of programmed operations to Moldovan's pendulum. These functions include:

1. *Shoot*: Launches the pendulum ball.
2. *Shoot and Catch after First Impulse*: Catches the ball as it returns.
3. *Shoot and Catch after Second Impulse*: Captures the ball after a second impulse.
4. *Shoot and Catch after Third Impulse*: Secures the ball post-third impulse.
5. *Resonance*: Propels the ball upon immediate impulse detection.
6. *Resonance after Second Impulse*: Projects the ball after the second impulse is detected.
7. *All Functions*: Cycles through all modes sequentially.

I determined the correct delays for the piezo actuator using a specific function, which is activated by long pressing the push button. This refinement facilitates optimal timing for the "Shoot and Catch" mechanisms, enabling a smooth, bounce-free interception of the pendulum ball.

The current settings for the catching delays are as follows, but may be further refined in the future:

- Shoot and Catch after First Impulse: 8 microseconds
- Shoot and Catch after Second Impulse: 5 microseconds
- Shoot and Catch after Third Impulse: 1 microsecond

Users can adjust the current delay settings by pressing and holding the pushbutton for two seconds when a menu item is highlighted. Although this functionality is under development, it introduces the potential for enhanced customization. To safeguard against accidental alterations, I deactivated this feature during standard operation, ensuring optimal values are preserved and not changed by accident. The complete source code is available in Appendix E.

3.5 Simplification of Activation Procedures

I realized operational simplification by reducing the steps needed to activate the pendulum's functions. This makes the system more user-friendly and approachable for students and faculty at the Technical University of Munich. There is no need for users to manually initiate the reset button upon system startup, or to read any instructions on a panel like in Figure 10 a). Users can now directly choose desired functions using the encoder and pushbutton combination.

The new user-friendly interface features direct access to functions, navigable via an encoder and selectable through an integrated button, streamlining user interaction. Figure 19 b) showed the GUI development goal; Figure 49 depicts the finalized menu layout. A green "ready" sign indicates device readiness, while the voltage level is presented in blue on the right. The selected function is clearly highlighted, distinguishing it from other options to enhance user clarity in selection.

While striving for readability on the two-inch display, I maximized the text size, which necessitated shorter function names. As a result, users may initially be unclear about the meanings of abbreviations such as "Catch" or "Resonance after 2." Despite potential ambiguity, I favored readability, and implemented expanded explanations upon selecting each function, facilitating immediate understanding of its purpose.

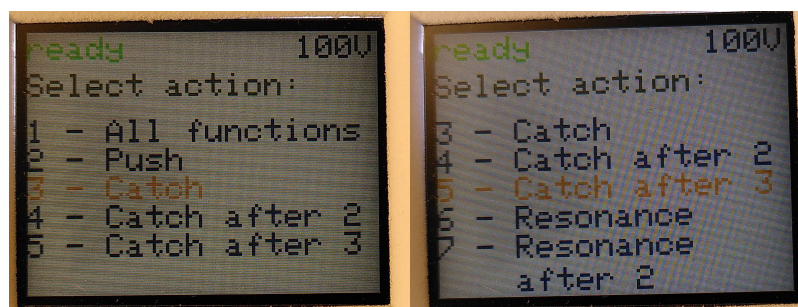


Figure 49 Final GUI

3.6 Fiscal Overview

The total expenditure, including value-added tax, was €800.76, as shown in Table 8. The initial sum of €99.72 covers the collective expenses of creating, assembling, and delivering five printed circuit boards (PCBs) from China within seven days. Costs also cover the assembled

parts on the boards upon arrival. Subsequent manual assembly of certain parts incurred an additional charge of €84.90. Additionally, purchasing the piezoelectric actuator contributed €616.14. Appendix C presents a comprehensive cost breakdown. It should be noted that this figure does not include the cost of components procured for developmental testing and contingencies, nor does it include costs for screws and thread inserts.

Table 8 Financial Summary

Description	Cost (EUR)
PCBs (Creation, Assembly, Delivery)	99.72
Manual Assembly	84.90
Piezoelectric Actuator	616.14
Total	800.76

3.7 Errors in Initial Design

The initial model conception had several unintentional errors, which I addressed and rectified in the subsequent version; refer to Appendix A for the detailed corrected schematic diagram.

Bootstrap diode

Regarding the bootstrap diode, I identified an underestimation of the required voltage tolerance; it did not accommodate the circuit's peak potential voltage of 110V when combining V_p and V_{cc} . Consequently, I replaced the original diode with an appropriate fast-switching diode capable of withstanding the circuit's voltage demand.

Voltage Divider for Reading Piezo Voltage

In the voltage divider configuration, used for reading piezo voltages, the initial connection to ground led to functional inaccuracies (Figure 50 a). For this reason, I had to attach two external resistors and connect a wire to the microcontroller to read the piezo voltage. In the amended design, I corrected this error (see Figure 50 b).

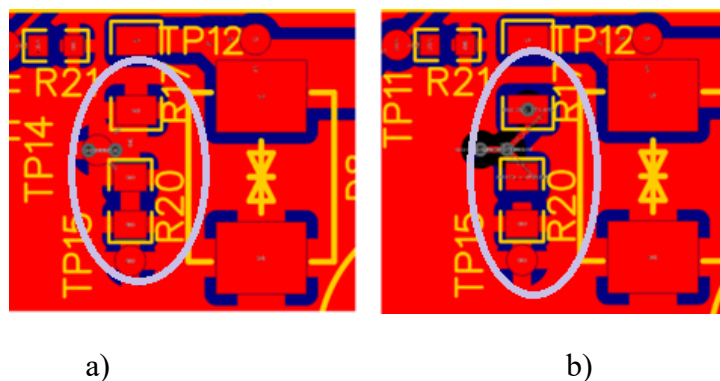


Figure 50 Voltage divider resistor mistake; a): the resistor R17 and R20 connected to ground. b) right solution

Controlling the Voltage of the Piezo

Maintaining stable voltage within the piezo power supply is crucial, as I discovered deviations using oscilloscope analysis (Appendix H discusses the incident further). Using the Raspberry Pi Pico's analog reference voltage to power the digital-to-analog converter (DAC) generated undesirable noise. I have since revised the design to provide a direct connection between the DAC and the voltage regulator (VDD), thus enabling a controllable and noise-reduced voltage range between 50V and 100V.

Figure 51 illustrates a connection using a red wire from VDD to the voltage divider, which bypasses the non-functional DAC and fixes the operation voltage at 100V. Despite this, a higher voltage is advantageous for achieving the greatest possible projectile launch height, according to equation (3). Nevertheless, alterations in voltage necessitate corresponding adjustments to the timing of the catch mechanism, as voltage variations are directly correlated with the maximum attainable height of the ball. The amended design has this error corrected, and the voltage is adjustable between 50 and 100 volts without the necessity for additional wiring.

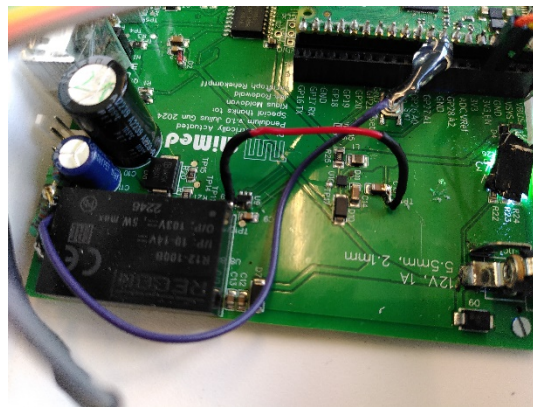


Figure 51 PCB modifications: Red wire is connecting VDD directly to the voltage divider. Lila wire is the voltage measuring wire , amended version works without wires

3.8 Summary of Results

This section reviewed my project goals to develop an improved piezoelectric pendulum for educational use at the Technical University of Munich, targeting courses such as Mechatronic Device Technology. Upgrades involved updating electronic components, transitioning to a more accessible C++ programming language, and simplifying the user interface. I made comparative analyses with Moldovan's design, ensuring the retention of essential functionalities while removing superfluous features.

4. Summary and Outlook

The implementation of the device successfully fulfills the initially established criteria for developing a piezoelectric pendulum with upgraded electronics. The upgraded electronics show advantageous improvements over Moldovan's version because of the use of a modern programming language and a user-centric design. The modernized pendulum system also ensures ease of operation and cost-efficiency for long-term academic use.

For prospective development of a piezoelectrically actuated pendulum, the following modifications could enhance overall system performance:

Amplifier

Future advancements could involve incorporating an amplifier for enhanced control over the piezo actuator. For example, it could create musical beats by producing louder sounds with longer pauses using high voltage, or softer sounds with shorter pauses using low voltage. This would be an exceptional feature for a piezoelectric demonstrator, especially because the ball can be caught at each instance.

Galvanic isolation

To mitigate the impact of high voltages from the piezo actuator on digital components, it is prudent to implement comprehensive electrical isolation. This can be achieved by using optocouplers or iCoupler devices, which facilitate partitioning signal channels, thereby enhancing signal purity. However, this method necessitates reengineering the piezo power supply and related elements. The current design incorporates two distinct grounding systems, GNDD (for digital parts) and GND (for common parts). This paves the way for possible future implementation of galvanic isolation.

Improving Efficiency in Ball Exchange with Swivel Clasp Integration

In the past, the process of exchanging the ball necessitated cutting the old string and securing a new one, followed by time-consuming adjustments to align the ball to the precise height of the piezo using a screw. Integrating a small, light swivel clasp, such as those used for jewelry, offers a more streamlined solution. This clasp can be affixed permanently to one end of the thread, while the opposite end is fashioned with a loop knot, simplifying the task of ball replacement. This method facilitates quick interchange of balls with varied weights, reducing downtime and increasing overall operational efficiency for research activities.

External Piezo Connector

I recommend incorporating an external plug-and-play interface for the piezo connection, situated on the device's casing. This provision would facilitate seamless interchangeability and the testing of various piezo actuators without the need for internal access, thus supporting a flexible and modular experimental setup. This would also make it easier to connect an oscilloscope during operation and to test the piezo actuator while the casing is open.

Experimental Optimization of Actuator Timing for Enhanced Ball-Catching

Subsequent investigations could explore how ball mass and applied voltage influence the piezo's requisite catchment delay. Systematic experimentation with a range of ball weights (e.g., 1g to 15g) and distinct voltage levels (50V, 75V, 100V) may yield analytical data from which a mathematical model of the relationship can be derived.

Muth et al., 1995, p. 699 used the setup of a steel ball hanging on a copper string and a piezoelectric actuator shown in Figure 52 a). They showed that there is a considerable delay between the time that high voltage reaches the piezo, and the start of the actual charging of the piezo, which shoots the ball away. They measured the contact time, which is the difference between the time when the piezo is switched on and the time when the ball leaves the actuator. The study was done for various ball weights. Figure 52 b) shows the contact time of the ball in relation to its weight.

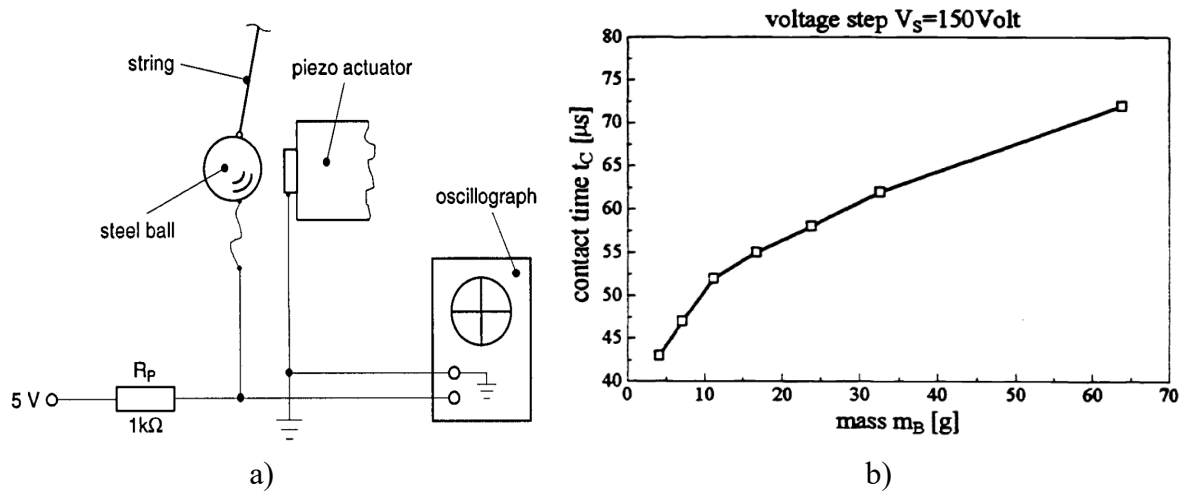


Figure 52 Left: Setup for measuring the contact time t_d ; right: contact time t_d as a function of the mass m_B (Images used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 699))

This system may facilitate an automated process to empirically identify the optimal delay for the piezoelectric device in catching a ball. In a hypothetical scenario, the piezoelectric actuator would eject the ball at two-second intervals, with a microcontroller substituting for an oscilloscope. This contrasts with the depicted setup in Figure 52 a), where the microcontroller (not an oscilloscope) would document the time of contact following the piezoelectric release after a short delay. The duration of the delay may be adjusted using a proportional-integral-derivative (PID) algorithm, thereby accelerating the identification of the optimal timing. Once the ball sticks to the piezoelectric material after launch, the microcontroller records the correct delay time. The system could then modify the voltage and begin a new cycle, establishing a range of successful catch delays across different voltages.

Calculated Optimization of Actuator Timing for Enhanced Ball-Catching

The timing of the actuator's response is of great importance in order to avoid premature or delayed force application, which disrupts the intended sticking effect of the ball. This time interval can be established through experimentation, as I did in this thesis, or by developing complex calculations. Discovering the precise moment for the bounce-free catch might be achieved by observing real-time voltage at the impact point. Such computations should consider the initial voltage, the ball's weight, and the piezoelectric material's response time. Muth et al., 1995, p. 699 already discussed one way to measure the piezoelectric delay for a setup like mine. Figure 52 a) shows the setup, and Figure 52 b) shows empirical values from that study. It is also vital to factor in the time it takes for the electronics to react for an accurate assessment.

Matlab and Simulink Integration

The integration of Matlab and Simulink with the pendulum facilitates the collection and analysis of data on impulse detection and enables precise control over the device's operation. Such integration permits the execution of tests involving various voltages, allowing for the automated determination of catch timing. The outcome is a refined understanding of the system's behavior and control, which is essential for elucidating the application of the piezoelectric phenomenon within the device.

Simultaneous Operation

Constructing an additional piezo pendulum, paired with an existing one via a WiFi chip, presents a viable method for synchronous communication. Opting for the Raspberry Pi Pico W, which inherently supports WiFi, facilitates this connectivity. Both pendulums could shoot and catch the ball simultaneously or in alternation, providing an engaging showcase of collaborative movement akin to the simultaneous operation mode demonstrated by Moldovan.

Moldovan's control system, beyond serving educational purposes, has demonstrated its capabilities in another practical setting. As illustrated by its deployment by Piezosystem Jena GmbH (Figure 53), the pendulum has been effectively used as a dynamic demonstrator. The photo shows two of Moldovan's piezo controls that have been expertly configured to facilitate a continuous back-and-forth motion of a ball. One unit launches the ball, which is then detected and swiftly halted by the control system on the opposing side, and subsequently propelled back in a reciprocal manner, creating a dynamic and ongoing exchange.



Figure 53 Demonstrator using two piezo controls developed by Klaus Moldovan (Image used with kind permission from Eike Sode (Piezosystem Jena GmbH))

References

- APC International (2024) [Online]. Available at <https://www.americanpiezo.com/piezo-theory/generators.html> (Accessed 14 May 2024).
- Arburg (2024) [Online]. Available at <https://www.arburg.com/de/de/produktwelt/additive-fertigung/freeformer/> (Accessed 14 May 2024).
- Bogdanović, F., Osrečki, Ž., Žilak, J., Koričić, M. and Suligoj, T. (2021) 'Comparison of Discrete Bipolar Transistors and MOSFETs for High-Speed Switching Application', *2021 44th International Convention on Information, Communication and Electronic Technology (MIPRO)*, pp. 22–27.
- Chiaberge, M., Tonoli, A., Botto, G. and M. De Giuseppe, S. Carabelli, F. Maddaleno (2009) 'Model and Design of a Power Driver for Piezoelectric Stack Actuators'.
- Chopra, I. and Sirohi, J. (2014) *Smart structures theory* [Online], New York NY USA, Cambridge University Press. Available at <https://www.cambridge.org/core/books/smart-structures-theory/EAA178BFD89113EA46BD95529272B74B>.
- Dosch, J. J., Inman, D. J. and Garcia, E. (1992) 'A Self-Sensing Piezoelectric Actuator for Collocated Control', *Journal of Intelligent Material Systems and Structures*, vol. 3, no. 1, pp. 166–185.
- Fischer, G. S., Cole, G. and Su, H. (2011) 'Approaches to creating and controlling motion in MRI', *Annual International Conference of the IEEE Engineering in Medicine and Biology Society. IEEE Engineering in Medicine and Biology Society. Annual International Conference*, vol. 2011, pp. 6687–6690 [Online]. DOI: 10.1109/IEMBS.2011.6091649.
- Gnad, G. (2005) *Ansteuerkonzept für piezoelektrische Aktoren*.
- Günther, D., Otnad, T. and Büscher, W. (2013) *Skript Praktikum Mikrotechnik V2.7, Lehrstuhl für Mikrotechnik und Medizingerätetechnik*, Technische Universität München.
- Heider, U. (1976) *Piezokeramik als Antriebselement in einem Nadeldrucker* /.
- Horowitz, P. and Hill, W. (2015) *The Art of Electronics*, 3rd edn, USA, Cambridge University Press.
- Kagerer, M. (2016) 'Mikrofertigung und Mikromontage zur Herstellung eines individuellen piezoelektrisch betriebenen Mikrotropfengerzeugers'.

- Kagerer, M., Eiler, K., Otnad, T., Irlinger, F. and Lueth, T. C. (2012) 'Piezo inkjet drop-on-demand experimentation platform manufactured with rapid prototyping techniques enabling future technologies'.
- Kaur, J. S. (2016) 'A Review on Piezoelectric Smart Road Sensor 1' [Online]. Available at https://www.academia.edu/70385897/A_Review_on_Piezoelectric_Smart_Road_Sensor_1.
- Li, H., Tong, Y. and Li, C. (2024) 'Modeling and Control of a Linear Piezoelectric Actuator', *Actuators*, vol. 13, no. 2, p. 55 [Online]. DOI: 10.3390/act13020055.
- Li, T., Chen, Y. and Ma, J. (2009) 'Development of a miniaturized piezoelectric ultrasonic transducer', *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control*, vol. 56, no. 3, pp. 649–659.
- Liseli, J. B., Agnus, J., Lutz, P. and Rakotondrabe, M. (2020) 'An Overview of Piezoelectric Self-Sensing Actuation for Nanopositioning Applications: Electrical Circuits, Displacement, and Force Estimation', *IEEE Transactions on Instrumentation and Measurement*, vol. 69, no. 1, pp. 2–14.
- Moldovan, K. (1993) 'Ansteuerung auf der Basis eines Mikrocontrollers fuer eine aktorisch-sensorisch genutzte piezoelektrische Keramik'.
- Moldovan, K. (1994a) *Das Piezo-Pendel*.
- Moldovan, K. (1994b) *Mikrocontrollersystem SAB80C517A*.
- Moldovan, K. (1994c) *Piezopendel-Betriebsanleitung*.
- Moussa, R. R., Ismaeel, W. S. and Solban, M. M. (2022) 'Energy generation in public buildings using piezoelectric flooring tiles; A case study of a metro station', *Sustainable Cities and Society*, vol. 77, p. 103555 [Online]. DOI: 10.1016/j.scs.2021.103555.
- Muth, M., Moldovan, K. and Goetz, B. (1995) 'Smart actuators: a novel technique for active damping', *PROCEEDINGS VOLUME 2443 Smart Structures and Materials 1995: Smart Structures and Integrated Systems*, pp. 692–702.
- Norman, D. (2013) *The design of everyday things: Revised and expanded edition* [Online]. Available at <https://scholar.google.com/citations?user=h-2jebwaaaaj&hl=en&oi=sra>.
- Rumschoettel, D., Kagerer, M., Irlinger, F. and Lueth, T. C. (2015) *Compact model for the characterization of a piezoelectric bend-mode droplet generator* [Online]. Available at https://www.researchgate.net/publication/282375780_Compact_model_for_the_characterization_of_a_piezoelectric_bend-mode_droplet_generator.
- Rumschoettel, D., Markus Kagerer, Franz Irlinger and Tim C. Lueth (2014) 'Compact model for the characterization of a piezoelectric bend-mode droplet generator'.
- Scherz, P. (2007) *Practical electronics for inventors: New sections on: test equipment, tools, remote cotnrol, object-oriented microcontrollers, modern computers, transducers, sample projects* [Online], 2nd edn, New York, McGraw-Hill. Available at <http://www.loc.gov/catdir/enhancements/fy0713/2007271320-b.html>.
- Schweidler, F. (2004) *Numerische Simulation eines Piezo-Pendels*, Technische Universität München.
- Seyfert, L., Ammar, B. H. and Norbert, S. (2021) 'Contribution of flexoelectricity in piezo-electric PVDF films for EH applications', *Active and Passive Smart Structures and Integrated Systems XV*. Online Only, United States, 3/22/2021 - 3/27/2021, SPIE / International Society for Optical Engineering, p. 11.
- Shrout, T. R. and Zhang, S. J. (2007) 'Lead-free piezoelectric ceramics: Alternatives for PZT?', *Journal of Electroceramics*, vol. 19, no. 1, pp. 113–126 [Online]. DOI: 10.1007/s10832-007-9047-0.

Tietze, U., Schenk, C. and Gamm, E. (2016) *Electronic Circuits: Handbook for Design and Application*, 2nd edn, Springer Publishing Company, Incorporated.

Wang, Y., Cole, G. A., Su, H., Pilitsis, J. G. and Fischer, G. S. (2009) 'MRI compatibility evaluation of a piezoelectric actuator system for a neural interventional robot', *Annual International Conference of the IEEE Engineering in Medicine and Biology Society. IEEE Engineering in Medicine and Biology Society. Annual International Conference*, vol. 2009, pp. 6072–6075 [Online]. DOI: 10.1109/IEMBS.2009.5334206.

Wang, Y.-S., Chen, C.-Y. and Lin, C.-H. (2015) 'Microchip based ion source powered by a lighter piezo-generator for mass-spectrometry detection', pp. 128–131.

Table of Figures

Figure 1 Energy harvesting floor at the entrance to the Chair for Construction Chemicals, Technical University of Munich.....	7
Figure 2 Structure and contacting of a stacked piezo actuator (Image used with kind permission from PI Ceramic GmbH, available at https://www.piceramic.com/en/expertise/piezo-technology/properties-piezo-actuators/electrical-operation [Accessed 14 May 2024]).....	10
Figure 3 Linear piezoelectric actuator in cut view. The ball hits the hardened head on the right side. (Image used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 699)).....	10
Figure 4 Schematic of parasitic resistance and capacitance of piezo actuator, forming a low-pass filter.....	11
Figure 5 Simulated stress of the piezo actuator on in contact with a ball in the axial direction (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 42)).....	12
Figure 6 Voltage across piezo during charging and discharging with exemplary values.....	13
Figure 7 Schematic of piezo actuator and pendulum (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 23)).....	13
Figure 8 The characteristic curve of the force F_b during the first contact of the ball with the piezoceramic. The sketches above the curve show the position of the ball at each point in time. (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 12)).....	14
Figure 9 Representation of the piezo pendulum engineered by Klaus Moldovan. Ball on a string (1), piezo actuator (2), piezo holder (3), piezo base (4), cable connection to the control unit (5), control unit incl. display (6) and buttons (7).....	16
Figure 10 a) Piezo pendulum engineered by Klaus Moldovan with later wooden panel modification Right: Control of the piezo pendulum engineered by Klaus Moldovan without wooden panel.....	16
Figure 11 Basic view of the parts in the piezo pendulum engineered by Klaus Moldovan (Image used with kind permission from Klaus Moldovan (Moldovan, 1993, p. 8)).....	17
Figure 12 Hardware structure of the piezo pendulum engineered by Klaus Moldovan (Image used with kind permission from Klaus Moldovan (Moldovan, 1994a, p. 3)).....	17
Figure 13 Automatic time delay configuration of the piezo pendulum engineered by Klaus Moldovan (Image used with kind permission from Klaus Moldovan (Moldovan, 1994a, p. 11)).....	18
Figure 14 Experimental setup of the piezo pendulum according to Heider (Heider, 1976, p. 15) (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 5)).....	20
Figure 15 Three-dimensional representation of piezoceramic(left) and steel ball (right) in the correct size ratio (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 28)).....	20
Figure 16 Two-dimensional representation of stress in the piezoceramic at impact of steel ball. The compressive stress at the free end (right) reaches its first maximum. The ball is (approximately) stationary in this moment. (Image used with kind permission from Florian Schweidler (Schweidler, 2004, p. 32)).....	20
Figure 17 a) Velocity v_{BI} of the steel ball as a function of its mass m_B (Image used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 698)); b) Electromechanical efficiency η as a function of the steel ball mass m_B (Image used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 698)).....	21
Figure 18 Piezo control of Moldovan from the inside.....	22

Figure 19 a)Concept of new pendulum. Ball on string (1), piezo actuator (2), piezo holder (3), piezo control unit(4), LCD display (5), encoder with push-functionality (6); b)Basic structure of the GUI.....	24
Figure 20 Structure of communication between control unit and the actuator and sensors.....	25
Figure 21 Flow chart of simplified menu selection process.....	25
Figure 22 Activity diagram of a simplified program for catching a ball bounce-free.....	26
Figure 23 Raspberry Pi Pico.....	28
Figure 24 Voltage curve of P-820.30 actuator when charging.....	31
Figure 25 Way of current when controlling the MOSFETs in a simple full-bridge setup; a) Piezo is charging; b) Piezo is discharging.....	32
Figure 26 Way of current when controlling the MOSFETs in a half-bridge setup; a) Piezo is charging, only Q1 active; b) Piezo is discharging, only Q2 active; c) Way of current at destructive shoot-through when Q1 and Q2 active.....	33
Figure 27 Simple bootstrap circuit.....	34
• Figure 28 Simplified hardware overview of functional model.....	38
• Figure 29 Equipment setup: How the parts are connected to one another.....	38
Figure 30 Gatedriver circuit.....	42
Figure 31 Impulse detection circuitry (based on (Moldovan, 1994a, p. 12)).....	43
Figure 32 Piezo power supply.....	44
Figure 33 Voltage at the piezo V_p over the the input voltage V_{ADJ} , the voltage that is provided after the voltage divider at Testpoint 11 in Figure 32.....	44
Figure 34 Role of buffer capacitors in charging the piezo; a) Piezo is not charging; b) Piezo is charging.....	45
Figure 35 Power jack.....	45
Figure 36 DC/DC converter - 12V to 3.3V.....	46
Figure 37 Power connection to Raspberry Pi Pico.....	46
Figure 38 Display connections.....	47
Figure 39 Encoder with pushbutton.....	47
Figure 40 Connector for Display, LEDs on front & Encoder.....	47
Figure 41 Ball with soldered copper wire.....	47
Figure 42 Iterations of the 3D printed cases.....	48
Figure 43 Thread supports.....	48
Figure 44 Thread holding screw.....	Error! Bookmark not defined.
Figure 45 Flow chart of menu selection process.....	49
Figure 46 Flow chart of “Push and catch after one bounce”.....	50
Figure 47 a) Piezo pendulum from top with view on menu b) Piezo pendulum from side.....	51
Figure 48 a) Side view of piezo pendulum; b) Piezo pendulum from the inside.....	51
Figure 49 Old pendulum and new pendulum next to each other.....	52
Figure 50 Final GUI.....	55
Figure 51 Voltage divider resistor mistake; a): the resistor R17 and R20 connected to ground. b) right solution.....	56
Figure 52 PCB modifications: Red wire is connecting VDD directly to the voltage divider. Lila wire is the voltage measuring wire , amended version works without wires.....	57
Figure 53 Left: Setup for measuring the contact time t_d ; right: contact time t_d as a function of the mass m_B (Images used with kind permission from Klaus Moldovan (Muth et al., 1995, p. 699)).....	59
Figure 54 Demonstrator using two piezo controls developed by Klaus Moldovan (Image used with kind permission from Eike Sode (piezosystem jena GmbH)).....	60
Figure 55 Schematic Mircocontroller Connections, Display.....	66
Figure 56 Schematic Piezo Power Supply.....	67
Figure 57 Schematic Gatedriver.....	68

Figure 58 Schematic Impulse Detection (based on (Moldovan, 1994a, p. 12)).....	69
Figure 59 Schematic Power Supply step-down 12V to 3.3V.....	71
Figure 60 Schematic Encoder and LED's.....	72
Figure 61 PCB layout Top.....	72
Figure 62 PCB layout Bottom.....	73
Figure 63 Flow chart of program selection process.....	78
Figure 64 Technical drawing of piezo pendulum (not for scale).....	98
Figure 65: Oscilloscope Readings - Yellow (TP11) and Blue (TP12). The DAC is set at 3.3V, yielding a Vp of 100V. VADJ displays a sawtooth waveform fluctuating between -5V and 10V. VADJ frequency remains stable without auditory disturbances.....	99
Figure 66 Oscilloscope Reading - Yellow (TP11) shows a Vp of 64V; Blue (TP12) corresponds to the DAC output nominally set to 0V but actually at 0.65Vrms, resulting in a consistent Vp of 64V. Variable VADJ frequency is observed alongside audible high-pitched tones from the Recom DC converter.....	99
Figure 67 Oscilloscope Reading - a) Yellow (TP11) indicates a Vp of 64V; Blue traces the I2C DAC VOUT prior to R19, with the DAC output configured to 0V. b) Yellow (TP11) reflects a Vp of 52V; Blue (TP12) with VADJ connected to ground via a cable, showing noise elimination, ideal performance, and the absence of auditory signals.....	100



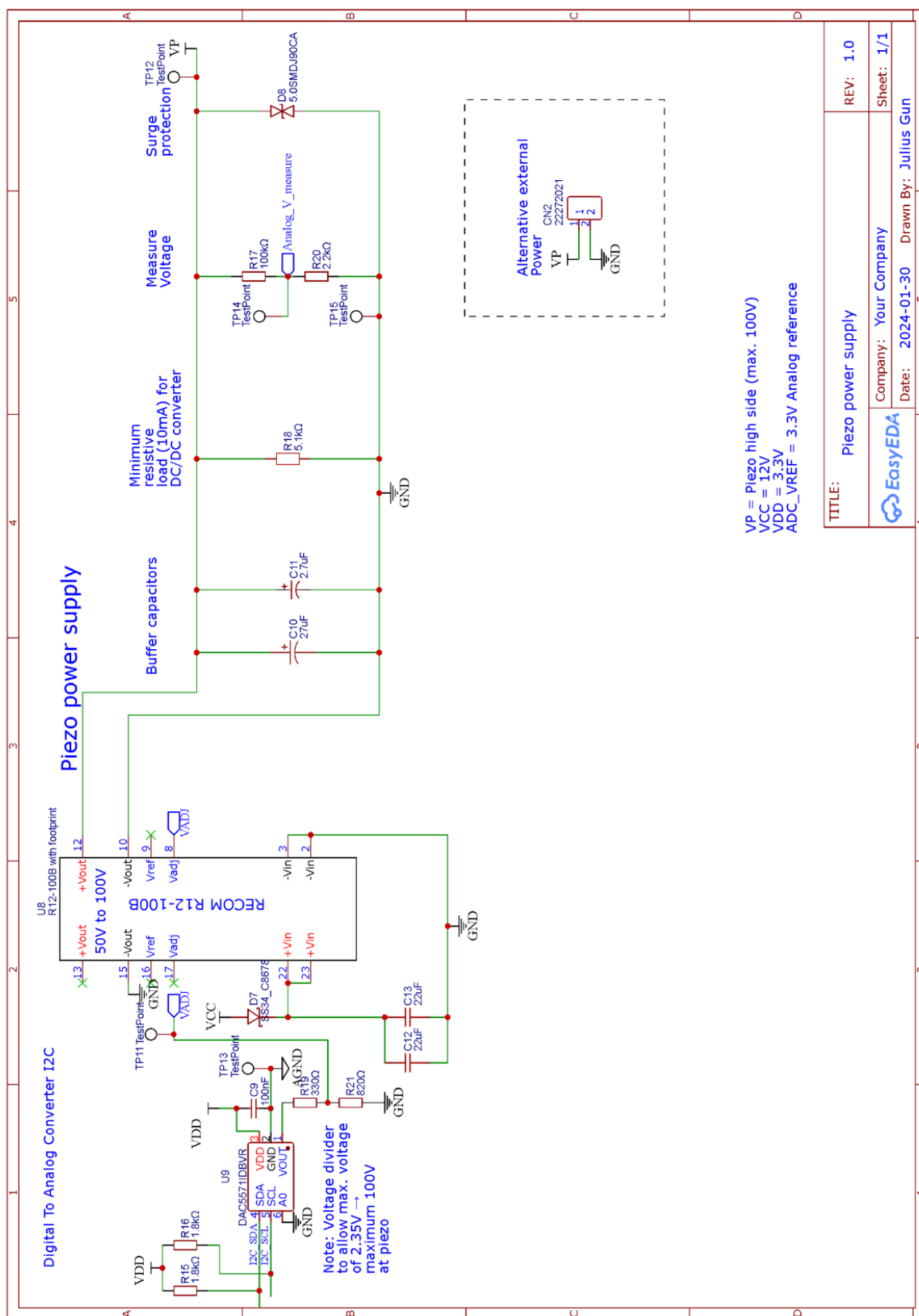


Figure 55 Schematic Piezo Power Supply

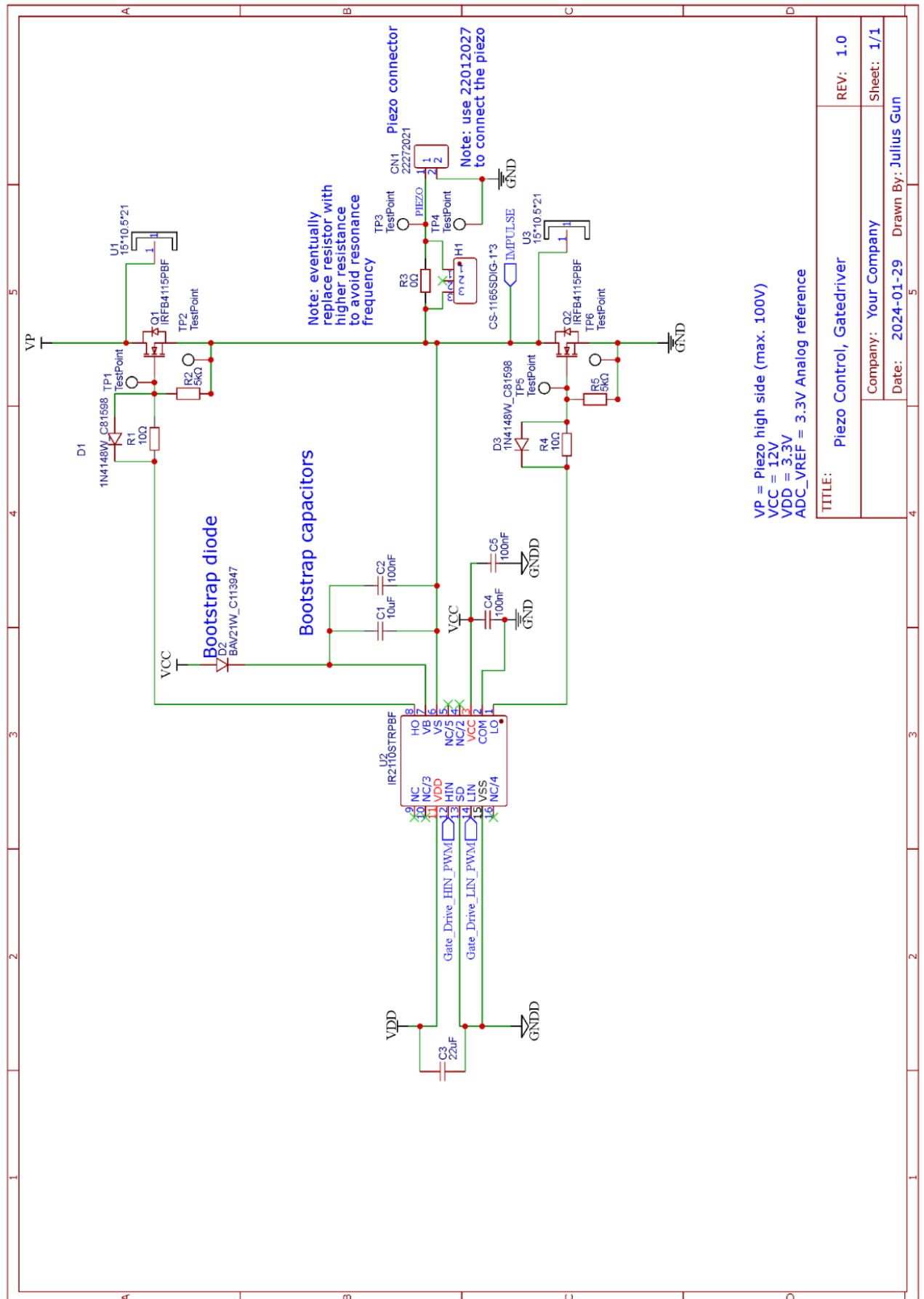


Figure 56 Schematic Gatedriver

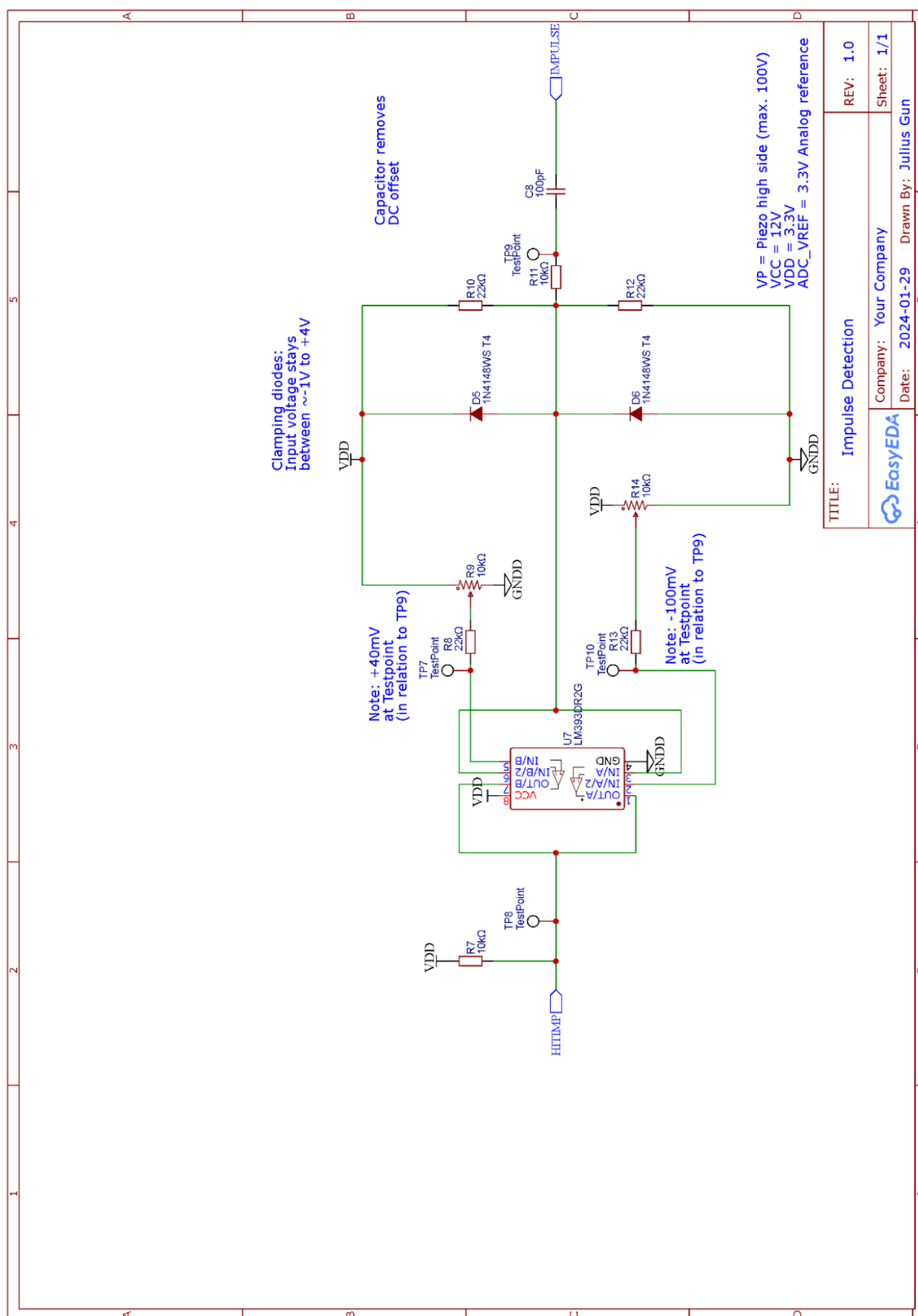


Figure 57 Schematic Impulse Detection (based on (Moldovan, 1994a, p. 12))

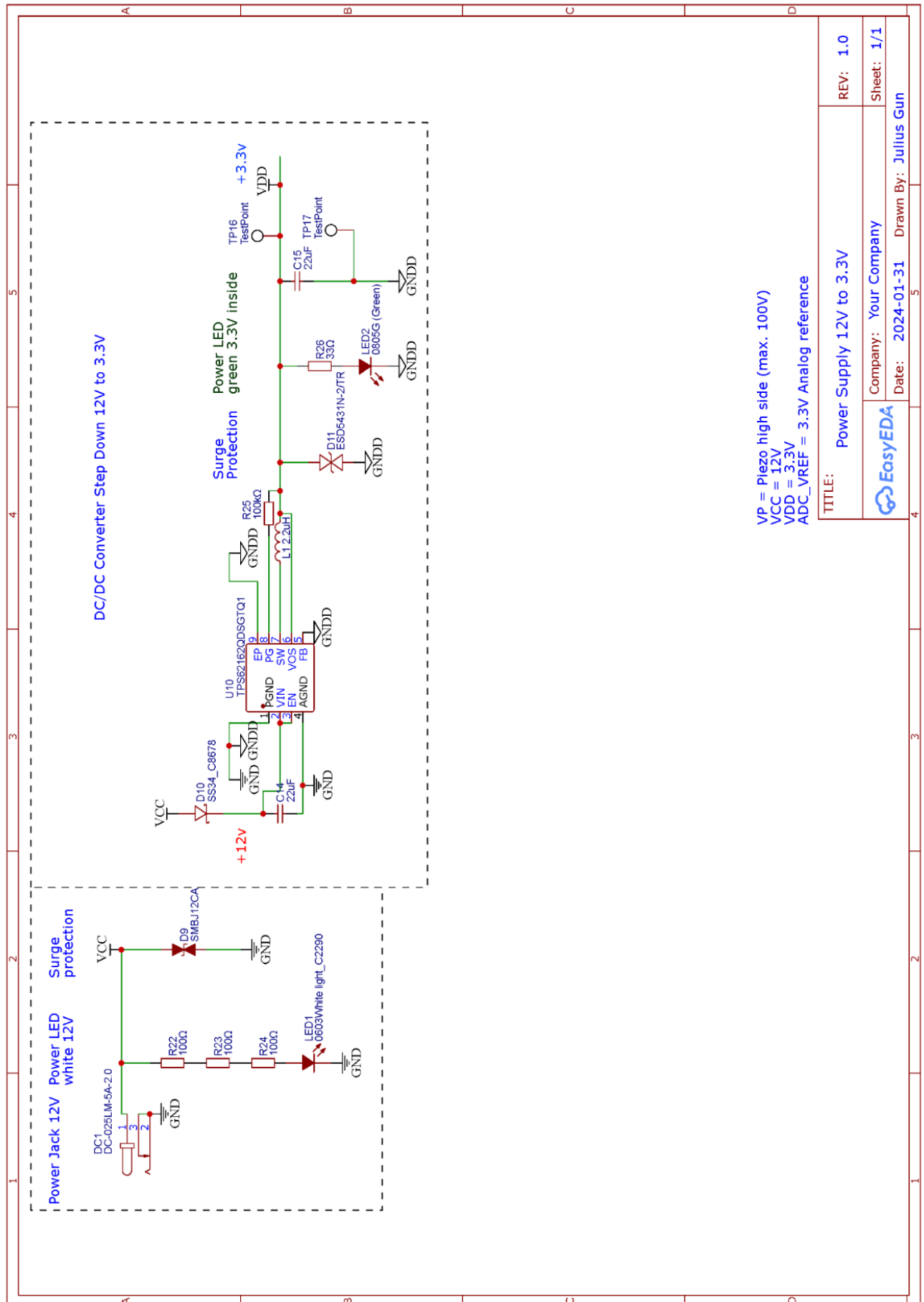


Figure 58 Schematic Power Supply step-down 12V to 3.3V

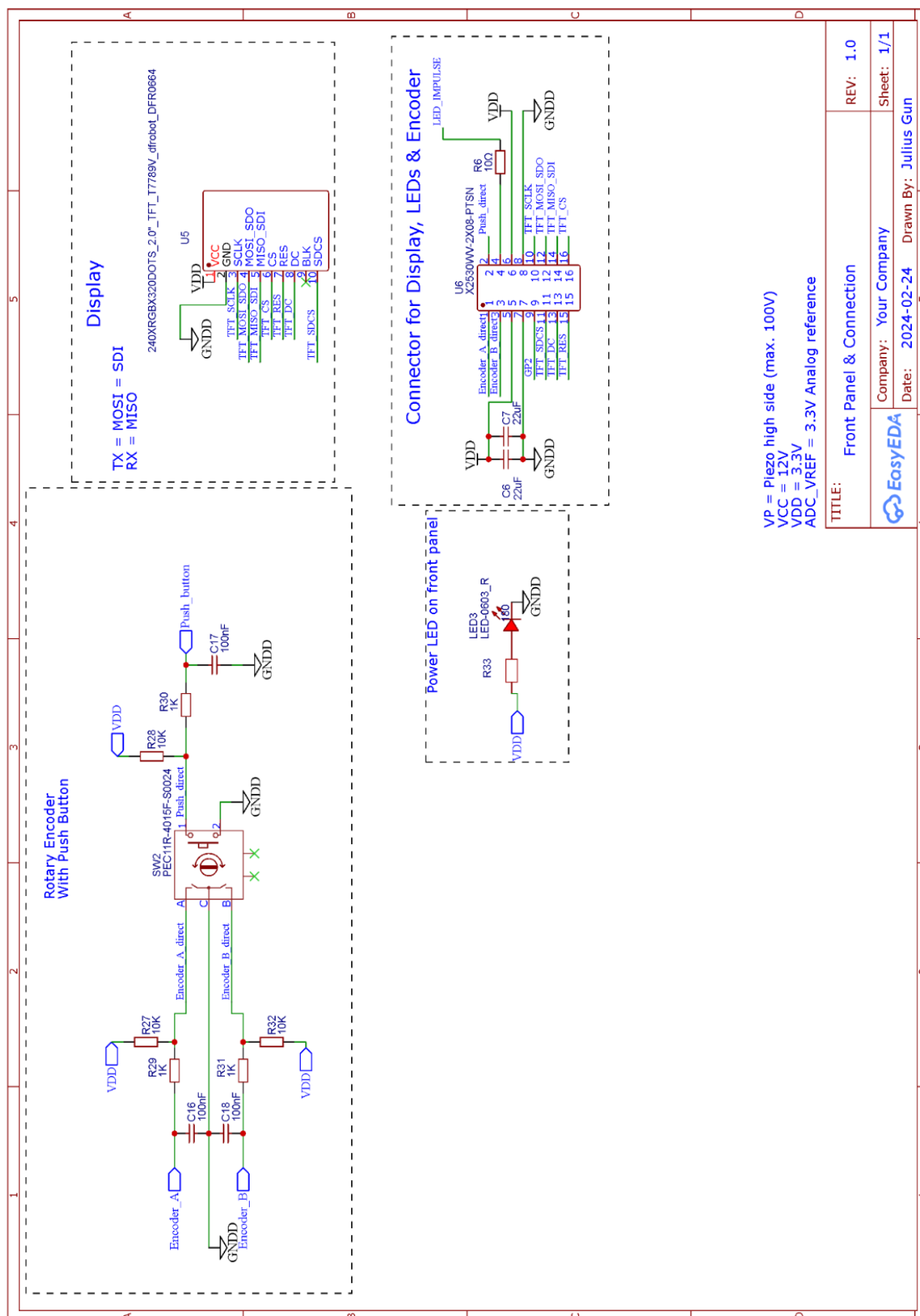


Figure 59 Schematic Encoder and LED's

B PCB layout

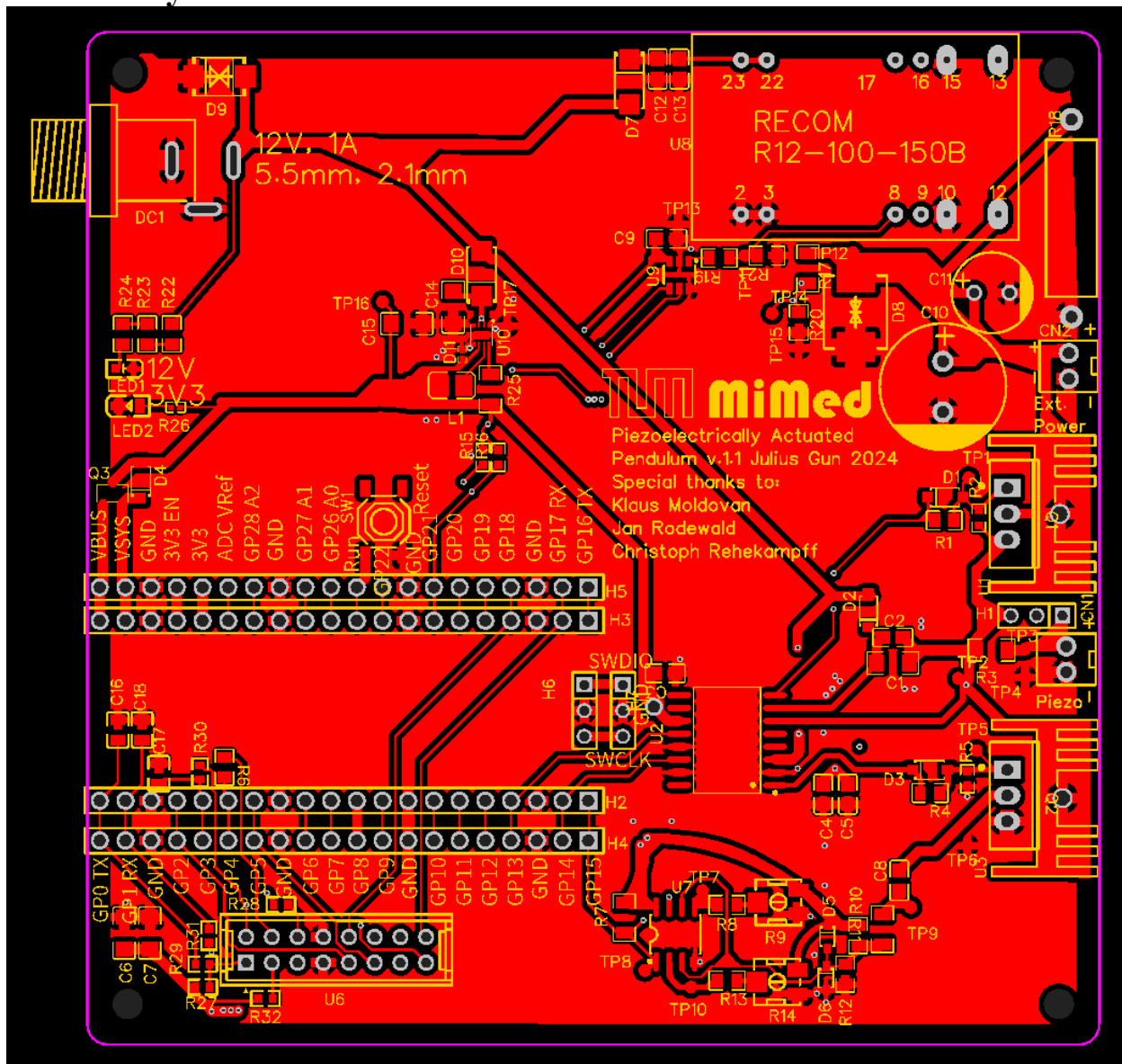


Figure 60 PCB layout Top

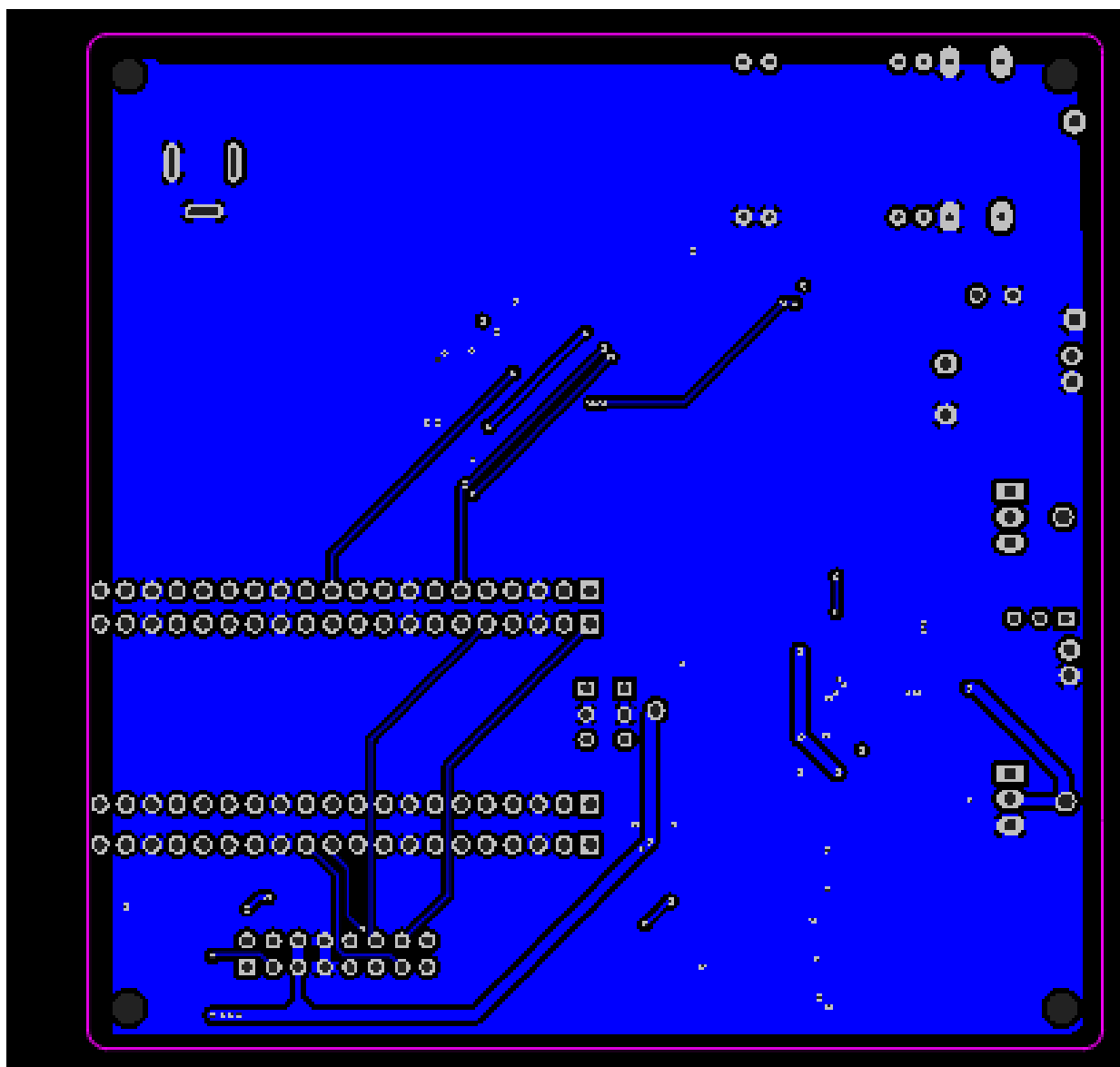


Figure 61 PCB layout Bottom

C Bill Of Materials

This table presents a comprehensive cost breakdown. The total expenditure, inclusive of value-added tax, was **800.76€** as can be seen in Table 8. The initial sum of 99.72€ accounts for the collective expenses of creating, assembling, and delivering five printed circuit boards (PCBs) from China within a seven-day period. Costs also cover the assembled parts on the boards upon arrival. Subsequent manual assembly of certain parts incurred an additional charge of 84.90€. Additionally, the purchase of the piezoelectric actuator contributed 616.14€. It should be noted that this figure does not include the cost of components procured for developmental testing and contingencies, nor does it include costs for screws and thread inserts.

Table 9 Financial Summary

Description	Cost (EUR)
PCBs (Creation, Assembly, Delivery)	99.72
Manual Assembly	84.90
Piezoelectric Actuator	616.14
Total	800.76

JLCPCB PCB assembly

Quantity	Name	Designator	Footprint	Manufacturer	Supplier Part	Total Price in €, VAT included.
5	10uF	C1	C1206	SAMSUNG(三星)	C13585	0.15
20	100nF	C2,C4,C5, C9	C0805	SAMSUNG(三星)	C28233	0.15
15	22uF	C3,C12,C13	C0805	SAMSUNG(三星)	C45783	0.4
20	22uF	C6,C7,C14,C15	C1206	SAMSUNG(三星)	C12891	1
5	100pF	C8	C0805	FH(风华)	C66331	0.05
5	27uF	C10	CAP-TH_BD12.5-P5.00-D0.6-FD	rubycon(红宝石)	C441634	2.8
5	2.7uF	C11	CAP-TH_BD8.0-P3.50-D0.6-FD	AISHI(艾华集团)	C106386	0.35
15	100nF	C16,C17, C18	C0805	YAGEO	C49678	0.1
5	22272021	CN2	CONN-TH 22272021	MOLEX	C293316	0.8
15	1N4148W_C81598	D1,D3,D4	SOD-123F_L2.7-W1.6-LS3.8-RD	ST(先科)	C81598	0.15
5	1N5819WS	D2	SOD-323_L1.8-W1.3-LS2.5-RD	Hottech(合科泰)	C191023	0.05
10	1N4148WS T4	D5,D6	SOD-323_L1.8-W1.3-LS2.5-RD	CJ(江苏长电/长晶)	C2128	0.1
10	SS34_C8678	D7,D10	SMA_L4.3-W2.6-LS5.2-RD	MDD	C8678	0.25
5	5.0SMD-J90CA	D8	SMC_L6.9-W5.9-LS7.9-BI	Brightking(君耀电子)	C522717	2.25
5	SMBJ12CA	D9	SMB_L4.3-W3.6-LS5.4-BI	Brightking(君耀电子)	C78407	0.25
5	ESD5431N-2/TR	D11	DFN1006-2L-BI	WILLSEMI(韦尔)	C239673	0.1
5	DC-025LM-5A-2.0	DC1	DC-IN-TH_DC-025LM-5A-2.0	XKB Connectivity(中国星坤)	C381107	2.5
15	CS-1165SDIG-1*3	H1,H6,H7	HDR-TH_3P-P2.54_CS-1165SDIG	CST(永丰盈)	C2886705	0.95
20	22850120A	H2,H3,H4	HDR-TH_20P-P2.54-V-F	JILN(锦凌)	C429947	3

Quantity	Name	Designator	Footprint	Manufacturer	Supplier Part	Total Price in €, VAT included.
	NG1SYA01	,H5				
5	2.2uH	L1	L2520	chilisin(奇力新)	C280577	0.2
5	0603White light C2290	LED1	LED0603-R-RD_WHITE	KENTO	C2290	0.05
5	0805G (Green)	LED2	LED0805-R-RD	KENTO	C2297	0.05
5	SI2301CDS -T1-GE3	Q3	SOT-23-3_L3.0-W1.7-P0.95-LS2.9-BR	VISHAY(威世)	C10487	0.35
15	10Ω	R1,R4,R6	R0805_NEW	UNI-ROYAL(厚声)	C17415	0.05
10	5kΩ	R2,R5	R0603	Viking(光颀)	C2828621	0.3
5	0Ω	R3	R1206	UNI-ROYAL(厚声)	C17888	0.01
10	10kΩ	R7,R11	R1206	UNI-ROYAL(厚声)	C17902	0.05
20	22kΩ	R8,R10,R12,R13	R0805	UNI-ROYAL(厚声)	C17560	0.05
10	10kΩ	R9,R14	RES-ADJ-SMD_3P-L4.5-W4.5-P2.35-BL-FS	BOURNS	C780127	5.45
10	1.8kΩ	R15,R16	R0603	UNI-ROYAL(厚声)	C4177	0.03
10	100kΩ	R17,R25	R1206	UNI-ROYAL(厚声)	C17900	0.05
5	5.1kΩ	R18	RES-TH_BD5.0-L15.5-P19.50-D0.7	VO(翔胜)	C479363	0.3
5	33kΩ	R19	R0805	UNI-ROYAL(厚声)	C17633	0.03
5	2.2kΩ	R20	R0805	UNI-ROYAL(厚声)	C17520	0.03
5	82kΩ	R21	R0603	UNI-ROYAL(厚声)	C23254	0.03
15	100Ω	R22,R23,R24	R0805	UNI-ROYAL(厚声)	C17408	0.05
5	33Ω	R26	R0402	UNI-ROYAL(厚声)	C25105	0.03
15	10K	R27,R28,R32	R0603	UniOhm	C25804	0.04
15	1K	R29,R30,R31	R0603	UniOhm	C21190	0.04
5	TS-1187A-B-A-B	SW1	SW-SMD_4P-L5.1-W5.1-P3.70-LS6.5-TL-2	XKB Connectivity(中国星坤)	C318884	0.1
5	IR2110STR-PBF	U2	SOIC-16_L10.3-W7.5-P1.27-LS10.3-BL	Infineon(英飞凌)	C2968	5.95
5	PCB		PCB manufacture & assembly & shipping & all parts above included in the final price	JLCPCB		99.72

As can be seen the total price for five PCB boards was **99.72€**, including manufacturing of the PCB, assembly, shipping from China (one week) and all the parts that were already assembled when the boards arrived.

Parts that were assembled later manually:

Quantity	Name	Designator	Footprint	Manufacturer	Supplier Part	Total Price in €, VAT included.
1	PEC11R-40 15F-S0024	SW2	SW-TH_PEC11R-4XXXXF-SXXXXX	BOURNS	C143789	2.55
1	X2530WV- 2X08-PTSN	U6	CONN-TH_16P-P2.54_XKB- CONNECTIVITY_X2530WV-2X0 8-PTSN	XKB Connectivity(中国星 坤)	C2882573	0.65
1	LM393- DR2G	U7	SOIC-8_L5.0-W4.0-P1.27-LS6.0- BL	onsemi(安森 美)	C7955	0.19
1	R12-100B with foot- print	U8	RECOM_DC/DC_CONVERT- ER_R12-B	RECOM Power	945-2050-5- ND	44.35
1	DAC5571I DBVR	U9	SOT-23-6_L2.9-W1.6-P0.95- LS2.8-BR	TI(德州仪器)	C24061	0.99
1	TP- S62162QD SGTQ1	U10	WSON-8_L2.0-W2.0-P0.50-TL-EP	TI(德州仪器)	C2072257	1.36
1	Molex connector		P=2.54mm	MOLEX	C185230	0.04
2	IRF- B4115PBF	Q1, Q2	TO-220	Infineon Technologies	C2651	2.50
2	Heatsink 15.5*10.5*2 1	U1, U3	-	XSD	C4649	0.16
1	PEC11R-40 15F-S0024	SW2	Plugin Rotary Encoders ROHS	BOURNS	C143789	2.55
1	TJ-L5FYTHBTCGS- FLC4V-A5		Purple 5mmâ€†â½¢ç ¯â€² Plugin,P=2.54mm Light Emitting Diodes (LED) ROHS	TOGIALED	C20613334	0.08
1	X2530H-2x 08-PT	U6	-25â„ƒ~+85â„ƒ,f 2.54mm 2x8P 8 2 2.54mm ä, â„ P=2.54mm Rectan- gular Connectors Housings ROHS	XKB Connec- tion	C2882596	0.14
1	X2530WV- 2x08-PTSN	U6	2x8P 16P 2 Tin 2.54mm 2.54mm 8 -	XKB Connec- tion	C2882573	0.43
1	400BXW27 MEFR12.5 X20	C10	27uF 400V Â±20% 850mA@100kHz Plugin,D12.5xL20mm Aluminum Electrolytic Capacitors - Leaded ROHS	Rubycon	C441634	0.56
1	MF3WS-5K 1Â±1%- 9T73	R18	Metal Film Resistors 5.1kÎ© 3W Â±100ppm/â„ƒ,f Â±1% Plugin Through Hole Resistors ROHS	VO	C479363	0.06

Quantity	Name	Designator	Footprint	Manufacturer	Supplier Part	Total Price in €, VAT included.
1	22272021	CN1	1x2P 2P 1 2.54mm 2 Brass Plugin,P=2.54mm Wire To Board / Wire To Wire Connector ROHS	MOLEX	C293316	0.17
1	CS-1165SD IG-1*3	DC1	2.54mm Plugin 3A 3P -40â,,f~+105â,,f 250V Straight 1x3P 8.5mm Top Square Holes Plugin,P=2.54mm Female Headers ROHS	CST	C2886705	0.05
1	EGW2G- M2R7F12O T	C11	2.7uF 400V $\hat{A} \pm 20\%$ Plugin,D8xL12mm Aluminum Electrolytic Capacitors - Leaded ROHS	AISHI(Aihua Group)	C106386	0.07
1	DFR0664	U5	Display 2"LCD-Anzeige 3,3-5VDC SPI 35,7x60mm Farbe: RGB DFROBOT	DFROBOT		20.03
1	P-820.30		Linear piezo actuator PI P-820.30	Physik Instru- mente (PI)		616.42
1	HNP12- 120L6		Steckernetzteil, Festspannung 12 V/DC 1000 mA 12 W Stabilisiert	HN Power		7.97
Subtotal						701.32
Total:						<u>801.04€</u>

D Flowcharts

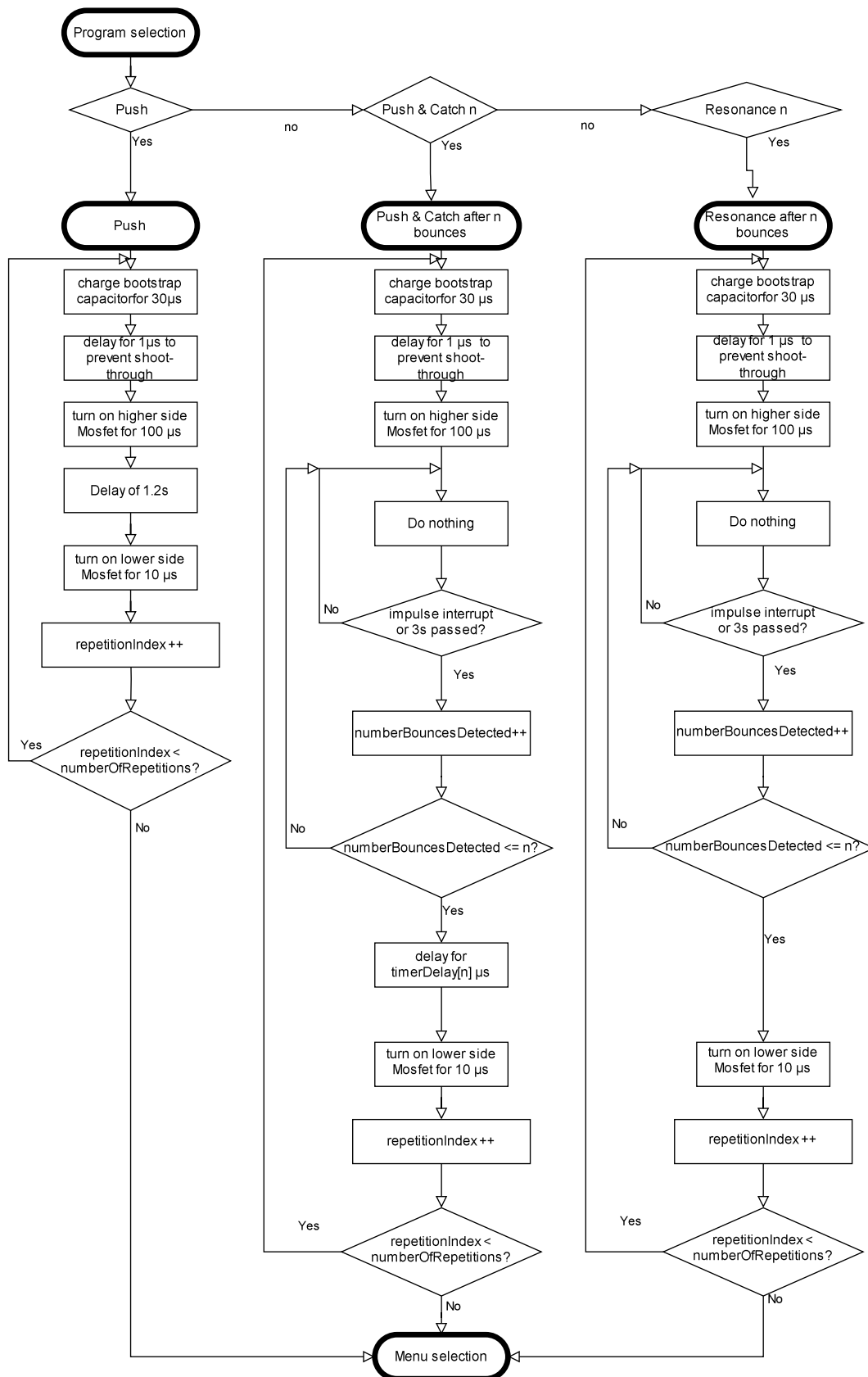


Figure 62 Flow chart of program selection process

E Code

An updated version the code file will be available at:
<https://github.com/julius-gun/piezopendulum>

```

/*!
 * @file piezoPendulum.ino
 * @brief This script is the main script to use the Piezo pendulum. The Raspberry Pi Pico RP2040 controls
the display, reads the encoder and button inputs and controls the piezostack.
 * @n This demo supports at least Raspberry Pi Pico. Other boards are not tested.
 * @authors [Julius Gun, Jan Rodewald] //Jan Rodewald developed the first version V0.1
 * @version V1.2
 * @date 2024-06-14
 * https://github.com/julius-gun/piezopendulum
 */

// #include <Arduino.h>
//
-----
// --- Read the libraries
-----
-
//
-----
-----
#include "pico/stdlib.h"
#include "hardware/irq.h" // might be needed to access lower-level interrupt functions
#include "DFRobot_GDL.h" // official Display Library
// #include <DAC7571.h> // digital to analog converter library from
https://github.com/jrj12591/DAC7571/
#include "src/DAC7571.h"
#include <SD.h>
#include <Wire.h>
#include <SPI.h>
#include <ButtonControl.h>
//
-----
-----
#define SYSTICK_FREQUENCY 133000000UL // 133MHz for RP2040
//
-----
-----
// --- Define the debug level
-----
// RPI_PICO_Timer Timer; // Instantiate a timer object

// --- Initialize DAC5571 with default I2C address 0x4C → A0 connected to
GND-----

```

```

//
-----
-----
DAC5571 dac;
#define I2C_SDA 20
#define I2C_SCL 21
#define I2C_PORT i2c0
const float VDD = 3.3; // Set the voltage that the DAC will run at
// Voltage regulation
float currentVoltage;
//
-----
-----
//          ---          Define          fixed          variables
-----
//
-----
-----
// Display
#define TFT_DC 6
#define TFT_RST 7
#define TFT_MISO 16
#define TFT_CS 17
#define TFT_SCLK 18
#define TFT_MOSI 19
// Analog read Voltage
#define V_MEASURE 27
// Input pins definitions
#define ENCODER_A 1
#define ENCODER_B 0
ButtonControl PUSH_BUTTON(3);
#define HIT_IMPULSE 15 // Impulse detection
// Output pins definitions
#define LED_IMPULSE 4
#define LED_POWER 2
#define GATE_DRIVE_HIN 13
#define GATE_DRIVE_LIN 14
// Colors in RGB565 format
#define COLOR_BLACK 0x0000
#define COLOR_WHITE 0xFFFF
#define COLOR_TUM_BLUE 0x3396 // TUM Blue
#define COLOR_GREY 0x6CAF
#define COLOR_MIMED_YELLOW 0xFE60 // MiMed Yellow
#define FONT_COLOR_HEADLINES COLOR_TUM_BLUE // White
#define FONT_COLOR_READY_GREEN 0x07E0 // GREEN
#define FONT_COLOR_READY_RED 0xF800 // RED
#define COLOR_FILL_RECT COLOR_WHITE
// define positions
#define SCREEN_WIDTH 240 // 240
#define SCREEN_HEIGHT 320 // 320

```

```

#define TEXT_SIZE 3

// Text positions and dimensions (based on the selected font's characteristics)
#define CHARACTER_WIDTH (6 * TEXT_SIZE)
#define CHARACTER_HEIGHT (8 * TEXT_SIZE)

// Dynamic starting positions for text based on screen sections (as a fraction of SCREEN_HEIGHT)
#define STARTING_Y_READY (SCREEN_HEIGHT * .05) // down from the top
#define STARTING_Y_HEAD (SCREEN_HEIGHT * 0.12) // down from the top of the screen
#define STARTING_Y_MODES (SCREEN_HEIGHT * 0.25) // down from the top
#define STARTING_Y_VARIABLE (SCREEN_HEIGHT * 0.8) // down from the top

#define STARTING_X 0
#define SCREEN_CLC_MODES_HEIGHT 160
#define SCREEN_CLC_READY_WIDTH 82
#define SCREEN_CLC_READY_HEIGHT 18

// define delays
#define TIME_DELAY_MAIN_LOOP 1
#define TIME_ENCODER_DELAY 500
#define TIME_READY_DELAY 1500
#define TIME_DELAY_INIT 500

// Initialize the display using hardware SPI communication
DFRobot_ST7789_240x320_HW_SPI screen(/*dc=*/TFT_DC, /*cs=*/TFT_CS, /*rst=*/TFT_RST);

// Miscellaneous variables
bool SERIAL_DEBUG = 0; // CHANGE TO 0 in normal mode. Do you want to see the debug messages? 1 = yes, 0 = no
bool EXPERT_MODE = 0; // CHANGE TO 0 in normal mode. Without it the delay finder function is disabled. 1 = yes, 0 = no
int menuItemIndex = 0; // Use this variable to store the current index of the menu
bool currentStateClock; // Store the status of the clock pin (HIGH or LOW)
bool lastEncoderA; // Store the PREVIOUS status of the clock pin (HIGH or LOW)
bool currentENCODER_B;
bool lastEncoderB;
bool buttonState = HIGH; // Current state of the button
bool lastButtonState = 0; // Use this to store if the push button was pressed or not
unsigned int lastEncoderUpdateTime = 0;
unsigned int lastPushDebounceTime = 0; // Last time the input pin was toggled
const unsigned int PUSH_DEBOUNCE_DELAY = 50; // Debounce delay in milliseconds

//
-----
-----
// --- Define menu items
-----
-

```

```

//
-----
-----

// max Values
#define MAX_MENU_ITEMS 7 // Number of menu items
String menuOptionsShort[MAX_MENU_ITEMS] = {
    "All functions",
    "Push",
    "Catch",
    "Catch after 2",
    "Catch after 3",
    "Resonance",
    "Resonance      after 2"};

// once a function is selected, the long description is displayed instead of the short one, filling the
lower side of the screen
String menuOptionsLong[MAX_MENU_ITEMS] = {
    "Cycle Through all Functions      Sequentially",
    "Push the      Pendulum Ball",
    "Push and      Catch after First Impulse",
    "Push and      Catch after      Second Impulse",
    "Push and      Catch after      Third Impulse",
    "Resonance:  Propels the Ball upon Immediate  Impulse Detection",
    "Resonance 2: Propels the Ball upon Second    Impulse Detection"};
// Resonance and catch after third

// for 64V
// Timer delay for each menu item in microseconds
volatile unsigned int timerDelays[MAX_MENU_ITEMS] = {
    0, // Delay for "All functions"
    0, // Delay for "Push"
    14, // Delay for "Catch"
    5, // Delay for "Catch 2"
    5, // Delay for "Catch 3"
    0, // Delay for "Resonance"
    0 // Delay for "Resonance 2"
};

String headline = "Select action:";

String readyString[2] = {
    "execute",
    "ready"};
unsigned int readyState = 1;
unsigned int lastButtonPushed = 0;

// Piezo variables
volatile bool ballDetected = false;

// LED impulse variables

```

```

long ledOnTime = 0;

//
-----
-----
//          ---          Define          functions
-----
--
//
-----
-----

// forward declarations for platformIO
bool checkButtonPressed(); // Button handling
int getEncoderPosition();
void inputMethods();
int changeMenuItem(int iChange);
void displayClearSection(int x, int y, int width, int height); // Display functions
void displayMenuText();
void displayReadyString();
void displayVoltage();
void chargePiezo(); // Piezofunctions
void waitForBallImpacts(int numberOfDetections);
void interruptBallDetected();
void ledImpulse();
void performPiezoAction(int piezoSelect);
void executePushAction();
void executePushAndCatchAction(int bounceCount);
void executeResonanceAction(int bounceCount);
void handleSerialInput();
void dischargePiezo(uint32_t timeToDischargePiezoMicroseconds);
void chargeBootstrapCapacitor(uint32_t timeToChargeCapacitorMicroseconds);
float readVoltagePiezo(); // voltage
void setPiezoActuatorVoltage(float voltage);
void setupI2CDAC(); // Setup helper functions
void setupGatedriver();
void setupEncoder();
void setupInterrupts();
void setupScreen();

//
-----
-----
// -----Functions-----
void enablePiezoInterrupt() // Enable the interrupts for the ball detection pin

{
    attachInterrupt(digitalPinToInterrupt(HIT_IMPULSE), interruptBallDetected, RISING);
}

```

```

// void disablePiezoInterrupt() // Disable the interrupts for the ball detection pin → rather not use this
// function because it takes too long to reenale the interrupt
// {
//   // Disable the interrupts for the ball detection pin
//   detachInterrupt(digitalPinToInterrupt(HIT_IMPULSE));
// }

// unsigned long lastDetectionTime = 0;           // Tracks the time since the last detection
volatile unsigned int lastImpulseDebounceTime = 0; // Timestamp of the last time the ISR was triggered

// Interrupt Service Routine (ISR) for the ball detected pin
const uint16_t IMPULSE_DEBOUNCE_DELAY_CATCH = 4; // Debounce delay for resonance in milliseconds

void interruptBallDetected()
{
  // Check if enough time has passed since the last detected interrupt (debouncing)
  if ((millis() - lastImpulseDebounceTime) > IMPULSE_DEBOUNCE_DELAY_CATCH)
  {
    ballDetected = true;
    digitalWrite(LED_IMPULSE, HIGH); // Turn on the LED
    ledOnTime = millis();             // Set the time the LED was turned on
    // Update the lastImpulseDebounceTime to the current time
    lastImpulseDebounceTime = millis();
  }
}

void delayAndDoStuff(int timeDelayMilliseconds)
{
  unsigned int currentTime = millis();
  while (millis() - currentTime < timeDelayMilliseconds)
  {
    ledImpulse();
  }
}

bool longPressActive = false;
/**
 * Checks if the button is pressed and returns a value indicating the button state, with debouncing.
 *
 * @return true if the button is pressed, false otherwise.
 */
int LONG_PRESS_DELAY = 1000;
bool checkButtonPressed()
{
  if (PUSH_BUTTON.longClick())
  {
    int commandToExecute = menuItemIndex + 1;
    if (commandToExecute > 2 && commandToExecute < 6)
    {
      if (PUSH_BUTTON.getLongClickTime() > LONG_PRESS_DELAY) // press button for 1s to start delay function

```

```

{
    longPressActive = true;
    return false;
}
else
{
    longPressActive = false;
    return true;
}
}
else
{
    longPressActive = false;
    return true;
}
}
else
{
    return false;
}
}

// variables for the encoder
const int ENCODER_COUNTER_CLOCKWISE = -1; // ⤿ Counter clockwise rotation
const int ENCODER_NO_CHANGE = 0;
const int ENCODER_CLOCKWISE = 1; // ⤵ Clockwise rotation

// Function to check if encoder has changed
int getEncoderPosition()
{
    int changeValue = ENCODER_NO_CHANGE; // Default to no change
    unsigned long currentEncoderTime = millis();

    // Read the current state of the encoder pins
    int currentEncoderA = digitalRead(ENCODER_A);
    int currentEncoderB = digitalRead(ENCODER_B);

    // Check if there has been any change in the state of the encoder pins
    if (lastEncoderA != currentEncoderA || lastEncoderB != currentEncoderB)
    {
        if (currentEncoderTime > (lastEncoderUpdateTime + TIME_ENCODER_DELAY))
        {
            // Check the transition of state to determine direction
            if (((currentEncoderA == 1 && currentEncoderB == 0) && (lastEncoderA == 0 && lastEncoderB == 0)) ||
                ((currentEncoderA == 1 && currentEncoderB == 1) && (lastEncoderA == 1 && lastEncoderB == 0)) ||
                ((currentEncoderA == 0 && currentEncoderB == 1) && (lastEncoderA == 1 && lastEncoderB == 1)) ||
                ((currentEncoderA == 0 && currentEncoderB == 0) && (lastEncoderA == 0 && lastEncoderB == 1)))
            {
                changeValue = ENCODER_COUNTER_CLOCKWISE; // ⤿ Counter clockwise rotation
            }
        }
    }
}

```

```

    else if (((currentEncoderA == 0 && currentEncoderB == 1) && (lastEncoderA == 0 && lastEncoderB == 0)) ||
((currentEncoderA == 1 && currentEncoderB == 1) && (lastEncoderA == 0 && lastEncoderB == 1)) || ((currentEncoderA == 1 && currentEncoderB == 0) && (lastEncoderA == 1 && lastEncoderB == 1)) || ((currentEncoderA == 0 && currentEncoderB == 0) && (lastEncoderA == 1 && lastEncoderB == 0)))
    {
        changeValue = ENCODER_CLOCKWISE; // ⤵ Clockwise rotation
    }

    // If a valid item change has occurred, update the time
    lastEncoderUpdateTime = millis();
}
lastEncoderA = currentEncoderA;
lastEncoderB = currentEncoderB;
}

return changeValue; // Return the change in menu item (-1, 0, 1)
}

int changeMenuItem(int step)
{
    menuItemIndex += step;
    if (menuItemIndex >= MAX_MENU_ITEMS)
    {
        menuItemIndex = MAX_MENU_ITEMS - 1;
    }
    else if (menuItemIndex < 0)
    {
        menuItemIndex = 0;
    }
    return menuItemIndex; // Return the updated menu item index.
}

void inputMethods()
{
    // Check for encoder rotation
    int encoderChange = getEncoderPosition();
    if (encoderChange != ENCODER_NO_CHANGE)
    {
        changeMenuItem(encoderChange);
        displayMenuText();
    }

    // Check for button press
    int newButtonPushed = millis();
    if (newButtonPushed > lastButtonPushed + TIME_READY_DELAY && readyState != 1)
    {
        readyState = 1;
        displayReadyString();
    }
    else if (checkButtonPressed() == 1)

```

```

{
  if (newButtonPushed > lastButtonPushed + TIME_READY_DELAY)
  {
    lastButtonPushed = newButtonPushed;
    readyState = 0;
    displayReadyString();
  }
}

// Execute the selected action
if (readyState == 0) // Condition met when execute button clicked
{
  menuSelectAction();
}

if (longPressActive && EXPERT_MODE)
{
  delayTimeHelper();
}
}

void menuSelectAction()
{
  int commandToExecute = menuItemIndex + 1; // Menu item index matches performPiezoAction case
  if (SERIAL_DEBUG)
  {
    Serial.print("Piezo command selected: ");
    Serial.println(menuOptionsShort[menuItemIndex]); // Print the string
  }
  delayAndDoStuff(500); // Delay so that the push button does not cause the ball to not be in
the right position
  ballDetected = false; // if the push button caused the interrupt to be triggered, falsify it
again
  performPiezoAction(commandToExecute); // menuItemIndex starts at 0, and case 1 is "Push"
  readyState = 1; // Reset the ready flag to prevent repeated command execution without
button presses
  displayMenuText(); // Update the display to show the Menu text
  displayReadyString(); // Update the display to show the change in state
  displayVoltage();
}

// Modular helper function implementations
void displayClearSection(int x, int y, int width, int height)
{
  screen.fillRect(x, y, width, height, COLOR_FILL_RECT); // Clear the designated screen section
}

void displayMenuText()
{
  // Calculate the number of items to display before and after the selected item.
  const int itemsBeforeAfter = 2;

```

```

// Clear the menu area before displaying the new text
displayClearSection(STARTING_X, STARTING_Y_MODES, screen.width(), SCREEN_CLC_MODES_HEIGHT);

// Display 5 menu options at a time: 2 before, 1 current, and 2 after
for (int offset = -itemsBeforeAfter; offset <= itemsBeforeAfter; ++offset)
{
    int menuIndex = menuItemIndex + offset;

    // Skip if the menuIndex is out of the array bounds
    if (menuIndex < 0 || menuIndex >= MAX_MENU_ITEMS)
    {
        continue;
    }

    // Calculate the Y position for each item; adjust it based on your screen's layout
    int yPos = STARTING_Y_MODES + (CHARACTER_HEIGHT + 2) * (offset + itemsBeforeAfter);

    // Set cursor position
    screen.setCursor(STARTING_X, yPos);

    // Set the text color: highlight the current menu item, otherwise use the default color
    uint16_t color = (offset == 0) ? COLOR_MIMED_YELLOW : FONT_COLOR_HEADLINES;
    screen.setTextColor(color);

    // Set the text size for uniformity
    screen.setTextSize(TEXT_SIZE);

    // Print the menu index and option
    screen.println(String(menuIndex + 1) + " - " + menuOptionsShort[menuIndex]);
}

ledImpulse();
}

// instead of displayMenuText the full function name is displayed for comprehension
void displayCurrentFunctionDescription()
{
    displayClearSection(STARTING_X, STARTING_Y_MODES, screen.width(), SCREEN_CLC_MODES_HEIGHT);
    screen.setCursor(STARTING_X, STARTING_Y_MODES);
    screen.setTextColor(COLOR_MIMED_YELLOW);
    screen.setTextSize(TEXT_SIZE);
    screen.println(String(menuItemIndex + 1) + " - " + menuOptionsLong[menuItemIndex]);
    ledImpulse();
}

// instead of displayMenuText the full function name is displayed for comprehension
void displayDelayHelperText()
{
    displayClearSection(STARTING_X, 200, screen.width(), 40);
    screen.setCursor(STARTING_X, 220);

```

```

screen.setTextColor(COLOR_MIMED_YELLOW);
screen.setTextSize(TEXT_SIZE);
screen.println("Set delay time");
ledImpulse();
}

void displayReadyString()
{
    ledImpulse();
    // Calculate the dimensions needed for the text based on text length and size
    int textLength = readyString[readyState].length();
    int textWidth = CHARACTER_WIDTH * textLength * 1.4; // Width of entire text string
    int textHeight = CHARACTER_HEIGHT; // Height of text string

    // Calculate starting X so the text is horizontally centered
    int textX = STARTING_X;
    int textY = STARTING_Y_READY - (textHeight / 2); // Adjust Y so the text is vertically centered

    // Clear the rectangle area where the ready string will be displayed
    displayClearSection(textX, textY, textWidth, textHeight);

    // Set the cursor to the calculated starting position for the ready string
    screen.setCursor(textX, textY);
    // Set the text color depending on the state of readyState
    if (readyState == 0)
    {
        screen.setTextColor(FONT_COLOR_READY_RED);
    }
    else
    {
        screen.setTextColor(FONT_COLOR_READY_GREEN);
    }
    // Set the text size and print the string
    screen.setTextSize(TEXT_SIZE);
    screen.println(readyString[readyState]);
    if (readyState == 0)
    {
        if (SERIAL_DEBUG)
        {
            displayTimeMicroseconds();
        }
    }
    else
    {
        displayVoltage();
    }
}

void displayHeader(String headerText)
{

```

```

displayClearSection(STARTING_X, STARTING_Y_HEAD, screen.width(), CHARACTER_HEIGHT);
screen.setCursor(STARTING_X, STARTING_Y_HEAD);
screen.setTextColor(COLOR_BLACK);
screen.setTextSize(TEXT_SIZE);
screen.println(headerText);
}

void displayVoltage()
{
    currentVoltage = (int)readVoltagePiezo();
    int textWidth = 300; // Width of entire text string
    int textX = 245;
    int textY = STARTING_Y_READY - (CHARACTER_HEIGHT / 2);
    screen.setCursor(textX, STARTING_Y_READY - (CHARACTER_HEIGHT / 2));
    screen.setTextColor(COLOR_TUM_BLUE);
    screen.setTextSize(TEXT_SIZE);
    // Clear the rectangle area where the ready string will be displayed
    displayClearSection(140, textY, textWidth, CHARACTER_HEIGHT);
    screen.print(currentVoltage, 0);
    screen.print("V");
}

void displayTimeMicroseconds()
{
    int textX = 140;
    int textY = STARTING_Y_READY - (CHARACTER_HEIGHT / 2);
    displayClearSection(textX, textY, 200, CHARACTER_HEIGHT);
    int charHeight = CHARACTER_HEIGHT;
    screen.setTextSize(TEXT_SIZE - 1);
    screen.setCursor(textX, STARTING_Y_READY - (charHeight / 2));
    screen.setTextColor(COLOR_TUM_BLUE);
    int timeDelay = timerDelays[menuItemIndex];
    screen.print("delay: ");
    screen.print(timeDelay);
    screen.print("us");
}

// --- Piezo functions
-----

/*in a bootstrap configuration, the low side MOSFET (Q2) is turned on to discharge
the piezo and charge the bootstrap capacitor, so that the piezo can be charged
again. The bootstrap capacitor loses its charge over time, so it needs to be
recharged before switching the high side MOSFET (Q1)*/

void chargeBootstrapCapacitorFast()
{
    delayMicroseconds(1); // → never comment this, this is shoot through delay time
    digitalWrite(GATE_DRIVE_LIN, HIGH);
    digitalWrite(GATE_DRIVE_LIN, LOW);
}

```

```

}
void chargeBootstrapCapacitor(uint32_t timeToChargeCapacitorMicroseconds)
{
    delayMicroseconds(1); // → never comment this, this is shoot through delay time
    digitalWrite(GATE_DRIVE_LIN, HIGH);
    // delayMicroseconds(timeToChargeCapacitorMicroseconds);
    delayMicroseconds(30);
    digitalWrite(GATE_DRIVE_LIN, LOW);
    delayMicroseconds(10);
}

/**
 * @brief Turns the piezo actuator instantly on, using the gatedriver.
 */
void chargePiezo()
{
    digitalWrite(GATE_DRIVE_HIN, HIGH); // SHOOT: Turn on higher side MOSFET (Q1)
    delayMicroseconds(100);             // todo: optimize time
    digitalWrite(GATE_DRIVE_HIN, LOW);  // Turn off higher side MOSFET (Q1)
    delay(25);                          // now: 50ms todo: how long does it take to charge the capacitor? + 10%
}

unsigned int DISCHARGE_PIEZO_TIME = 30; // this time needs to be adjusted according to the piezo
void dischargePiezo(uint32_t timeToDischargePiezoMicroseconds)
{
    delayMicroseconds(1); // → never comment this, this is shoot through delay time
    delayMicroseconds(timeToDischargePiezoMicroseconds);
    digitalWrite(GATE_DRIVE_LIN, HIGH);
    delayMicroseconds(DISCHARGE_PIEZO_TIME);
    digitalWrite(GATE_DRIVE_LIN, LOW);
}

volatile int NUMBER_OF_PIEZO_REPETITIONS = 1; // Number of times a selected function in the menu is repeated
int MAXIMUM_TIME_WAITING_FOR_BALL = 1500000; // microseconds → 1.5 seconds
int piezoOnTime;

void waitForBallImpacts(int numberOfDetections)
{
    unsigned int startTime = micros();
    ballDetected = false; // Reset the ballDetected flag
    int numberBouncesDetected = 0;
    while (numberBouncesDetected < numberOfDetections)
    {
        // Timeout check
        if (micros() - startTime > MAXIMUM_TIME_WAITING_FOR_BALL)
        {
            break; // Exit the loop
        }
    }
    if (ballDetected)

```

```

{
    numberBouncesDetected++;
    ballDetected = false; // Reset ballDetected for the next detection
}
}
}

int LED_ON_TIME = 150;
// Function to handle the LED impulse
void ledImpulse()
{
    if (digitalRead(LED_IMPULSE))
    {
        // Check if time passed since the LED was turned on
        if (millis() - ledOnTime > LED_ON_TIME)
        {
            digitalWrite(LED_IMPULSE, LOW); // Turn off the LED
        }
    }
}

unsigned int CHARGE_BOOTSTRAP_CAP_TIME_MICROSECONDS = 300;
void executePushAction()
{
    for (int repetitionIndex = 0; repetitionIndex < NUMBER_OF_PIEZO_REPETITIONS; repetitionIndex++)
    {

        chargeBootstrapCapacitor(CHARGE_BOOTSTRAP_CAP_TIME_MICROSECONDS);
        chargePiezo();
        delayAndDoStuff(100);
        dischargePiezo(0);
        ledImpulse();
        delayAndDoStuff(500);
        ledImpulse();
    }
}

void executePushAndCatchAction(int bounceCount)
{
    int timeToDischargePiezoMicroseconds = timerDelays[bounceCount + 1];
    for (int repetitionIndex = 0; repetitionIndex < NUMBER_OF_PIEZO_REPETITIONS; repetitionIndex++)
    {
        delayAndDoStuff(500);
        chargeBootstrapCapacitor(CHARGE_BOOTSTRAP_CAP_TIME_MICROSECONDS);
        chargePiezo(); // Shoot the ball away
        waitForBallImpacts(bounceCount);
        dischargePiezo(timeToDischargePiezoMicroseconds); // Catch the ball bounce-free
        delayAndDoStuff(800);
    }
}

```

```

int NUMBER_OF_BALL_REPETITIONS_RESONANCE = 8;
int chargeBootstrapCapacitorTime = 12;
void executeResonanceAction(int bounceCount)
{
    delayAndDoStuff(500);
    for (int repetitionIndex = 0; repetitionIndex < NUMBER_OF_BALL_REPETITIONS_RESONANCE; repetitionIndex++)
    {
        chargeBootstrapCapacitor(chargeBootstrapCapacitorTime);
        chargePiezo(); // Shoot the ball away
        waitForBallImpacts(bounceCount);
        dischargePiezo(0); // Shoot the ball again, this time higher then before
        chargeBootstrapCapacitorTime += 1; // Increase the charge time for the bootstrap capacitor so that the
        ball flies higher each following bounce
    }
}

void executeAllPiezoActions()
{
    // Perform all actions in sequence
    readyState = 0; // Reset the ready flag to prevent repeated command execution without button presses
    displayReadyString(); // Update the display to show the change in state
    for (int currentMenuIndex = 2; currentMenuIndex < MAX_MENU_ITEMS + 1; currentMenuIndex++)
    {
        menuItemIndex = currentMenuIndex - 1;
        if (SERIAL_DEBUG)
        {
            displayTimeMicroseconds();
        }
        performPiezoAction(currentMenuIndex); // menuItemIndex starts at 0, and case 1 is "Push"
        delayAndDoStuff(800);
    }
    menuItemIndex = 0;
    readyState = 1;
    displayReadyString(); // Update the display to show the "ready" state
    displayMenuText(); // Update the display to show the first menu item
}

void performPiezoAction(int piezoSelect)
{
    displayCurrentFunctionDescription(); // Update the display to show the long description of the selected
    function
    ledImpulse();
    // Log the selected command if serial debug is enabled
    delayAndDoStuff(700); // Delay to show description
    // Execute action based on the selected command
    switch (piezoSelect)
    {
        case 1: // "All functions" - Perform all actions in sequence
            executeAllPiezoActions();
    }
}

```

```

    break;

    case 2: // "Push"
        executePushAction();
        break;

    case 3: // "Push & Catch"
        executePushAndCatchAction(1); // Catch after 1 pulse
        break;

    case 4: // "Catch after 2nd pulse"
        executePushAndCatchAction(2); // Catch after 2 pulses
        break;

    case 5: // "Catch after 3rd pulse"
        executePushAndCatchAction(3); // "Catch after 3rd pulse"
        break;

    case 6: // "Resonance (Impulse on detection)"
        executeResonanceAction(1); // Resonance after 1 pulse
        break;

    case 7: // "Resonance after 2nd pulse"
        executeResonanceAction(2); // Resonance after 2 pulses
        break;

    default:
        // Optional default case to handle undefined actions
        break;
}
}

/**
 * @brief Checks the voltage of the piezo sensor.
 *
 * This function reads the analog value from the V_MEASURE pin and converts it to voltage.
 * The voltage is calculated using the formula: current_voltage = analogRead(V_MEASURE) / 4095 * V_DD *
 * 46.454.
 *
 * @return The voltage of the piezo sensor.
 */

float readVoltagePiezo()
{
    int current_voltage = analogRead(V_MEASURE);
    delayAndDoStuff(10);
    current_voltage *= 0.0374; // Voltage divider equals to: analogRead(V_MEASURE) * (2200 + 100000) / 2200
    return current_voltage;
}

```

```

/**
 * Sets the voltage for the piezo.
 *
 * @param voltage The desired voltage for the piezo.
 */
void setPiezoActuatorVoltage(float voltage)
{
    if (voltage > 100) // max voltage
    {
        voltage = 100;
    }
    else if (voltage < 50) // min voltage
    {
        voltage = 50;
    }
    float slope = 21.25; // 85/4 = 21.25; 85 V output per 4 V input
    float V_send_i2c = (voltage - 50) / slope;
    V_send_i2c = V_send_i2c / .71304; // Voltage divider equals to 82e3 / (82e3 + 33e3)
    // Serial.println(V_send_i2c);
    // Set the voltage to the piezo V_P
    dac.writedac(V_send_i2c, VDD);
    delay(20); // Small delay to allow DAC to settle
}

unsigned long DELAY_HELPER_INTERVAL_TIMER = 1000; // Desired interval in which a new value can be set
// Function to help to find the optimal delay time for executePushAndCatchAction()
void delayTimeHelper()
{
    readyState = 1; // Reset the ready flag
    displayReadyString(); // Update the display to show state change
    int encoderChange = 0; // Variable to store the encoder change value
    String delayHelpHeadline = "Turn to set delay";
    String delayIncreasingHeadline = "Delay Up";
    String delayDecreasingHeadline = "Delay Down";
    displayHeader(delayHelpHeadline);
    int commandToExecute = menuItemIndex + 1; // Menu item index matches performPiezoAction case
    int lastEncoderPosition = 0;
    int currentEncoderPosition; // Function to get current encoder position
    delayAndDoStuff(500); // Delay to prevent accidental rotation of encoder
    while (longPressActive)
    {
        readyState = 0;
        displayReadyString(); // Update the display to show the change in state
        performPiezoAction(commandToExecute);
        readyState = 1; // Reset the ready flag to prevent repeated command execution without button presses
        displayReadyString(); // Update the display to show the change in state
        displayTimeMicroseconds();
        // Use a flag to determine if the interval has finished
        bool intervalHasFinished = false;
        unsigned long previousTimer = millis(); // Start time for the interval

```

```

int lastTimerDelay;
int currentTimerDelay;
lastTimerDelay = timerDelays[menuItemIndex];
// Loop until the encoder is changed or the interval has elapsed, or the button is pressed
while (!intervalHasFinished)
{

    unsigned long currentTimer = millis();
    // Check for encoder changes
    currentEncoderPosition = getEncoderPosition();
    if (currentEncoderPosition != ENCODER_NO_CHANGE && timerDelays[menuItemIndex] >= 0)
    {
        timerDelays[menuItemIndex] += currentEncoderPosition; // Adjust the delay time
        displayTimeMicroseconds(); // Update the display to show the new delay time
    }
    if (lastEncoderPosition != currentEncoderPosition && currentEncoderPosition != ENCODER_NO_CHANGE) // If
change happened
    {
        lastEncoderPosition = currentEncoderPosition; // Update the last accounted position
    }
    else if (currentEncoderPosition == ENCODER_NO_CHANGE)
    {
        currentEncoderPosition = lastEncoderPosition; // If no change, set the current position to the last
position
    }

    // Exit the loop
    if (PUSH_BUTTON.click() == 1)
    {
        longPressActive = false;
        break;
    }
    if (currentTimer - previousTimer > DELAY_HELPER_INTERVAL_TIMER)
    {
        intervalHasFinished = true;
    }
} // End of inner while loop for adjusting delay time
// lastTimerDelay != timerDelays[menuItemIndex]
if (lastTimerDelay != timerDelays[menuItemIndex] + 1 || lastTimerDelay != timerDelays[menuItemIndex] -
1)
{
    if (currentEncoderPosition > 0) // Increase the value
    {
        displayHeader(delayIncreasingHeadline);
        timerDelays[menuItemIndex] += 1;
    }
    else // Decrease the value
    {
        displayHeader(delayDecreasingHeadline);
        if (timerDelays[menuItemIndex] >= 1)

```

```

    {
        timerDelays[menuItemIndex] -= 1;
    }
    else
    {
        {
            timerDelays[menuItemIndex] = 0;
            longPressActive = false;
            break;
        }
    }
}
else
{
    displayHeader(delayHelpHeadline); // Update the display to show the help text
}
} // End of the main while loop that continually allows for delay time adjustments
readyState = 1;          // "ready" state
displayReadyString();    // Update the display to show "ready" state
displayMenuText();       // Update the display for the menu
displayHeader(headline); // Print the header at the top
}

void setupPins()
{
    PUSH_BUTTON.setMinLongClickTime(100); // Set the minimum time for a long click in milliseconds
    pinMode(LED_BUILTIN, OUTPUT);          // Initialize LED's as output
    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, HIGH);
    pinMode(LED_IMPULSE, OUTPUT);
    pinMode(LED_POWER, OUTPUT);
    digitalWrite(LED_POWER, HIGH);
    pinMode(HIT_IMPULSE, INPUT_PULLUP);
    enablePiezoInterrupt(); // Enable interrupts for the ball detection pin
    pinMode(V_MEASURE, INPUT); // Initialize voltage measurement pin
    analogReadResolution(12); // 12 BITS → 4096 possible values
}

void setupGatedriver()
{
    pinMode(GATE_DRIVE_HIN, OUTPUT);
    pinMode(GATE_DRIVE_LIN, OUTPUT);
}

void setupi2C_DAC()
{
    gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
    gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
    dac.begin(); // Initialize the DAC
    setPiezoActuatorVoltage(100);
}

```

```

void setupEncoder()
{
    pinMode(ENCODER_A, INPUT_PULLUP);
    pinMode(ENCODER_B, INPUT_PULLUP);
    currentStateClock = digitalRead(ENCODER_A);
    lastEncoderA = currentStateClock;
    currentENCODER_B = digitalRead(ENCODER_B);
    lastEncoderB = currentENCODER_B;
}

void setupScreen()
{
    while (1) // Wait until the external 12V supply is connected, so as to initialize the display. The display
    is not powered from the microcontroller in this version.
    {
        if (readVoltagePiezo() > 10)
        {
            break;
        }
    }
    digitalWrite(TFT_RST, LOW); // Reset the display
    screen.begin();
    screen.setRotation(1);
    screen.fillScreen(COLOR_FILL_RECT);
    displayMenuText(); // show mode text
    displayReadyString(); // show ready string
    displayHeader(headline); // Print the header at the top
}

void setupSerial()
{
    Serial.print(F("CPU Frequency = "));
    Serial.print(F_CPU / 1000000);
    Serial.println(F(" MHz"));
    // inputString.reserve(200); // Reserve 200 bytes for the inputString to reduce dynamic allocation
    int startTimerSerial = millis();
    Serial.print(startTimerSerial);
    Serial.begin(1000000);
    delay(300);
}

void setup()
{
    setupEncoder();
    setupGatedriver();
    // setupi2C_DAC(); // doesn't work in PCB version 1.0
    setupPins();
    setupSerial();
    // executePushAction();
}

```

```

setupScreen();
delay(TIME_DELAY_INIT);
}

void loop()
{
  inputMethods();
  ledImpulse();
  // testTimes();
}

```

F Drawings

An updated version of all CAD files and drawings will be available at: <https://github.com/julius-gun/piezopendulum>

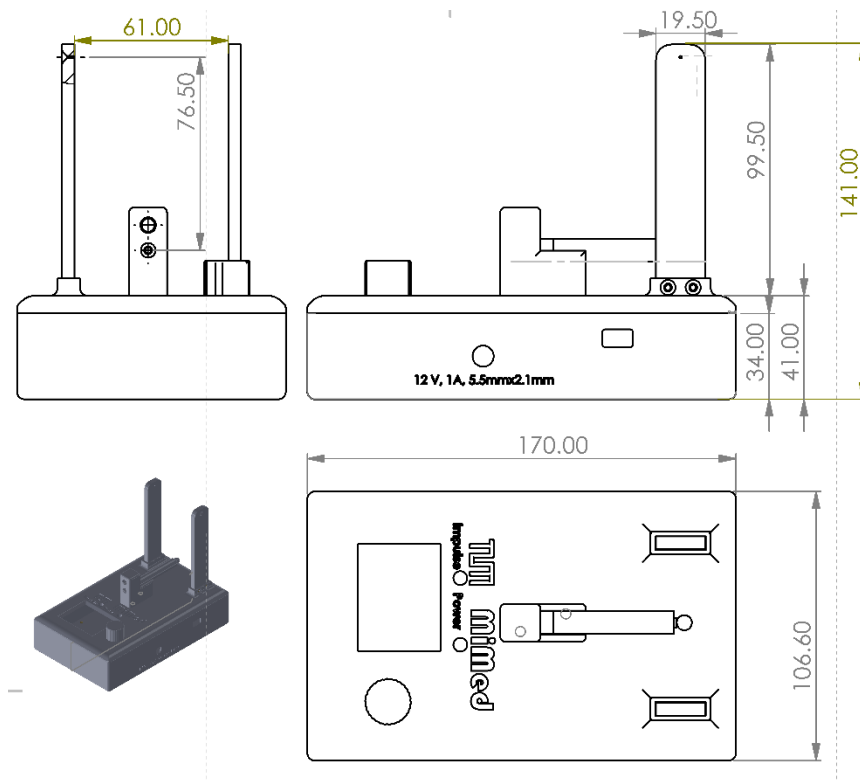


Figure 63 Technical drawing of piezo pendulum (not for scale)

G Matlab Code

Code for 2.6.2.2.1

```
C_piezo = 1e-6; %Capacity of piezo
C_buffer = 30e-6; %Capacity of buffer capacitors C10 and C11
V_high_max_wanted = 100;
Q1_1 = V_high_max_wanted * C_buffer; %Charge when C10 and C11 are loaded
Q2_2 = C_piezo/(C_buffer + C_piezo) * Q1_1; %Charge C10 and C11 are unloaded
V2 = Q2_2 / C_piezo
```

H Oscilloscope Readings

The stability of the voltage within the Recom R12-100B power supply presented challenges, as evidenced by oscilloscope imagery. Figure 64 illustrates output inconsistencies at test points 11 and 12 when the I2C digital-to-analog converter (DAC) was set to 3.3V (see 2.6.2.2.1 for the location of the testpoints).

Yellow is the output of the voltage divider and is measured at test point 11 (TP11).

Blue is V_p and measured at the test point 12 (TP12).

Setting the DAC to 3.3V caused an output voltage (V_p) of 100V, but introduced much noise indicated by the blue trace. Figure 65 documents variations in voltage and frequency at different DAC settings and physical connections, with peeping noises correlating with unstable VADJ frequencies.

In Figure 66 a), the DAC output voltage setting is adjusted to 0V. The yellow trace (TP11) displays a V_p (test point voltage) of 64V; meanwhile, the blue trace monitors the I2C DAC VOUT preceding R19, showing the DAC output configured to a 0V level.

The integration of 330 (instead of 33k) Ohm and 820 (instead of 82k) Ohm resistors for R19 and R21, respectively, resulted in the initial control of V_p . However, these modifications led to the early breakdown of the Digital-to-Analog Converter (DAC) on two occasions. Further investigation revealed that the Raspberry Pi Pico's analog reference voltage, used as the DAC's power supply, may have been the cause of the failures due to its incompatibility with the power requirements of the device, resulting in observed disturbances. The decision to utilize the analog reference voltage from the Raspberry Pi Pico for the DAC was based on its accessibility and the assumption of its reliability as a power source. However, the resulting interference and failure of the DAC led to the circuit being revised.

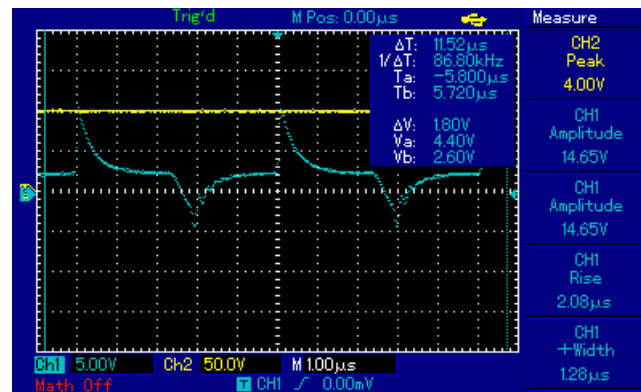


Figure 64: Oscilloscope Readings - Yellow (TP11) and Blue (TP12). The DAC is set at 3.3V, yielding a Vp of 100V. VADJ displays a sawtooth waveform fluctuating between -5V and 10V. VADJ frequency remains stable without auditory disturbances

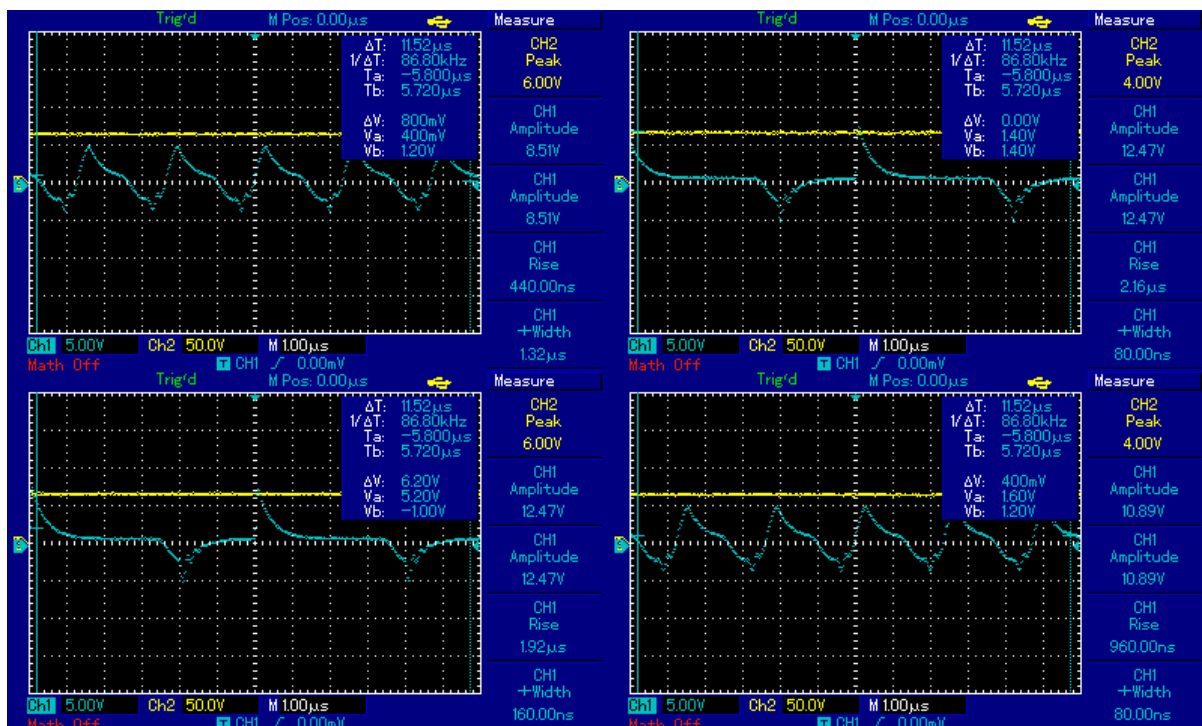
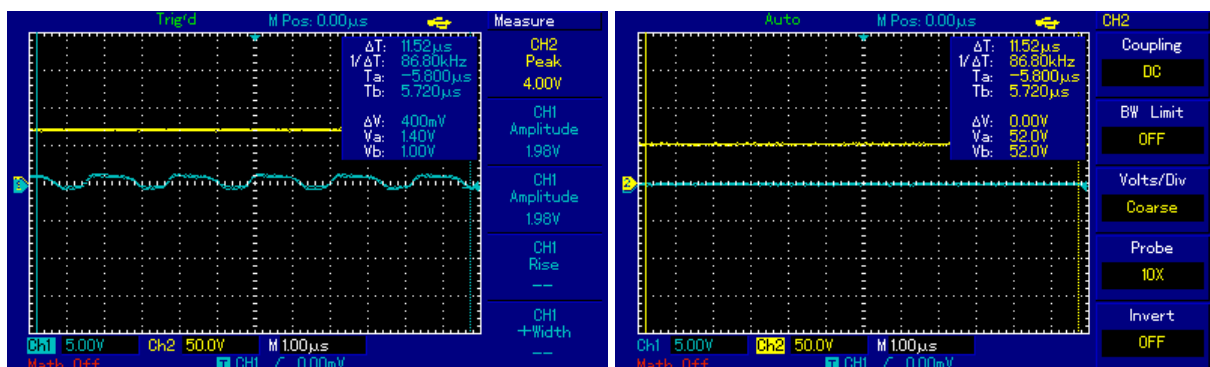


Figure 65 Oscilloscope Reading - Yellow (TP11) shows a Vp of 64V; Blue (TP12) corresponds to the DAC output nominally set to 0V but actually at 0.65Vrms, resulting in a consistent Vp of 64V. Variable VADJ frequency is observed alongside audible high-pitched tones from the Recom DC converter.



a)

b)

Figure 66 Oscilloscope Reading - a) Yellow (TP11) indicates a V_p of 64V; Blue traces the I2C DAC VOUT prior to R19, with the DAC output configured to 0V. b) Yellow (TP11) reflects a V_p of 52V; Blue (TP12) with VADJ connected to ground via a cable, showing noise elimination, ideal performance, and the absence of auditory signals.